

A NOTE ON

ARTIFICIAL INTELLIGENCE

Somnath Mukherjee

Assistant Professor
MCA, RERF

September 17, 2026

Contents

1	Introduction to Artificial Intelligence	13
1.1	Introduction	13
1.2	What is Artificial Intelligence?	13
1.2.1	What is AI?	13
1.2.2	Current Capabilities of Artificial Intelligence	14
1.2.3	AI vs Conventional Programming	15
1.3	History of Artificial Intelligence	15
1.3.1	Theoretical Foundations (1920s–1930s)	15
1.3.2	The Birth of AI (1950s)	16
1.3.3	Early Growth and Knowledge-Based Systems (1960s–1970s)	16
1.3.4	Neural Networks, Expert Systems, and Renewed Progress (1980s–1990s)	17
1.3.5	The Deep Learning Era (2010s)	17
1.3.6	The Generative AI and Agentic Era (2022–2026)	18
1.4	Definitions of Artificial Intelligence	19
1.4.1	John McCarthy (1956)	19
1.4.2	Elaine Rich (1983)	19
1.4.3	Russell & Norvig (2021)	19
1.5	Approaches to Artificial Intelligence	19
1.5.1	Cognitive Science Approach	19
1.5.2	Turing Test Approach	20
1.5.3	Weak AI (Narrow AI)	20
1.5.4	Strong AI (Artificial General Intelligence)	21
1.5.5	Super AI	21
1.5.6	Applied AI	21
1.6	Problems of Artificial Intelligence	21
1.6.1	Knowledge Representation	22
1.6.2	Reasoning	22
1.6.3	Learning	23
1.6.4	Planning	23
1.6.5	Natural Language Understanding	23
1.6.6	Computer Vision	23
1.6.7	Robotics	23
1.7	AI Techniques	24

1.7.1	Search	24
1.7.2	Knowledge Representation	24
1.7.3	Inference	25
1.7.4	Learning	25
1.7.5	Planning	25
1.8	Tic-Tac-Toe Problem	25
1.8.1	Introduction	25
1.8.2	State Space	25
1.8.3	Game Tree (Basic Idea)	26
1.8.4	Why it is an AI Problem	26
1.9	Chapter Summary	27
1.10	Key Points to Remember	28
1.11	Questions	29
1.11.1	Short Questions	29
1.11.2	Long Questions	31
2	Problem Space and Search	33
2.1	Introduction	33
2.2	Defining a Problem as a State Space Search	34
2.2.1	Problem Definition	34
2.2.2	Production Systems	35
2.2.3	Characteristics of Production Systems	35
2.3	Problem Characteristics	36
2.3.1	Is the Problem Decomposable?	36
2.3.2	Can Solution Steps be Ignored or Undone?	36
2.3.3	Is the Universe Predictable?	37
2.3.4	Is a Good Solution Absolute or Relative?	37
2.3.5	Is the Solution a State or a Path?	37
2.3.6	What is the Role of Knowledge?	37
2.3.7	Does the Task Require Interaction with a Person?	37
2.4	Problem Space	38
2.4.1	States, Operators, and Goal Test	38
2.4.2	Representing a Problem Space (State-Space Graph)	38
2.5	Water Jug Problem	40
2.5.1	Problem Statement	40
2.5.2	State Space Representation	40
2.5.3	Production Rules	40
2.5.4	Solution Path	40
2.6	8-Puzzle Problem	41
2.6.1	Problem Statement	41
2.6.2	State Space Representation	41
2.6.3	Why Brute-Force Search is Insufficient	42
2.7	Search Techniques	42
2.7.1	Blind (Uninformed) Search vs. Informed Search	42
2.7.2	Breadth-First Search (BFS)	43

2.7.3	Depth-First Search (DFS)	43
2.7.4	Depth-First Iterative Deepening (DFID)	44
2.7.5	Bidirectional Search	44
2.7.6	Comparison of Uninformed Search Strategies	45
2.8	Issues in the Design of Search Programs	45
2.9	Chapter Summary	47
2.10	Key Points to Remember	47
2.11	Questions	49
2.11.1	Short Questions	49
2.11.2	Long Questions	51
3	Heuristic Search Techniques	54
3.1	Introduction	54
3.2	Heuristic Functions	55
3.2.1	Characteristics of Heuristic Search	55
3.3	Generate-and-Test	56
3.4	Hill Climbing	57
3.4.1	Types of Hill Climbing	58
3.4.2	State Space Diagram for Hill Climbing	59
3.4.3	Regions in the State Space Diagram	59
3.4.4	Problems in Hill Climbing and Remedies	60
3.5	Best-First Search	60
3.6	A* Algorithm	61
3.6.1	Key Terminology	62
3.6.2	The Evaluation Function	62
3.6.3	A* Algorithm — Procedure	62
3.6.4	Advantages and Disadvantages	63
3.7	AO* Algorithm	63
3.7.1	Key Differences from A*	64
3.7.2	AO* Algorithm — Procedure	64
3.7.3	Advantages and Disadvantages	65
3.8	Comparison of Heuristic Search Techniques	65
3.9	Chapter Summary	66
3.10	Key Points to Remember	66
3.11	Questions	68
3.11.1	Short Questions	68
3.11.2	Long Questions	70
4	Game Playing	72
4.1	Introduction	72
4.2	Game Playing as a Search Problem	73
4.2.1	Why Games are Hard for AI	73
4.2.2	Game Trees	73
4.2.3	Two-Player, Zero-Sum Games	74
4.3	The Minimax Search Procedure	74

4.3.1	The Minimax Principle	74
4.3.2	Minimax Algorithm	75
4.3.3	Worked Example	75
4.3.4	Properties of Minimax	76
4.4	Alpha-Beta Pruning	77
4.4.1	Motivation — Reducing the Search Space	77
4.4.2	Alpha and Beta Values	77
4.4.3	Alpha-Beta Algorithm	78
4.4.4	Worked Example	78
4.4.5	Effectiveness of Alpha-Beta Pruning	79
4.5	Iterative Deepening in Game Playing	79
4.5.1	Why Iterative Deepening is Used in Games	79
4.5.2	Iterative Deepening with Alpha-Beta	80
4.6	Additional Refinements	81
4.6.1	Evaluation Functions	81
4.6.2	Horizon Effect	81
4.6.3	Move Ordering	81
4.7	Case Study — Tic-Tac-Toe Revisited	81
4.8	Chapter Summary	82
4.9	Key Points to Remember	83
4.10	Questions	85
4.10.1	Short Questions	85
4.10.2	Long Questions	86
5	Knowledge Representation Using Rules	88
5.1	Introduction to Knowledge Representation and Reasoning	88
5.2	What Needs to Be Represented	89
5.3	Components of Knowledge Representation and Reasoning	89
5.3.1	Knowledge, Propositions, and Attitudes	90
5.3.2	Symbols, Representation, and Reasoning	90
5.4	Why Knowledge Representation and Reasoning in AI?	91
5.4.1	Understanding Behavior	91
5.4.2	The Knowledge Representation Hypothesis	91
5.4.3	Why Knowledge Representation?	92
5.4.4	Why Reasoning?	92
5.5	The Role of Logic	92
5.6	Representing Knowledge in Logic	93
5.6.1	Logic as a Universal Language	93
5.6.2	Propositional Logic	93
5.6.3	Subject and Predicate	93
5.6.4	Predicate Logic	93
5.6.5	Ontologies and Predicates	94
5.6.6	Logic and Other Notations	94
5.6.7	Existential-Conjunctive (EC) Logic	94
5.6.8	Key Takeaways	94

5.7	Varieties of Logic	95
5.7.1	Typed Logic	95
5.7.2	Lambda Calculus	95
5.7.3	Conceptual Graphs	96
5.7.4	Higher-Order Logic (HOL)	96
5.7.5	Comparison of Representations	96
5.8	Names, Types, and Measures	96
5.9	Unity Amidst Diversity: Requirements of a Good Logic	97
5.10	Knowledge Engineering	98
5.10.1	Translating Informal Specifications	98
5.10.2	Principles of Knowledge Representation	99
5.10.3	Approaches to Formalization	99
5.10.4	Ontological Commitments in Practice	99
5.10.5	Reasoning Strategies	100
5.10.6	Comparing the Two Approaches	100
5.11	Frames: Representing Structured Knowledge	100
5.11.1	Structure of a Frame	100
5.11.2	Hierarchies and Inheritance	101
5.11.3	Mapping Frames to Logic	101
5.11.4	Multiple Inheritance	102
5.11.5	Advantages and Limitations of Frames	102
5.12	Rules and Data	103
5.12.1	Historical Systems	103
5.12.2	Plurals and Sets	103
5.12.3	Comparing SQL, Prolog, and CLIPS	104
5.13	Object-Oriented Systems	105
5.13.1	Classes and Encapsulation	105
5.13.2	Mapping Frames to Objects	106
5.13.3	Procedural Integration	106
5.13.4	Logic-Based OOPS and Multiple Views	106
5.14	Natural Language Semantics	107
5.14.1	Natural vs. Artificial Languages	107
5.14.2	Challenges in Language Processing	107
5.14.3	Steps in Language Analysis	107
5.14.4	Resolving Ambiguities	108
5.14.5	Inference and Knowledge Representation	108
5.14.6	Key Takeaways	108
5.15	Levels of Representation	108
5.15.1	Brachman's Levels (1979)	108
5.15.2	Brooks's Competence Levels (1986)	109
5.15.3	Zachman's Design Levels	109
5.15.4	Key Takeaways	110
5.16	Summary	110
6	Logic Programming with Prolog and LISP	111

6.1	Introduction	111
6.2	Prolog: Logic Programming	111
6.2.1	Facts	111
6.2.2	Rules	112
6.2.3	Queries	112
6.2.4	Lists and Recursion	112
6.2.5	Key Features of Prolog	113
6.3	LISP: Symbolic Programming	113
6.3.1	Atoms and Lists	113
6.3.2	Defining Functions	113
6.3.3	Conditionals	113
6.3.4	List Manipulation	114
6.3.5	Recursion	114
6.3.6	Key Features of LISP	114
6.4	Prolog vs. LISP	115
6.5	Summary	115
7	Learning and Machine Learning	116
7.1	Introduction	116
7.2	What is Learning?	117
7.2.1	Example: Learning to Play a Game	117
7.3	Artificial Intelligence and Machine Learning	117
7.4	Why Do We Need Machine Learning?	118
7.4.1	Machine Learning versus Conventional Programming	119
7.5	A Learning System	119
7.5.1	Data	119
7.5.2	Features and Labels	120
7.5.3	Model or Hypothesis	120
7.5.4	Learning Algorithm	120
7.5.5	Performance Measure	120
7.6	Types of Learning	120
7.7	Supervised Learning	120
7.7.1	Basic Supervised Learning Workflow	122
7.7.2	Classification	122
7.7.3	Regression	122
7.8	Learning a Class from Examples	123
7.9	Unsupervised Learning	124
7.9.1	Clustering	124
7.9.2	Association	124
7.9.3	Dimensionality Reduction	125
7.10	Semi-supervised Learning	125
7.11	Reinforcement Learning	125
7.11.1	Example: Robot in a Maze	126
7.12	Training, Validation and Test Data	126
7.13	Generalization	127

7.13.1	Training Error and Generalization Error	127
7.14	Overfitting and Underfitting	127
7.14.1	Overfitting	127
7.14.2	Underfitting	128
7.15	Noise in Data	128
7.16	Model Selection and Inductive Bias	129
7.17	Learning as Optimization	129
7.18	From Data to a Learned Model	129
7.19	Applications of Machine Learning	130
7.20	Learning, Machine Learning and Deep Learning	131
7.21	Chapter Summary	131
7.22	Key Points to Remember	132
7.23	Short Questions	132
7.24	Long Questions	133
8	Neural Networks and Deep Learning	135
8.1	Introduction	135
8.2	What is an Artificial Neural Network?	135
8.3	From Biological Neuron to Artificial Neuron	136
8.4	Basic Terminology of Neural Networks	137
8.5	Artificial Neuron	137
8.5.1	Role of Weights	138
8.5.2	Role of Bias	138
8.6	McCulloch–Pitts (MP) Neuron	138
8.6.1	Example: OR Gate	139
8.7	Single-Layer Feed-Forward Perceptron	139
8.7.1	Decision Boundary of a Perceptron	140
8.8	Perceptron Learning Algorithm	140
8.8.1	Worked Example: OR Gate	141
8.9	Limitations of a Single-Layer Perceptron	141
8.9.1	The XOR Problem	142
8.10	Multilayer Perceptron	142
8.11	How an MLP Computes an Output	143
8.12	Activation Functions	143
8.12.1	Step Function	144
8.12.2	Linear Activation	144
8.12.3	Sigmoid Function	144
8.12.4	Hyperbolic Tangent (Tanh)	144
8.12.5	ReLU	144
8.12.6	Leaky ReLU	145
8.12.7	Softmax	145
8.13	Training a Neural Network	145
8.14	Gradient Descent	146
8.15	Backpropagation	147
8.15.1	Main Steps of Backpropagation	147

8.16	From Neural Networks to Deep Learning	148
8.17	Why Deep Networks Became Practical	148
8.18	Convolutional Neural Networks: Basic Concept	149
8.18.1	Image as a Matrix	149
8.18.2	Convolution Filter or Kernel	149
8.18.3	Stride	149
8.18.4	Padding	150
8.18.5	Feature Map	150
8.18.6	Pooling	150
8.18.7	Why CNNs are Useful for Images	150
8.19	ANN versus CNN	151
8.20	Putting the Ideas Together	151
8.21	Advantages of Neural Networks	152
8.22	Limitations of Neural Networks	152
8.23	Applications of Neural Networks and Deep Learning	153
8.24	Chapter Summary	153
8.25	Key Points to Remember	154
8.26	Short Questions	154
8.27	Long Questions	156
9	Natural Language Processing	157
9.1	Introduction	157
9.2	What is Natural Language Processing?	158
9.2.1	Natural Language Understanding	158
9.2.2	Natural Language Generation	159
9.2.3	Why Human Language is Difficult for Machines	159
9.2.4	A Simple NLP Processing View	160
9.3	Language Models	160
9.3.1	Probability of a Sentence	160
9.3.2	The N-gram Idea	161
9.3.3	Estimating N-gram Probabilities	161
9.3.4	The Zero-Probability Problem	162
9.3.5	Language Model Applications	162
9.4	Text Classification	162
9.4.1	Types of Text Classification	163
9.4.2	Basic Text Classification Pipeline	163
9.4.3	Bag-of-Words Representation	163
9.4.4	TF-IDF Representation	164
9.4.5	Training a Classifier	164
9.4.6	Evaluating Text Classification	164
9.5	Information Retrieval	165
9.5.1	Basic Information Retrieval System	165
9.5.2	The Inverted Index	166
9.5.3	Boolean Retrieval	166
9.5.4	Precision and Recall in Information Retrieval	167

9.6	Information Extraction	168
9.6.1	Named Entity Recognition	168
9.6.2	Relation Extraction	169
9.6.3	Information Extraction Example	169
9.6.4	Information Extraction and Information Retrieval	169
9.7	Putting the Major NLP Tasks Together	170
9.8	Applications of NLP	170
9.9	Limitations and Challenges of NLP	171
9.10	Chapter Summary	171
9.11	Key Points to Remember	172
9.12	Short Questions	172
9.13	Long Questions	173
10	Expert Systems: Foundations and Architecture	175
10.1	Introduction	175
10.2	What is an Expert System?	176
10.2.1	Why Expert Systems Were Important	176
10.3	Expert Systems, AI, and Knowledge-Based Systems	176
10.4	Goals and Characteristics of Expert Systems	177
10.4.1	Major Goals	177
10.4.2	Characteristics	177
10.5	Architecture of an Expert System	178
10.5.1	Knowledge Base	178
10.5.2	Inference Engine	179
10.5.3	Working Memory	179
10.5.4	User Interface	179
10.5.5	Explanation Facility	179
10.5.6	Knowledge Acquisition Tools	179
10.6	Knowledge Representation in Expert Systems	180
10.6.1	Facts	180
10.6.2	Rules	180
10.6.3	Frames	181
10.6.4	Knowledge Representation Forms	181
10.7	Inference Mechanisms	181
10.7.1	Forward Chaining	181
10.7.2	Backward Chaining	182
10.7.3	Forward Chaining vs. Backward Chaining	183
10.8	Handling Uncertainty	183
10.8.1	Sources of Uncertainty	183
10.8.2	Deterministic Reasoning	183
10.8.3	Probabilistic Reasoning	184
10.8.4	Fuzzy Reasoning	184
10.8.5	Confidence in Rules	185
10.9	How an Expert System Works	185
10.10	Knowledge Acquisition and Knowledge Engineering	185

10.10.1 Sources of Knowledge	186
10.10.2 The Knowledge Acquisition Bottleneck	187
10.10.3 Knowledge Validation	187
10.11 Expert System Shells	187
10.11.1 Benefits of Shells	187
10.12 Types of Expert Systems	188
10.13 Historical Examples	188
10.13.1 MYCIN	188
10.13.2 DENDRAL	189
10.13.3 XCON / R1	189
10.13.4 PROSPECTOR	189
10.14 Applications of Expert Systems	189
10.15 Advantages of Expert Systems	190
10.16 Limitations of Expert Systems	190
10.17 Expert Systems and Machine Learning	191
10.18 A Compact Worked Example	192
10.19 Chapter Summary	192
10.20 Key Points to Remember	193
10.21 Short Questions	194
10.22 Long Questions	195
11 Philosophy, Ethics and Future of AI	197
11.1 Introduction	197
11.2 Philosophy of Artificial Intelligence	197
11.2.1 Four Approaches to AI	198
11.2.2 Thinking Humanly	198
11.2.3 Thinking Rationally	199
11.2.4 Acting Humanly	199
11.3 Can Machines Really Think?	199
11.3.1 The Turing Test	199
11.3.2 The Difference Between Performance and Consciousness	200
11.4 Limits of Artificial Intelligence	200
11.4.1 Narrow Intelligence and General Intelligence	200
11.4.2 Common Limitations of Current AI	201
11.5 Why AI Ethics Matters	201
11.6 Ethical Principles for AI	202
11.6.1 Safety	202
11.6.2 Fairness	202
11.6.3 Privacy	202
11.6.4 Transparency and Explainability	202
11.6.5 Accountability and Responsibility	203
11.6.6 Human Rights and Human Values	203
11.6.7 Limiting Harmful Uses	203
11.7 Bias and Fairness in AI	203
11.8 Safety and Robustness of AI	204

11.8.1 A Practical Safety View	204
11.9 Asimov's Laws of Robotics as a Thought Experiment	204
11.10 Machines with Moral Status	204
11.11 Artificial General Intelligence and AI Safety	205
11.12 The Future of Artificial Intelligence	206
11.12.1 AI Components	206
11.12.2 Real-Time AI and Bounded Decision-Making	206
11.13 AI Architectures	207
11.14 From Narrow AI to More General Systems	208
11.15 Generative AI	208
11.16 The Future of AI: Opportunities and Challenges	209
11.17 AI and Employment	209
11.18 Human-AI Collaboration	210
11.19 A Balanced View of the Future	210
11.20 Chapter Summary	211
11.21 Key Points to Remember	211
11.22 Short Questions	212
11.22.1 1. What is the philosophy of AI?	212
11.22.2 2. What is weak AI?	212
11.22.3 3. What is strong AI?	212
11.22.4 4. What are the four approaches to AI?	212
11.22.5 5. What is the Turing Test?	212
11.22.6 6. What is AGI?	212
11.22.7 7. What is Explainable AI?	212
11.22.8 8. Mention four ethical principles of AI.	212
11.22.9 9. What is moral status?	213
11.22.10 10. What is Generative AI?	213
11.23 Long Questions	213
11.24 Discussion Questions	213
11.25 Selected References for This Chapter	214
A Prolog Programs	215
B LISP Programs	216
C Slot and Filler Structures (Supplementary)	217
D University Question Bank	218

Chapter 1

Introduction to Artificial Intelligence

1.1 Introduction

Artificial Intelligence is concerned with the design of intelligence in an artificial device.

The term was coined by McCarthy in 1956.

There are two ideas in the definition.

1. Intelligence
2. Artificial device

Given this scenario different interpretations have been used by different researchers as defining the scope and view of Artificial Intelligence.

1.2 What is Artificial Intelligence?

1.2.1 What is AI?

- There are two possibilities:
 - A system with intelligence is expected to behave as intelligently as a human.
 - A system with intelligence is expected to behave in the best possible manner.
- What type of behavior we are concerned about?
 - Are we looking at the thought process or reasoning ability of the system?
 - Or are we only interested in the final manifestations of the system in terms of its actions?

1.2.2 Current Capabilities of Artificial Intelligence

Modern AI systems have achieved remarkable success in a wide range of specialized domains. With recent advances in Machine Learning, Deep Learning, and Generative AI, intelligent systems are capable of performing tasks that were once considered exclusive to humans. However, current AI systems still lack true human-level intelligence, common-sense reasoning, emotional understanding, and the ability to perform effectively in all situations. Therefore, today's AI is often referred to as *Narrow AI* or *Weak AI*.

The current capabilities of Artificial Intelligence include:

- **Computer Vision:** AI systems can recognize faces, detect and classify objects, analyze medical images, and assist autonomous vehicles in understanding their surroundings.
- **Robotics:** AI enables autonomous robots for manufacturing, warehouse automation, surgical assistance, space exploration, and self-driving vehicles.
- **Natural Language Processing (NLP):** AI systems can perform machine translation, text summarization, sentiment analysis, question answering, document generation, and conversational interactions through intelligent chatbots and virtual assistants.
- **Expert Systems:** AI-based expert systems assist in medical diagnosis, financial planning, fault diagnosis, legal consultation, and decision support within specialized domains.
- **Speech Processing:** AI systems can recognize spoken language, convert speech to text, generate natural-sounding speech, and provide real-time language translation.
- **Planning and Scheduling:** AI is widely used in airline scheduling, logistics, supply chain management, manufacturing, satellite mission planning, and smart transportation systems.
- **Machine Learning:** AI systems can classify images, detect fraud, recommend products, predict customer behavior, and analyze large volumes of data for decision making.
- **Games:** AI systems have achieved superhuman performance in games such as Chess, Go, Poker, Dota 2, and StarCraft II by learning optimal strategies.
- **Generative AI:** Modern AI systems can generate human-like text, images, audio, videos, computer programs, and creative content based on user instructions.
- **Healthcare:** AI assists in disease diagnosis, drug discovery, personalized medicine, medical image analysis, and patient monitoring.
- **Education:** AI supports intelligent tutoring systems, personalized learning, automated assessment, and virtual teaching assistants.

- **Finance and Cybersecurity:** AI is used for fraud detection, credit risk assessment, algorithmic trading, malware detection, intrusion detection, and cybersecurity monitoring.

1.2.3 AI vs Conventional Programming

Artificial Intelligence and Conventional Programming differ in their approach to solving problems. Conventional programming follows explicitly defined rules and algorithms written by the programmer, whereas AI systems learn from data, reason, and make decisions even in uncertain situations.

The major differences between Artificial Intelligence and Conventional Programming are shown in Table 1.1.

Table 1.1: Comparison between Artificial Intelligence and Conventional Programming

Feature	Artificial Intelligence	Conventional Programming
Approach	Learns from data and experience	Follows predefined rules and algorithms
Problem Solving	Can solve complex and uncertain problems	Best suited for well-defined problems
Learning Ability	Can improve through learning and training	Cannot learn unless reprogrammed
Decision Making	Makes intelligent decisions based on knowledge and inference	Makes decisions strictly according to programmed logic
Data Handling	Can process structured and unstructured data	Primarily processes structured data
Adaptability	Adapts to new situations	Limited adaptability
Accuracy	May improve with training but can make probabilistic decisions	Produces deterministic and predictable results
Examples	ChatGPT, Self-driving cars, Face recognition, Recommendation systems	Calculator, Payroll system, Library management system, Banking software

1.3 History of Artificial Intelligence

The development of Artificial Intelligence did not begin with computers themselves, but with foundational questions in mathematics and logic about what could, in principle, be computed or proven. This section traces the major milestones that shaped AI from its theoretical origins to the present day.

1.3.1 Theoretical Foundations (1920s–1930s)

- **1931 — Kurt Gödel** laid the foundations of theoretical computer science by publishing the first universal formal language and proving his *incompleteness theorems* —

showing that any sufficiently powerful formal mathematical system is either inconsistent (flawed) or contains true statements that cannot be proven within the system.

- **1936** — **Alan Turing** reformulated Gödel’s result (along with Alonzo Church’s related extension) in terms of an abstract computing device — now known as the **Turing Machine** — providing a formal definition of what it means for a function to be computable, and laying the mathematical groundwork for the modern computer.

1.3.2 The Birth of AI (1950s)

- **1950** — **Alan Turing** published “*Computing Machinery and Intelligence*,” proposing the **Imitation Game** — later known as the **Turing Test** (Chapter 1, Section 4.2) — as an operational test for machine intelligence.
- **1956** — **John McCarthy** coined the term “**Artificial Intelligence**” as the subject of the **Dartmouth Conference**, the first conference devoted entirely to the field. McCarthy is regarded as the **Father of Artificial Intelligence**.
- **1957** — **Newell, Shaw & Simon** demonstrated the **General Problem Solver (GPS)**, an early attempt at a universal, symbolic problem-solving program (see Chapter 2).
- **1958** — **John McCarthy** (MIT) invented the **Lisp** programming language, which became the dominant language for symbolic AI programming for decades.
- **1959** — **Arthur Samuel** (IBM) wrote the first game-playing program, for Checkers, sufficiently skilled to challenge strong human players — an early demonstration of machine learning through self-play.

1.3.3 Early Growth and Knowledge-Based Systems (1960s–1970s)

- **1963** — **Ivan Sutherland**’s MIT dissertation on *Sketchpad* introduced interactive computer graphics.
- **1966** — **Ross Quillian** (PhD dissertation, Carnegie Institute of Technology, now CMU) demonstrated **semantic networks** as a technique for knowledge representation.
- **1967** — **DENDRAL**, developed by Edward Feigenbaum, Joshua Lederberg, Bruce Buchanan, and Georgia Sutherland at Stanford, interpreted mass spectra of organic chemical compounds — the first successful **knowledge-based** program for scientific reasoning (see Chapter 3).
- **1967** — **Doug Engelbart** invented the computer mouse at SRI.
- **1968** — **Marvin Minsky & Seymour Papert** published *Perceptrons*, mathematically demonstrating the limitations of simple (single-layer) neural networks — a result

that contributed to reduced funding for neural network research over the following decade.

- **1972** — **Alain Colmerauer** developed **Prolog**, the logic programming language central to Unit 3 of this course.

1.3.4 Neural Networks, Expert Systems, and Renewed Progress (1980s–1990s)

- **Mid-1980s** — Neural networks became widely used again following the popularization of the **Backpropagation** algorithm (first described by Werbos in 1974), enabling practical training of multi-layer networks and reviving interest after the setback caused by *Perceptrons*.
- **1990** — Major advances occurred across nearly all areas of AI, with significant demonstrations in machine learning, intelligent tutoring systems, case-based reasoning, multi-agent planning and scheduling, reasoning under uncertainty, data mining, natural language understanding and translation, computer vision, virtual reality, and game-playing.
- **1997** — **Deep Blue** (IBM) defeated World Chess Champion Garry Kasparov — a landmark demonstration of the game-playing and search techniques covered in Chapter 4.
- **2002** — iRobot, founded by researchers from the MIT Artificial Intelligence Lab, introduced the **Roomba** autonomous vacuum-cleaning robot; over two million units had been sold by 2006.

1.3.5 The Deep Learning Era (2010s)

- **2012** — **AlexNet** won the ImageNet competition by a dramatic margin, demonstrating the power of deep convolutional neural networks and triggering the modern deep learning boom.
- **2016** — **AlphaGo** (Google DeepMind), combining deep neural networks with search techniques related to those in Chapter 4, defeated world champion Go player Lee Sedol in Seoul — a milestone widely regarded as far ahead of expectations for that era, since Go’s search space is vastly larger than Chess’s.
- **2017** — Google researchers published “*Attention Is All You Need*,” introducing the **Transformer** architecture, which underpins essentially all modern large language models.
- **2020** — **AlphaFold 2** (Google DeepMind) solved the fifty-year-old **protein-folding problem** at the CASP14 competition, demonstrating AI’s ability to tackle major open

problems in the natural sciences. In the same year, OpenAI released **GPT-3**, a 175-billion-parameter language model showing that scale alone could produce remarkably capable, few-shot language understanding.

1.3.6 The Generative AI and Agentic Era (2022–2026)

- **2022** — **ChatGPT** (OpenAI), based on the GPT-3.5/GPT-4 family, was released as a public research preview and reached 100 million users within two months — the fastest-growing consumer application in history at the time, and the moment generative AI entered mainstream public awareness.
- **2023** — **GPT-4** (OpenAI) demonstrated strong multimodal reasoning; **Anthropic** released **Claude 1**; **Google** released **Gemini 1.0**. Growing concern about the pace of AI progress led many prominent researchers and technologists to sign public statements calling for proactive AI safety governance. The UK hosted the first **AI Safety Summit** at Bletchley Park, the EU advanced the **AI Act**, and the US issued an Executive Order on AI safety — marking the beginning of formal AI regulation at a global scale.
- **2024** — Models such as **GPT-4o**, **Claude 3.5 Sonnet**, and **Gemini 1.5** introduced natively multimodal processing (text, image, audio, and video together). **OpenAI's o1** introduced explicit *chain-of-thought* reasoning at inference time, shifting focus from pure parameter scaling toward reasoning-time computation. The **2024 Nobel Prize in Physics** was awarded to Hopfield and Hinton for foundational work on neural networks, and the **Nobel Prize in Chemistry** was awarded to Hassabis, Jumper, and Baker for AlphaFold's contributions to protein structure prediction — a rare instance of AI research directly earning the field's highest scientific honours.
- **2025** — Often described as the beginning of the "**Age of Agentic AI**": large language models gained the ability to autonomously use tools, browse the web, and execute multi-step coding and research workflows with limited human supervision, rather than simply answering single-turn queries.
- **2026** — AI reached deep integration into everyday industry workflows, with autonomous **AI agents** increasingly handling complex, multi-step tasks (research, coding, and operational workflows) with limited supervision, alongside continued progress in physical/embodied AI (humanoid robotics) and an intensified global focus on AI governance, safety, and regulation as adoption became near-universal across major industries.

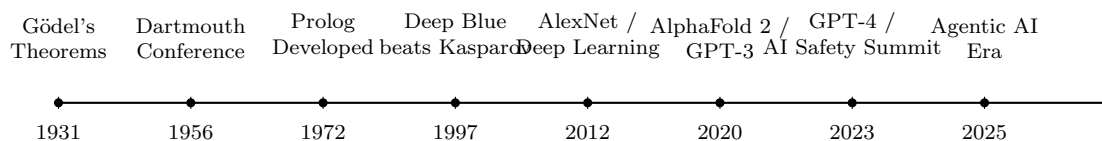


Figure 1.1: A timeline of major milestones in the history of Artificial Intelligence, from its theoretical foundations to the current agentic AI era.

The trajectory of this history is worth noting: AI progressed through several cycles of intense optimism followed by periods of reduced funding and interest — commonly called “**AI Winters**” — most notably following the limitations exposed by *Perceptrons* in 1968, and again in the late 1980s and early 1990s. Since roughly 2012, however, the field has experienced sustained and accelerating progress, driven first by deep learning and, from 2020 onward, by large-scale generative models and increasingly autonomous AI agents.

1.4 Definitions of Artificial Intelligence

1.4.1 John McCarthy (1956)

“Artificial Intelligence is the science and engineering of making intelligent machines, especially intelligent computer programs.”

John McCarthy is regarded as the **Father of Artificial Intelligence**. He coined the term *Artificial Intelligence* in 1956 during the Dartmouth Conference.

1.4.2 Elaine Rich (1983)

“Artificial Intelligence is the study of how to make computers do things at which, at the moment, people are better.”

This definition emphasizes the development of computer systems that can perform tasks requiring human intelligence.

1.4.3 Russell & Norvig (2021)

“Artificial Intelligence is the study of intelligent agents that perceive their environment and take actions that maximize their chances of achieving their goals.”

This modern definition focuses on intelligent agents that can perceive, reason, learn, and act rationally to achieve specific objectives.

1.5 Approaches to Artificial Intelligence

Artificial Intelligence can be viewed from different perspectives depending on its objectives and capabilities. Researchers have proposed various approaches to understand, develop, and evaluate intelligent systems.

1.5.1 Cognitive Science Approach

The Cognitive Science approach focuses on understanding how the human mind thinks, learns, reasons, and solves problems. It aims to develop intelligent systems that emulate human thought processes. This approach combines concepts from Artificial Intelligence, Psychology, Neuroscience, Linguistics, and Cognitive Science to model human intelligence.

1.5.2 Turing Test Approach

The Turing Test was proposed by Alan Turing in 1950 to determine whether a machine can exhibit intelligent behaviour similar to that of a human.

The Turing Test consists of three separate rooms containing:

- A human person
- A computer (machine)
- An interrogator

The working of the Turing Test is as follows:

- The interrogator communicates with the person and the computer through a teletype (text-based communication) so that the machine cannot imitate the appearance or voice of the person.
- The interrogator tries to determine which participant is the human and which is the machine.
- The machine attempts to convince the interrogator that it is the human.
- The human also tries to convince the interrogator that he or she is the human.
- If the interrogator is unable to distinguish the machine from the human, then the machine is said to have passed the Turing Test and is considered intelligent.

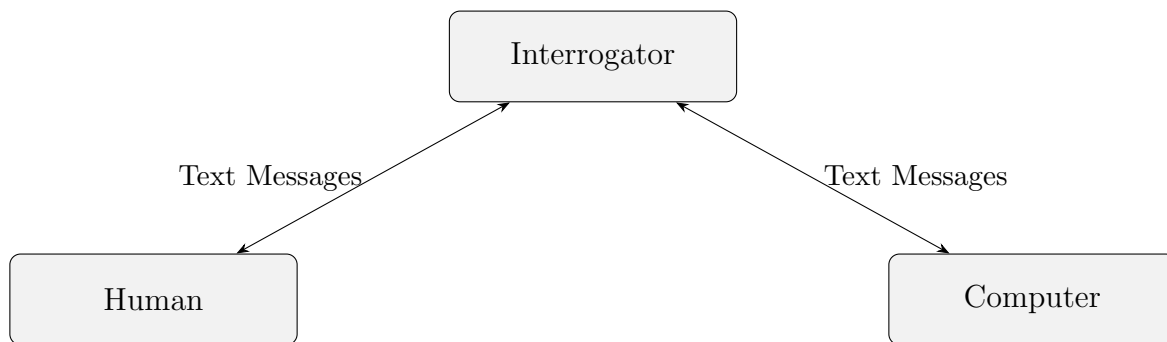


Figure 1.2: Turing Test

1.5.3 Weak AI (Narrow AI)

Weak AI, also known as Narrow AI, is designed to perform specific tasks within a limited domain. These systems can simulate intelligent behaviour but do not possess consciousness, self-awareness, or general reasoning abilities. Most AI applications available today, including virtual assistants, recommendation systems, image recognition, and language translation, belong to this category.

1.5.4 Strong AI (Artificial General Intelligence)

Strong AI, also known as Artificial General Intelligence (AGI), refers to machines capable of understanding, learning, reasoning, and solving problems across a wide range of tasks at a level comparable to human intelligence. Such systems would be capable of transferring knowledge from one domain to another and making independent decisions. As of 2026, Strong AI remains a research goal and has not yet been achieved.

1.5.5 Super AI

Super AI is a hypothetical form of Artificial Intelligence that would surpass human intelligence in all aspects, including creativity, reasoning, decision-making, emotional understanding, and scientific discovery. It remains a theoretical concept and has not been realized.

1.5.6 Applied AI

Applied AI focuses on developing practical AI solutions for real-world applications. It is widely used in healthcare, finance, education, agriculture, cybersecurity, manufacturing, transportation, and many other industries to automate tasks and improve decision-making.

1.6 Problems of Artificial Intelligence

Intelligence does not imply perfect understanding. Every intelligent being has limited perception, memory, and computational capabilities. Thus, intelligence exists on a spectrum, and different levels of intelligence can be achieved at different costs. For example, insects exhibit relatively simple forms of intelligent behaviour, whereas humans demonstrate highly complex forms of intelligence.

Artificial Intelligence (AI) seeks to understand the computations required for intelligent behaviour and to develop computer systems that exhibit such behaviour. In other words, AI attempts to understand how intelligent behaviour can be represented, reasoned about, learned, planned, and implemented by machines.

The study of intelligence in AI involves several important aspects of intelligent behaviour, including:

- **Perception:** The ability to acquire and interpret information from the surrounding environment.
- **Communication:** The ability to communicate using human languages and understand information expressed through language.
- **Reasoning:** The ability to draw conclusions from available information and knowledge.
- **Planning:** The ability to determine a sequence of actions required to achieve a desired goal.

- **Learning:** The ability to acquire knowledge or improve performance through experience and data.
- **Memory:** The ability to store, retrieve, and use previously acquired information.

Before proceeding with the study of Artificial Intelligence, several fundamental questions need to be considered. These questions help us understand the nature of intelligence and the challenges involved in developing intelligent systems.

1. **What are the underlying assumptions about intelligence?**

We need to determine what intelligence actually means and what properties or capabilities characterize an intelligent entity.

2. **What kinds of techniques will be useful for solving AI problems?**

AI problems are often complex and cannot always be solved using conventional programming techniques. Therefore, appropriate methods for knowledge representation, search, reasoning, learning, and decision-making must be considered.

3. **At what level can human intelligence be modelled?**

An important question is whether human intelligence can be completely or partially represented using computational models and, if so, to what extent.

4. **When will it be realized that an intelligent program has been built?**

We need suitable criteria for determining whether a computer program can genuinely be considered intelligent. This raises questions about how intelligent behaviour should be measured and evaluated.

The major problems addressed in Artificial Intelligence can therefore be organized into several important areas, including knowledge representation, reasoning, learning, planning, natural language understanding, computer vision, and robotics.

1.6.1 Knowledge Representation

Knowledge Representation is concerned with how information about the real world can be represented in a form that a computer can store, understand, and use. An intelligent system needs knowledge about objects, properties, events, relationships, and rules in order to perform intelligent tasks. Different representations, such as logic, semantic networks, frames, and ontologies, can be used to represent knowledge. The main challenge is to represent knowledge in a way that allows an AI system to reason effectively and make appropriate decisions.

1.6.2 Reasoning

Reasoning is the process of deriving new conclusions from existing knowledge and information. An intelligent system must be able to use facts, rules, and available evidence to

reach conclusions or make decisions. Reasoning may involve logical deduction, probabilistic reasoning, or reasoning in situations where information is incomplete or uncertain. The challenge is to develop methods that allow machines to draw useful and reliable conclusions.

1.6.3 Learning

Learning is the ability of an AI system to improve its performance by acquiring knowledge from data, examples, observations, or experience. Instead of being explicitly programmed for every possible situation, a learning system can identify patterns and use them to make predictions or decisions. Machine Learning provides important techniques for achieving this capability. The major challenge is to enable a system to learn accurately and generalize its knowledge to situations that it has not encountered before.

1.6.4 Planning

Planning is concerned with determining a sequence of actions that an intelligent agent should perform in order to achieve a particular goal. A planning system must consider the current state of the environment, the available actions, their possible consequences, and the desired goal state. The challenge becomes more complex when there are many possible actions, limited resources, uncertainty, or changing environments.

1.6.5 Natural Language Understanding

Natural Language Understanding (NLU) is concerned with enabling computers to understand and interpret human language. Human language contains ambiguity, context, implicit meaning, and variations in expression, making it difficult for computers to interpret correctly. An AI system may need to understand words, sentences, context, intent, and relationships between different pieces of information. NLU forms an important part of Natural Language Processing (NLP) and is used in applications such as virtual assistants, question answering systems, and conversational AI.

1.6.6 Computer Vision

Computer Vision is concerned with enabling computers to interpret and understand information contained in images and videos. A computer vision system may need to identify objects, recognize faces, understand scenes, or track movement. The challenge is to transform raw visual information into meaningful information that can be used for reasoning and decision-making. Computer Vision has applications in areas such as medical imaging, autonomous vehicles, surveillance, and image recognition.

1.6.7 Robotics

Robotics combines Artificial Intelligence with mechanical and computational systems to enable machines to perceive their environment, make decisions, and perform physical actions. An intelligent robot may need to sense its surroundings, understand its current state, plan

actions, and respond to changes in the environment. Robotics therefore brings together several AI problems, including perception, reasoning, planning, learning, and control. The major challenge is to enable robots to operate safely and effectively in complex and dynamic real-world environments.

1.7 AI Techniques

Solving problems in Artificial Intelligence requires more than just having sufficient knowledge; it also requires the ability to organize and use that knowledge effectively. An AI technique is a method that exploits knowledge to be represented in such a way that:

- The knowledge captures generalizations, i.e., situations that share important properties are grouped together, rather than being represented separately for each individual situation.
- It can be understood by the people who must provide it, even though for many programs the bulk of the data may be acquired without the aid of an external programmer.
- It can be easily modified to correct errors and to reflect changes in the real world and in the way the system deals with the world.
- It can be used in a great many situations even if it is not totally accurate or complete.
- It can be used to help overcome the sheer bulk of information by helping to reduce the range of possibilities that must usually be considered.

1.7.1 Search

Search is one of the most fundamental techniques used in AI to solve problems for which no direct or predetermined solution exists. Many AI problems can be formulated as a search through a space of possible states, starting from an initial state and moving towards a goal state using a set of permissible operations. Since the number of possible states can be extremely large, AI systems use search strategies (such as breadth-first, depth-first, and heuristic search) to explore the state space efficiently and find an acceptable solution without examining every possibility.

1.7.2 Knowledge Representation

As discussed earlier, Knowledge Representation deals with how facts about the world can be encoded in a form usable by a computer program. As an AI technique, it is concerned not just with storing facts, but with structuring knowledge (using logic, semantic networks, frames, rules, or ontologies) so that reasoning and inference can be carried out efficiently over it.

1.7.3 Inference

Inference is the process by which new facts are derived from existing knowledge using logical or probabilistic rules. Once knowledge has been represented in a suitable form, an AI system must be able to draw valid conclusions from it, apply rules to specific situations, and update its beliefs when new evidence becomes available. Inference forms the bridge between stored knowledge and intelligent decision-making.

1.7.4 Learning

Learning, as an AI technique, refers to the mechanisms by which a system improves its performance over time based on data, feedback, or experience, rather than depending entirely on knowledge explicitly supplied by a programmer. Techniques such as supervised learning, unsupervised learning, and reinforcement learning allow AI systems to detect patterns, build models, and generalize to previously unseen situations.

1.7.5 Planning

Planning as an AI technique involves generating a sequence of actions, from a set of available operations, that will transform an initial state into a desired goal state. Since real-world environments often involve numerous possible actions, resource constraints, and uncertainty, planning techniques must efficiently search through possible action sequences to construct a valid and, ideally, optimal plan.

1.8 Tic-Tac-Toe Problem

1.8.1 Introduction

Tic-Tac-Toe is a classic example used to illustrate fundamental AI concepts such as state space representation, search, and game trees. It is a simple two-player game played on a 3×3 grid, where one player uses 'X' and the other uses 'O'. Players alternately mark empty cells, and the first player to align three of their marks horizontally, vertically, or diagonally wins the game. If all nine cells are filled without either player achieving such an alignment, the game ends in a draw.

Despite its simplicity, Tic-Tac-Toe is a useful problem for introducing AI because it has a small, well-defined state space, a clear goal, and precisely specified rules, making it easy to illustrate concepts such as state space representation, search, and game trees without the complexity of real-world problems.

1.8.2 State Space

The state space of Tic-Tac-Toe consists of all possible configurations of the 3×3 board that can arise during the course of a game. Each state can be represented as a 3×3 matrix, with each cell containing one of three values: 'X', 'O', or blank (empty).

- The **initial state** is the empty board, with all nine cells blank.
- An **operator** (or move) consists of placing the current player's mark ('X' or 'O') into any blank cell.
- A **goal state** is any board configuration in which one player has three marks in a row, column, or diagonal (a win), or in which the board is completely filled with no winner (a draw).

Since players alternate turns and each move places a mark in a previously blank cell, the total number of reachable states is finite; the upper bound is $9!$ (approximately 362,880), though the actual number of legal, distinct states is smaller once winning positions and symmetries are accounted for.

1.8.3 Game Tree (Basic Idea)

A game tree is a tree structure used to represent all possible ways a game can be played from a given state. In the game tree for Tic-Tac-Toe:

- The **root node** represents the initial (empty) board.
- Each **node** represents a particular board configuration (state).
- Each **edge** represents a legal move made by a player, transforming a parent state into a child state.
- The **branching factor** decreases at each level, since the number of available blank cells reduces as the game progresses (9 choices at the first move, 8 at the second, and so on).
- **Leaf nodes** represent terminal states — a state in which a player has won, or the board is full and the game is a draw.
- The **depth** of the tree corresponds to the number of moves made; since there are only nine cells, the maximum depth of the tree is 9.

Game trees provide the basis for search-based decision-making in games. In Tic-Tac-Toe, algorithms such as Minimax can traverse this tree to determine the optimal move for a player by assuming that the opponent also plays optimally, alternating between maximizing and minimizing a utility value assigned to terminal states (e.g., +1 for a win, 0 for a draw, -1 for a loss).

1.8.4 Why it is an AI Problem

Tic-Tac-Toe qualifies as an AI problem, in the classical sense, because it exhibits several essential characteristics that AI techniques are designed to address:

- **State Space Representation:** The problem can be formally represented as a well-defined set of states, an initial state, a goal state, and operators (moves) that transform one state into another — a structure common to a wide range of AI problems.
- **Search:** Determining the best move at any given point requires searching through the space of possible future states, which is exactly the kind of exploration that AI search techniques are designed to perform.
- **Decision-Making under Adversarial Conditions:** Unlike simple search problems, Tic-Tac-Toe involves an opponent whose moves must be anticipated. This introduces the notion of adversarial search, where an intelligent agent must reason not only about its own goals but also about an opponent working against those goals.
- **Rational Behavior:** An intelligent player must select actions expected to maximize its chances of winning (or at least avoid losing), based on the current state and reasoning about future consequences — consistent with the "Ideal Rational Agent" notion introduced earlier.
- **Scalability of Concepts:** Although Tic-Tac-Toe itself has a small, tractable state space, the same underlying concepts (state space, search, game trees, and adversarial reasoning) extend directly to far more complex games such as Chess and Go, making it a useful pedagogical stepping stone into more advanced AI game-playing techniques.

Thus, Tic-Tac-Toe serves as a simple yet complete illustration of how AI techniques — state space representation, search, and decision-making — come together to solve a well-defined problem.

1.9 Chapter Summary

- **Artificial Intelligence (AI)** is concerned with designing systems that exhibit intelligent behaviour in artificial devices.
- The term **Artificial Intelligence** was coined by **John McCarthy** in 1956 at the Dartmouth Conference.
- AI can be viewed from different perspectives. A system may be considered intelligent based on how it **thinks** or how it **acts**, and its behaviour may be compared with humans or with the ideal rational behaviour.
- Modern AI systems are highly successful in specialized areas such as **Machine Learning, Natural Language Processing, Computer Vision, Robotics, Expert Systems, Speech Processing, and Generative AI**.
- Most current AI systems are examples of **Narrow AI** or **Weak AI**, because they are designed to perform specific tasks rather than demonstrate complete human-level intelligence.

- AI differs from conventional programming because AI systems may learn from data, use knowledge, reason about problems, and adapt their behaviour to new situations.
- The history of AI developed from theoretical foundations in mathematics and logic to modern systems based on Machine Learning, Deep Learning, Generative AI, and increasingly autonomous AI agents.
- Important milestones include **Turing's work on computation and the Turing Test, the Dartmouth Conference, Lisp, Prolog, Expert Systems, Neural Networks, Deep Blue, AlexNet, AlphaGo, Transformers, AlphaFold, GPT models, and Generative AI.**
- The major problems addressed by AI include **Knowledge Representation, Reasoning, Learning, Planning, Natural Language Understanding, Computer Vision, and Robotics.**
- Core AI techniques include **Search, Knowledge Representation, Inference, Learning, and Planning.** These techniques allow an AI system to represent knowledge, explore alternatives, draw conclusions, improve from experience, and achieve goals.
- The **Tic-Tac-Toe problem** is a classical AI problem used to demonstrate state-space representation, search, game trees, adversarial reasoning, and decision-making.
- The study of AI ultimately raises a fundamental question: **what does it mean for a machine to behave intelligently?** Understanding this question provides the foundation for studying the more advanced AI techniques covered in the later chapters.

1.10 Key Points to Remember

1. **Artificial Intelligence** is the study and development of intelligent behaviour in artificial systems.
2. **John McCarthy** coined the term Artificial Intelligence in **1956**.
3. AI may be studied in terms of **thinking** or **acting**, and behaviour may be compared with humans or with ideal rational behaviour.
4. **Narrow AI (Weak AI)** is designed for specific tasks.
5. **Strong AI (AGI)** refers to the idea of a machine with broad, general-purpose intelligence comparable to human intelligence.
6. **Super AI** is a hypothetical form of AI that would surpass human intelligence across a wide range of capabilities.
7. Important AI capabilities include **perception, communication, reasoning, planning, learning, and memory.**

8. Important AI problem areas include: **Knowledge Representation, Reasoning, Learning, Planning, Natural Language Understanding, Computer Vision, and Robotics.**
9. Important AI techniques include: **Search, Knowledge Representation, Inference, Learning, and Planning.**
10. **Search** explores a problem space to find a suitable path from an initial state to a goal state.
11. **Knowledge Representation** stores information about the world in a form that an AI system can use for reasoning.
12. **Inference** derives new knowledge or conclusions from existing facts and rules.
13. **Learning** enables a system to improve its performance from data, examples, feedback, or experience.
14. **Planning** determines a sequence of actions required to achieve a desired goal.
15. The **Turing Test** evaluates machine intelligence through its observable conversational behaviour.
16. **Tic-Tac-Toe** is a simple classical AI problem that demonstrates state-space search and adversarial decision-making.
17. The development of AI has passed through several stages, including symbolic AI, knowledge-based systems, neural networks, machine learning, deep learning, and Generative AI.

1.11 Questions

1.11.1 Short Questions

1. What is Artificial Intelligence?
2. Who coined the term Artificial Intelligence?
3. In which year was the term Artificial Intelligence coined?
4. What are the two basic ideas contained in the definition of Artificial Intelligence?
5. What is meant by Narrow AI?
6. What is Strong AI or Artificial General Intelligence?
7. What is Super AI?

8. What is Applied AI?
9. What is the Turing Test?
10. Who proposed the Turing Test?
11. What are the four approaches to Artificial Intelligence?
12. What is meant by thinking humanly?
13. What is meant by thinking rationally?
14. What is meant by acting humanly?
15. What is meant by acting rationally?
16. Mention any four current applications of Artificial Intelligence.
17. What is Knowledge Representation?
18. What is reasoning in Artificial Intelligence?
19. What is learning in Artificial Intelligence?
20. What is planning in Artificial Intelligence?
21. What is Natural Language Understanding?
22. What is Computer Vision?
23. What is Robotics in Artificial Intelligence?
24. What is meant by AI technique?
25. Mention any five important AI techniques.
26. What is search in Artificial Intelligence?
27. What is inference?
28. What is a state space?
29. What is a game tree?
30. Why is Tic-Tac-Toe considered an AI problem?
31. Mention any four important milestones in the history of AI.
32. What is meant by an AI Winter?

33. What is Generative AI?

1.11.2 Long Questions

1. Define Artificial Intelligence. Explain the different interpretations of intelligence and artificial devices.
2. Explain the current capabilities of Artificial Intelligence with suitable examples.
3. Compare Artificial Intelligence with conventional programming.
4. Explain the historical development of Artificial Intelligence from its theoretical foundations to the modern Generative AI era.
5. Discuss the major milestones in the history of Artificial Intelligence.
6. Explain the different definitions of Artificial Intelligence given by John McCarthy, Elaine Rich, and Russell and Norvig.
7. Explain the different approaches to Artificial Intelligence.
8. Explain the Turing Test with the help of a suitable diagram. Discuss its importance in Artificial Intelligence.
9. Differentiate between Weak AI, Strong AI, Super AI, and Applied AI.
10. Explain the major problems of Artificial Intelligence.
11. Explain Knowledge Representation, Reasoning, Learning, and Planning as important AI problem areas.
12. Explain Natural Language Understanding, Computer Vision, and Robotics as important areas of Artificial Intelligence.
13. What are AI techniques? Explain Search, Knowledge Representation, Inference, Learning, and Planning.
14. Explain the Tic-Tac-Toe problem as an Artificial Intelligence problem.
15. Explain the state-space representation of Tic-Tac-Toe.
16. What is a game tree? Explain how a game tree can be used to solve Tic-Tac-Toe.
17. Discuss the role of search and decision-making in game-playing problems.
18. Explain how the major AI techniques are connected to each other in solving an intelligent problem.

19. Discuss the evolution of Artificial Intelligence from symbolic reasoning to Machine Learning, Deep Learning, and Generative AI.
20. Explain why the study of Artificial Intelligence involves both technical and philosophical questions about intelligence and intelligent behaviour.

Chapter 2

Problem Space and Search

2.1 Introduction

Problem solving is one of the central concerns of Artificial Intelligence. It may be defined as the process of generating solutions from observed or given data. Unlike conventional programming, where a direct path from data to solution is usually known in advance, AI problem solving often has to rely on indirect, model-based methods, since a direct method is not always available.

To build a system capable of solving a given problem, the following broad steps are generally followed:

1. **Define the problem accurately**, including a detailed specification of what constitutes an acceptable solution.
2. **Analyze the problem carefully**, since certain features of the problem may have a significant effect on the choice of solution method.
3. **Isolate and represent the background knowledge** needed to solve the problem.
4. **Choose the best available problem-solving technique(s)** and apply them to reach a solution.

A **problem** is characterized by a set of *goals*, a set of *objects*, and a set of *operations*. These elements may be ill-defined at the outset and may evolve as problem solving proceeds. The abstract space encompassing all valid states reachable by applying combinations of operators to combinations of objects is called the **problem space**. A problem space may contain one or more solutions, where a solution is a specific combination of operations and objects that achieves the stated goals. The process of locating such a solution within the problem space is called **search**.

One of the earliest attempts at general-purpose problem solving was the **General Problem Solver (GPS)**, developed by Newell and Simon in 1957. GPS was built on their theoretical

work on logic machines and was, in principle, capable of tackling any formalized symbolic problem — theorem proving, geometric problems, and even chess playing. In practice, GPS successfully solved well-formalized puzzles such as the Towers of Hanoi, but it proved incapable of handling real-world problems, largely because real-world problems resist the kind of clean formalization GPS required.

2.2 Defining a Problem as a State Space Search

To solve a problem such as playing a game, we require the rules of the game, the criteria for winning, and a way of representing positions within the game. The opening position of the game can be treated as the **initial state**, and a winning position as the **goal state**. Legal moves connect the initial state to the goal state through a sequence of intermediate states.

In practice, the number of applicable rules can be extremely large — in games such as chess, the number of possible positions is said to exceed the number of particles in the known universe. This makes it impractical to enumerate rules exhaustively; instead, rules must be expressed as compactly and generally as possible, and the resulting state space must be searched systematically rather than exhaustively.

2.2.1 Problem Definition

A problem is defined by its *elements* and the *relations* between them. A formal description of a problem requires the following:

1. Defining a **state space** containing all possible configurations of the relevant objects — including configurations that may turn out to be invalid or unreachable.
2. Specifying one or more **initial states** — states describing possible starting situations from which problem solving may begin.
3. Specifying one or more **goal states** — states that would constitute an acceptable solution.
4. Specifying a set of **rules (operators)** that describe the actions available for moving between states.

The problem is then solved by applying these rules, guided by an appropriate control strategy, to move through the problem space until a path from an initial state to a goal state is found. This process — **search** — is fundamental to problem solving in AI:

- Search is a general mechanism used when no more direct method of solution is known.
- Search provides a framework into which more direct methods, where available, can be embedded for solving sub-parts of a problem.
- A very large number of AI problems can be formulated as search problems.

A problem space is naturally represented as a **directed graph**, where nodes correspond to states and edges correspond to the operators that transform one state into another. For simplicity, it is often convenient to represent a problem space as a **tree** rather than a general graph — this reduces the complexity of search algorithms, though at the cost of duplicating certain nodes that may otherwise be reachable via multiple paths in the graph. Formally, a tree is a connected graph with no cycles, in which any two vertices are joined by exactly one path.

Problem-solving methods are broadly categorized as follows:

- **Special-purpose methods:** tailor-made for a particular problem, exploiting specific features of that problem's situation.
- **General-purpose methods:** applicable across a wide variety of problems. A well-known general-purpose technique in AI is **means-end analysis** — a step-by-step, incremental reduction of the difference between the current state and the final goal.

2.2.2 Production Systems

A **production system** provides a structured way of performing and describing the search process. It consists of four basic components:

1. A **set of rules**, each with a left side (determining the applicability of the rule) and a right side (describing the operation to be performed if the rule is applied).
2. A **database** of current facts, established during the process of inference.
3. A **control strategy** that specifies the order in which rules are compared against facts in the database, and how conflicts are resolved when multiple rules or facts are applicable.
4. A **rule-firing module** responsible for actually applying the selected rule.

Each rule has a *precondition*, which is either satisfied or not by the current state of the database. If satisfied, the rule may be applied; applying it changes the database. The control strategy repeatedly selects and applies rules in this manner until a termination condition on the database is satisfied.

2.2.3 Characteristics of Production Systems

Production systems can be classified according to the following properties:

- **Monotonic production system:** the application of a rule never prevents the later application of another rule that could also have been applied at the time the first rule was selected.
- **Non-monotonic production system:** a production system in which this property does not hold.

- **Partially commutative production system:** if a particular sequence of rules transforms state P into state Q , then any allowable reordering of that sequence also transforms P into Q .
- **Commutative production system:** a system that is both monotonic and partially commutative.

In a strictly formal sense, there is no relationship between the class of a problem and the class of production system needed to solve it — any solvable problem can, in principle, be solved by a commutative production system, though such a system is often impractical to use directly. In practice, however, a strong relationship exists: **partially commutative, monotonic** production systems are well suited to *ignorable* problems, since they do not require backtracking. Production systems that are *not* partially commutative are better suited to problems involving permanent, irreversible changes of state.

2.3 Problem Characteristics

Since heuristics are inherently domain-specific and cannot be generalized across all problems, it is important to analyze the characteristics of a given problem before selecting a search or representation strategy. The following questions guide this analysis.

2.3.1 Is the Problem Decomposable?

A key question is whether a large problem can be broken down into a set of smaller, independent sub-problems, each solvable using a small collection of specific rules. For example, the indefinite integral of $x^3 + x^2 + 2x + 3\sin x$ can be evaluated by decomposing it into the integrals of each term separately. Where such decomposition is possible, considerably large problems can be solved using comparatively simple techniques — this decomposability is itself regarded as a form of intelligent behaviour.

2.3.2 Can Solution Steps be Ignored or Undone?

Problems differ in whether an incorrect or unhelpful step, once taken, can be ignored, undone, or not undone at all. This leads to three important classes of problems:

1. **Ignorable:** solution steps can simply be ignored if they turn out to be unhelpful. *Example: Theorem Proving* — if an attempted lemma proves useless, it can simply be abandoned in favour of another.
2. **Recoverable:** solution steps can be undone, provided the control strategy keeps track of the states visited so that backtracking is possible. *Example: 8-Puzzle* — an incorrect move can be reversed by backtracking to an earlier state.
3. **Irrecoverable:** solution steps, once taken, cannot be undone. *Example: Chess* — once a move is made, its consequences cannot be reversed.

2.3.3 Is the Universe Predictable?

A problem's universe may be predictable or unpredictable. In the **8-Puzzle**, every move has a certain, known outcome — this makes it possible to plan an entire sequence of moves in advance and to backtrack confidently if a planned sequence proves unwise. In games such as **Bridge**, however, the outcome of a given action cannot be known with certainty in advance. Solving such problems typically requires a process of **plan revision**, in which the plan is adjusted as it is executed, based on feedback — a process that tends to be computationally expensive.

2.3.4 Is a Good Solution Absolute or Relative?

For some problems, any one of several valid solution paths is acceptable, and there is no need to compare the path taken against alternative paths once a correct answer has been reached. Consider a small knowledge base with facts such as "Siva was a man," "Siva was a worker in a company," "Siva was born in 1905," "All men are mortal," "All workers in the company died in a 1952 accident," and "No mortal lives longer than 100 years." The question "Is Siva alive?" can be answered by at least two independent lines of reasoning — one based on Siva's age exceeding the mortal limit, another based on the 1952 accident. Since both paths agree on the answer, there is no need to explore or compare them further; a single successful path suffices — the solution is *relative* rather than requiring an absolute, uniquely optimal derivation.

2.3.5 Is the Solution a State or a Path?

In some problems, only the final state matters, and the sequence of operations used to reach it is irrelevant once the goal is achieved. In others, the entire path — the specific sequence of states and operators — is itself the required solution (for example, in planning problems, where the sequence of actions taken is the deliverable, not merely the final configuration).

2.3.6 What is the Role of Knowledge?

The amount and role of domain knowledge required varies greatly between problems. Some problems can be solved with very little domain-specific knowledge, relying mainly on a systematic, largely uninformed search of the state space. Others — particularly knowledge-intensive tasks such as diagnosis or design — depend heavily on substantial, well-organized domain knowledge to guide the search efficiently toward a solution and to avoid unpromising paths.

2.3.7 Does the Task Require Interaction with a Person?

Finally, a problem-solving task may be designed to run autonomously, without any interaction with a human user once it begins (a *solitary* problem), or it may require ongoing interaction with a person during the solution process (an *interactive* problem, e.g., a diagnostic expert system that queries a user for additional symptoms). This distinction affects

both the design of the control strategy and the way intermediate results must be communicated.

2.4 Problem Space

2.4.1 States, Operators, and Goal Test

A **state** is a representation of the elements of a problem at a given moment, described by a set of descriptors and the relations between them. A **state space** is the set of all states reachable from the initial state via some sequence of operators. Formally, a state space consists of:

- A representation of the states the system can occupy (e.g., a board configuration in a board game).
- A set of **operators** that transform one state into another (e.g., the legal moves available from a given board position).
- An **initial state**, representing the starting configuration.
- A set of **final (goal) states**, some of which may be desirable and others undesirable; this set is often represented implicitly, via a program (a *goal test*) that detects whether a given state is terminal.

State transitions are governed by a **successor function**: a transformation function that maps a state to another state, defining a relation of accessibility between states and specifying the conditions under which such a transformation is applicable.

2.4.2 Representing a Problem Space (State-Space Graph)

A state space naturally forms a **graph**, in which nodes represent states and directed arcs represent operators (actions). A **path** in this graph is a sequence of states connected by a sequence of actions, and a **solution** to the problem corresponds to a path — or, in some formulations, simply a goal state — within this graph.

Two structural forms are commonly used to represent a problem space:

- A **tree**, a hierarchical structure in which there is exactly one path from any given node to any other node.
- A **graph**, a non-hierarchical structure that may permit several distinct paths to reach the same node.

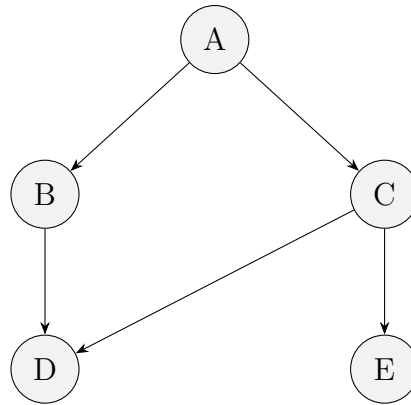


Figure 2.1: A problem space as a graph. Node D is reachable via two distinct paths (through B and through C).

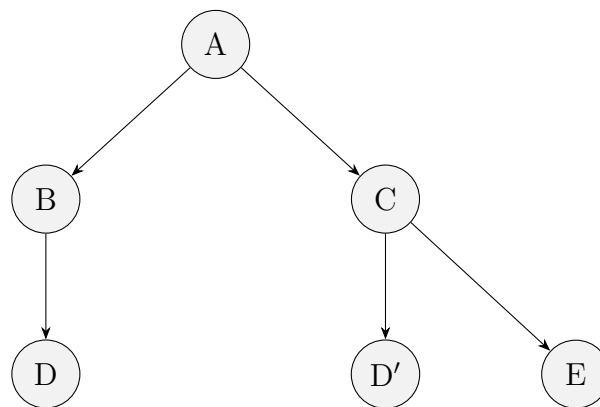


Figure 2.2: The same problem space represented as a tree. Node D is duplicated (as D and D') since only one path to each node is permitted.

Each solution path may be assigned a numeric **path cost**, and the **quality** of a solution is measured by this cost — an **optimal solution** being one with the lowest path cost among all valid solutions. Depending on the application, a problem-solving system may be required to find *any* solution, the *optimal* solution, or *all* solutions.

A complete problem description therefore consists of: the state space (defined explicitly or implicitly), the initial state, the goal state(s) — described completely or partially — and the operators, each consisting of a **precondition** (a partial description of the world that must hold for the operator to apply) and an **instruction** (specifying how the next state is generated). Operators should be kept as general as possible, so as to minimize their number while still covering the space of required actions.

2.5 Water Jug Problem

2.5.1 Problem Statement

Given two jugs — a four-gallon jug and a three-gallon jug, neither of which has any measuring markers — and an unlimited supply of water, the goal is to obtain exactly two gallons of water in the four-gallon jug.

2.5.2 State Space Representation

A state is represented as an ordered pair (x, y) , where x is the number of gallons of water currently in the four-gallon jug ($0 \leq x \leq 4$) and y is the number of gallons in the three-gallon jug ($0 \leq y \leq 3$).

- **Initial state:** $(0, 0)$ — both jugs empty.
- **Goal state:** $(2, n)$, for any valid value of n — i.e., exactly two gallons in the four-gallon jug, regardless of the amount in the three-gallon jug.

2.5.3 Production Rules

The major production rules available for this problem are summarized in Table 2.1, using x for the amount in the four-gallon jug and y for the amount in the three-gallon jug.

Table 2.1: Production Rules for the Water Jug Problem

#	Current State $\rightarrow$ Next State	Condition / Comment
1	$(x, y) \rightarrow (4, y)$	if $x < 4$; fill four-gallon jug from tap
2	$(x, y) \rightarrow (x, 3)$	if $y < 3$; fill three-gallon jug from tap
3	$(x, y) \rightarrow (0, y)$	if $x > 0$; empty four-gallon jug into drain
4	$(x, y) \rightarrow (x, 0)$	if $y > 0$; empty three-gallon jug into drain
5	$(x, y) \rightarrow (x + y, 0)$	if $x + y \leq 4$; pour three-gallon jug into four-gallon jug
6	$(x, y) \rightarrow (0, x + y)$	if $x + y \leq 3$; pour four-gallon jug into three-gallon jug
7	$(0, y) \rightarrow (y, 0)$	if $y > 0$; pour all of three-gallon jug into four-gallon jug
8	$(x, 0) \rightarrow (0, x)$	if $x > 0$; pour all of four-gallon jug into three-gallon jug
9	$(0, 2) \rightarrow (2, 0)$	pour the 2 gallons from three-gallon jug into four-gallon jug
10	$(2, 0) \rightarrow (0, 2)$	pour the 2 gallons from four-gallon jug into three-gallon jug

2.5.4 Solution Path

One possible sequence of states leading from the initial state to a goal state is shown in Table 2.2.

Table 2.2: One Solution to the Water Jug Problem		
Four-Gallon Jug	Three-Gallon Jug	Rule Applied
0	0	—
0	3	2
3	0	7
3	3	2
4	2	5 (pour, leaving remainder)
0	2	3
2	0	9

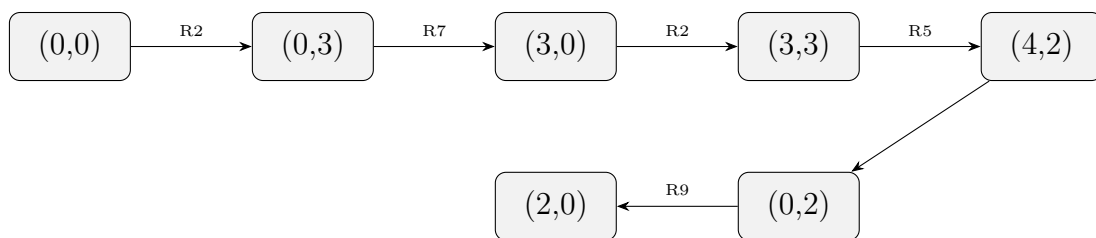


Figure 2.3: Solution path for the Water Jug Problem, from initial state (0,0) to goal state (2,0).

This example illustrates that even a simple, easily-stated problem requires a systematic search through a nontrivial state space to solve reliably — a search that becomes essential once problems grow beyond the point where a solution can be found "by hand."

2.6 8-Puzzle Problem

2.6.1 Problem Statement

The 8-puzzle consists of a 3×3 frame holding eight numbered square tiles (1 through 8) and one blank space. A tile adjacent to the blank space may be slid into it, effectively exchanging the positions of the tile and the blank. The objective is to find a sequence of moves that transforms a given start configuration into a specified goal configuration.

2.6.2 State Space Representation

Each state is represented as a 3×3 arrangement of the eight numbered tiles and the blank. Four operators are available at any state, corresponding to sliding the blank **UP**, **DOWN**, **LEFT**, or **RIGHT** — though not all four are applicable in every state, since the blank may be positioned along an edge or in a corner. A typical instance of the puzzle can be solved by an operator sequence such as UP, RIGHT, UP, LEFT, DOWN, taking the board from its start state to the goal state, as illustrated in Figure 2.4.

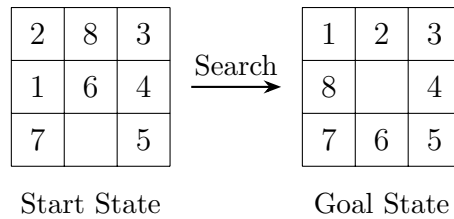


Figure 2.4: An instance of the 8-Puzzle: transforming the start state into the goal state via UP, RIGHT, UP, LEFT, DOWN.

2.6.3 Why Brute-Force Search is Insufficient

Although the 8-puzzle's state space is finite, it is large enough (up to $9!/2 \approx 181,440$ reachable states) that exhaustively generating and examining every state is computationally wasteful. Since the problem universe is fully **predictable** — every move has a known, deterministic outcome — it becomes possible to use **heuristic information** (such as an estimate of "distance" from the goal configuration) to guide the search toward the goal far more efficiently than a blind, brute-force traversal of the state space. This motivates the informed (heuristic) search techniques taken up in the next chapter.

2.7 Search Techniques

2.7.1 Blind (Uninformed) Search vs. Informed Search

Search strategies are broadly divided into two categories:

- **Blind (uninformed) search:** the search strategy has no additional information about states beyond that provided in the problem definition — it can distinguish a goal state from a non-goal state, but has no way of estimating how "close" a given state is to the goal. Nodes are generated and expanded in a fixed, systematic order.
- **Informed (heuristic) search:** the search strategy uses problem-specific knowledge, typically in the form of a heuristic function, to estimate how promising a given state is, allowing the search to be guided more directly toward the goal (discussed in the next chapter).

Search strategies are evaluated along four standard dimensions:

- **Completeness:** does the strategy guarantee finding a solution whenever one exists?
- **Time complexity:** how long does the search take, typically expressed as a function of the branching factor b and solution depth d ?
- **Space complexity:** how much memory is required during the search?
- **Optimality:** does the strategy guarantee finding the lowest-cost solution?

2.7.2 Breadth-First Search (BFS)

Breadth-First Search expands the shallowest unexpanded node first — all nodes at a given depth are expanded before any node at the next depth is considered (Figure 2.5). It is typically implemented using a **queue** (FIFO) data structure to hold the frontier of nodes awaiting expansion.

- **Completeness:** Complete, provided the branching factor b is finite.
- **Optimality:** Optimal if all step costs are equal.
- **Time complexity:** $O(b^d)$, where d is the depth of the shallowest goal.
- **Space complexity:** $O(b^d)$ — BFS must keep every generated node in memory, making it highly memory-intensive for large state spaces.

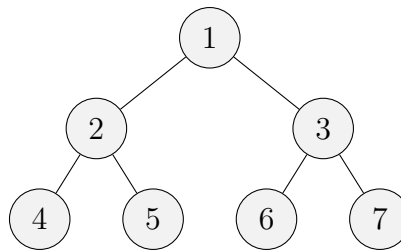


Figure 2.5: BFS expansion order: 1, 2, 3, 4, 5, 6, 7 (level by level).

2.7.3 Depth-First Search (DFS)

Depth-First Search expands the deepest unexpanded node first, backtracking only when a path reaches a state with no further successors (or a previously visited state), as shown in Figure 2.6. It is typically implemented using a **stack** (LIFO) data structure, or via recursion.

- **Completeness:** Incomplete in general (may loop forever down an infinite or very deep path), but complete on finite state spaces if repeated states are avoided.
- **Optimality:** Not optimal — it may find a solution that is far from the shallowest one.
- **Time complexity:** $O(b^m)$, where m is the maximum depth of the state space (which may be much larger than d).
- **Space complexity:** $O(bm)$ — a significant advantage over BFS, since only a single path from root to leaf, along with the unexplored siblings of each node on that path, needs to be stored.

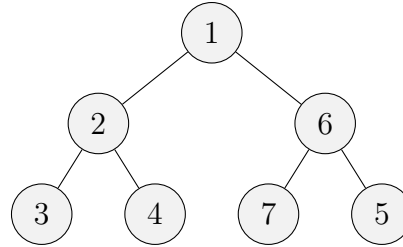


Figure 2.6: DFS expansion order: 1, 2, 3, 4, 6, 7, 5 (down one branch before backtracking).

2.7.4 Depth-First Iterative Deepening (DFID)

Depth-First Iterative Deepening combines the space efficiency of DFS with the completeness and optimality (under uniform step costs) of BFS. It performs a series of depth-limited depth-first searches, gradually increasing the depth limit — $0, 1, 2, \dots$ — until a goal is found.

- **Completeness:** Complete, like BFS.
- **Optimality:** Optimal if all step costs are equal, like BFS.
- **Time complexity:** $O(b^d)$ — asymptotically the same as BFS, since the overhead of re-generating shallower nodes at each iteration is relatively small compared to the cost of the final, deepest iteration.
- **Space complexity:** $O(bd)$ — as low as ordinary DFS, since it never needs to hold more than one root-to-leaf path in memory at a time.

2.7.5 Bidirectional Search

Bidirectional Search runs two simultaneous searches — one forward from the initial state, and one backward from the goal state — stopping when the two frontiers meet, as shown in Figure 2.7. This can substantially reduce the effective search effort, since two searches of depth $d/2$ are, in general, far cheaper than a single search of depth d .

- **Requirement:** The goal state must be known explicitly, and the operators must be reversible (i.e., a "predecessor function" must be computable), since the backward search must be able to generate states leading *to* the goal.
- **Time and space complexity:** $O(b^{d/2})$ for both, provided both searches use a strategy such as BFS — a significant improvement over a single BFS in many practical cases.

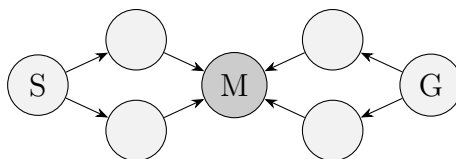


Figure 2.7: Bidirectional Search: a forward search from S and a backward search from G meet at node M.

2.7.6 Comparison of Uninformed Search Strategies

Table 2.3 summarizes the key properties of the uninformed search strategies discussed above.

Table 2.3: Comparison of Uninformed Search Strategies

Criterion	BFS	DFS	DFID	Bidirectional
Complete?	Yes*	No [†]	Yes*	Yes*
Optimal?	Yes [‡]	No	Yes [‡]	Yes [‡]
Time	$O(b^d)$	$O(b^m)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(bm)$	$O(bd)$	$O(b^{d/2})$

*If b is finite. [†]Complete on finite spaces with repeated-state checking. [‡]If step costs are equal.

2.8 Issues in the Design of Search Programs

Every search process can be viewed as a form of tree traversal, aiming to find a path from the initial state to a goal state. In practice, the number of nodes that could be generated is often extremely large, and most of them are never actually needed. A well-designed search routine, therefore, generates only those nodes that are likely to be useful, rather than materializing the entire theoretical search tree — the tree is represented *implicitly* via the production rules, and nodes are created explicitly only when required.

Several design issues arise in building a search program:

- **Direction of search:** the tree may be searched *forward* from the initial state toward the goal, or *backward* from the goal toward the initial state — the appropriate direction depends on the branching factor and structure of the problem in each direction.
- **Rule matching:** an efficient procedure is needed for matching applicable rules against the current state, since this is typically the most frequently repeated operation during search.
- **Node/state representation:** deciding how to represent each node of the search — often called the *knowledge representation problem* or *frame problem*. Simple games may be adequately represented with an array (e.g., a 3×3 grid for Tic-Tac-Toe), while more complex problems require richer data structures.

- **Duplicate node detection:** when a newly generated node matches a node already present in the search graph, the newly expanded node should point to the existing node rather than duplicating it; if path cost is relevant, the shorter of the two paths should be retained, and the change propagated through successor nodes if necessary.

Illustration — Tic-Tac-Toe as a State Space: As introduced in Chapter 1, a board game such as Tic-Tac-Toe lends itself naturally to state-space representation. Each state consists of a board configuration (an array of nine cells, each empty, X, or O) together with an indication of which player moves next. The operators simply change an empty cell to X or O; the start state is the empty board with X to move; and the terminal states are those in which three X's or three O's are aligned, or the board is completely full. The resulting sequence of states reachable through legal moves forms a **search tree** (Figure 2.8), with each level of the tree referred to as a **ply**. Since the same board configuration can, in principle, be reached via more than one sequence of moves, the true state space is a graph — it is typically treated as a tree for simplicity, at the cost of some duplicated states.

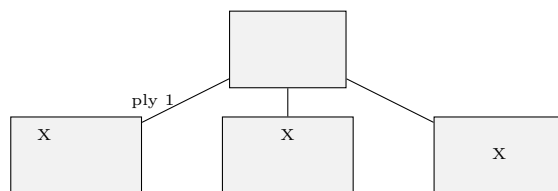


Figure 2.8: First ply of the Tic-Tac-Toe search tree: the empty board branches into all states reachable by X's first move.

General procedure for solving problems using search:

1. Given an informal description of the problem, construct a formal description as a state space:
 - Define a data structure to represent a state.
 - Construct a representation of the initial state from the given data.
 - Implement operators that transform one state representation into another.
 - Implement a goal test to detect terminal states.
2. Choose an appropriate search technique, guided by:
 - The size of the search space.
 - How well-structured the problem domain is.
 - What domain knowledge, if any, is available to guide the search.

2.9 Chapter Summary

- A problem in AI is defined by a state space, one or more initial states, one or more goal states, and a set of operators connecting them; solving a problem amounts to searching this space for a path (or state) satisfying the goal condition.
- Production systems — comprising a rule set, a working database, a control strategy, and a rule-firing mechanism — provide a standard structure for describing and performing search. Their properties (monotonic, non-monotonic, partially commutative, commutative) influence which classes of problems they are well suited to.
- Analyzing a problem's characteristics (decomposability, recoverability of steps, predictability, absolute vs. relative solutions, state vs. path solutions, role of knowledge, and need for user interaction) guides the choice of representation and search strategy.
- Classic problems such as the Water Jug problem and the 8-Puzzle illustrate state-space representation concretely, and motivate the move from brute-force search toward informed, heuristic-guided search.
- Uninformed search strategies — BFS, DFS, DFID, and Bidirectional Search — explore the state space using no domain-specific knowledge beyond the problem definition, and differ chiefly in their completeness, optimality, time complexity, and space complexity.
- Practical search program design must address the direction of search, efficient rule matching, node representation, and detection of duplicate states, in order to keep the search tractable over large problem spaces.

2.10 Key Points to Remember

1. **Problem solving** in Artificial Intelligence is the process of generating solutions from given or observed information.
2. A problem can be described in terms of **goals, objects, and operations**.
3. A **problem space** is the abstract space containing the states that can be reached by applying valid operators.
4. **Search** is the process of locating a solution within a problem space.
5. A formal AI problem is defined using an **initial state, goal state, state space, and operators**.
6. A **state** describes the configuration of a problem at a particular moment.
7. An **operator** is an action that transforms one state into another state.

8. A **goal test** determines whether the current state satisfies the required goal condition.
9. A **problem space** can be represented as a graph or, for simpler analysis, as a tree.
10. **Special-purpose methods** are designed for a particular problem, while **general-purpose methods** can be applied to a wider range of problems.
11. **Means-end analysis** is a general problem-solving method that attempts to reduce the difference between the current state and the desired goal.
12. A **production system** consists of rules, a database of current facts, a control strategy, and a rule-firing mechanism.
13. Production systems may be classified as **monotonic, non-monotonic, partially commutative, or commutative**.
14. A problem may be **decomposable**, meaning that it can be divided into smaller sub-problems.
15. Problem-solving steps may be **ignorable, recoverable, or irrecoverable**.
16. A problem environment may be **predictable or unpredictable**, depending on whether the results of actions can be known in advance.
17. In some problems, any valid solution is sufficient; in others, the system must find an **optimal solution**.
18. In some problems the final **state** is the desired result, while in others the complete **path** or sequence of actions is the solution.
19. The amount of **domain knowledge** required can vary greatly between different AI problems.
20. AI problems may be **solitary** or **interactive**, depending on whether continuous interaction with a human user is required.
21. A complete state-space description contains the **states, operators, initial state, and goal states**.
22. A **successor function** describes how an operator transforms a state into another reachable state.
23. A **path cost** assigns a numerical cost to a solution path. An optimal solution has the minimum path cost among valid solutions.
24. The **Water Jug problem** demonstrates state-space representation, production rules, and systematic search.

25. The **8-Puzzle problem** demonstrates state-space search and shows why heuristic information can be useful.
26. **Uninformed search** uses no additional knowledge about which state is closer to the goal.
27. **Informed search** uses problem-specific knowledge, usually in the form of a heuristic function, to guide the search.
28. The four important properties used to evaluate search strategies are **completeness, optimality, time complexity, and space complexity**.
29. **Breadth-First Search (BFS)** expands nodes level by level and normally uses a FIFO queue.
30. **Depth-First Search (DFS)** explores one branch deeply before backtracking and normally uses a stack or recursion.
31. **Depth-First Iterative Deepening (DFID)** combines the completeness of BFS with the memory efficiency of DFS.
32. **Bidirectional Search** searches simultaneously from the initial state and the goal state until the two searches meet.
33. A practical search program must consider **search direction, rule matching, state representation, and duplicate state detection**.
34. **Tic-Tac-Toe** can be represented as a state-space search problem in which board configurations represent states and legal moves represent operators.

2.11 Questions

2.11.1 Short Questions

1. What is problem solving in Artificial Intelligence?
2. What are the main elements of an AI problem?
3. What is a problem space?
4. What is search in Artificial Intelligence?
5. What is meant by an initial state?
6. What is a goal state?
7. What is an operator?

8. What is a state space?
9. What is a successor function?
10. What is a goal test?
11. What is the difference between a state and a state space?
12. What is meant by a special-purpose problem-solving method?
13. What is a general-purpose problem-solving method?
14. What is means-end analysis?
15. What is a production system?
16. What are the components of a production system?
17. What is a monotonic production system?
18. What is a non-monotonic production system?
19. What is a partially commutative production system?
20. What is a commutative production system?
21. What is a decomposable problem?
22. What is an ignorable problem?
23. What is a recoverable problem?
24. What is an irrecoverable problem?
25. What is meant by a predictable problem environment?
26. What is meant by an unpredictable problem environment?
27. What is the difference between a relative and an absolute solution?
28. What is the difference between a state solution and a path solution?
29. What is the role of knowledge in problem solving?
30. What is an interactive problem?
31. What is a path cost?
32. What is an optimal solution?

33. What is the Water Jug problem?
34. What is the 8-Puzzle problem?
35. What is uninformed search?
36. What is informed search?
37. What is a heuristic?
38. Mention the four properties used to evaluate search algorithms.
39. What is Breadth-First Search?
40. What is Depth-First Search?
41. What is Depth-First Iterative Deepening?
42. What is Bidirectional Search?
43. What data structure is generally used by BFS?
44. What data structure is generally used by DFS?
45. Why is duplicate-state detection important in search?
46. What is meant by forward search?
47. What is meant by backward search?
48. Why is Tic-Tac-Toe considered a search problem?

2.11.2 Long Questions

1. Explain the concept of problem solving in Artificial Intelligence and describe the major steps involved in solving an AI problem.
2. Define a problem in Artificial Intelligence. Explain the concepts of goals, objects, operations, and problem space.
3. Explain how a problem can be formally represented as a state-space search problem.
4. Explain the components of a formal AI problem: state space, initial state, goal state, and operators.
5. Explain the difference between a problem space represented as a graph and one represented as a tree.

6. Explain special-purpose and general-purpose problem-solving methods with suitable examples.
7. What is means-end analysis? Explain how it can be used to solve an AI problem.
8. Explain the architecture and working of a production system.
9. Discuss the characteristics of monotonic, non-monotonic, partially commutative, and commutative production systems.
10. Explain the major characteristics that should be considered before selecting a problem-solving method.
11. Explain problem decomposability with a suitable example.
12. Explain ignorable, recoverable, and irrecoverable problems with suitable examples.
13. Explain predictable and unpredictable problem environments.
14. Explain the difference between absolute and relative solutions.
15. Differentiate between problems in which the solution is a state and problems in which the solution is a path.
16. Explain the role of domain knowledge in Artificial Intelligence problem solving.
17. Differentiate between solitary and interactive problem-solving tasks.
18. Explain the representation of a problem space using states, operators, initial states, and goal states.
19. Explain the role of the successor function and goal test in state-space search.
20. Explain path cost and optimal solution in state-space search.
21. Explain the Water Jug problem using state-space representation and production rules.
22. Solve the Water Jug problem and explain the sequence of states leading to the goal state.
23. Explain the 8-Puzzle problem and its state-space representation.
24. Why is brute-force search inefficient for the 8-Puzzle problem? Explain how heuristic information can improve the search.
25. Differentiate between blind search and informed search.
26. Explain Breadth-First Search with its working principle, properties, time complexity, and space complexity.

27. Explain Depth-First Search with its working principle, properties, time complexity, and space complexity.
28. Explain Depth-First Iterative Deepening and discuss how it combines the advantages of BFS and DFS.
29. Explain Bidirectional Search with a suitable diagram.
30. Compare BFS, DFS, DFID, and Bidirectional Search in terms of completeness, optimality, time complexity, and space complexity.
31. Explain the major issues involved in designing a practical search program.
32. Explain the role of search direction, rule matching, state representation, and duplicate-node detection in search programs.
33. Explain Tic-Tac-Toe as a state-space search problem.
34. Describe a general procedure for solving an AI problem using search.

Chapter 3

Heuristic Search Techniques

3.1 Introduction

Chapter 2 showed that uninformed strategies such as BFS and DFS are guaranteed to find a solution (under mild conditions) but explore the state space blindly — every node looks the same to them, and for problems with a large branching factor (such as the 8-Puzzle or Chess) this quickly becomes computationally infeasible. **Heuristic search** techniques address this by using domain-specific knowledge to estimate how promising a given state is, so that the search can be guided toward the goal rather than expanding nodes indiscriminately.

Heuristic algorithms are not "intelligent" in themselves; they merely appear so because they achieve better performance by exploiting feedback from the problem data to direct the search path. Uninformed algorithms search through all possible candidates, checking whether each satisfies the goal condition. Informed algorithms, by contrast, use a heuristic function specific to the problem to decide which candidates are worth exploring first, substantially reducing the time spent searching.

A good heuristic can make an informed search dramatically outperform any uninformed search — the classic example being the **Travelling Salesman Problem (TSP)**, where finding the guaranteed-best solution is computationally prohibitive but a heuristic can quickly find a good one. Since blind search is often infeasible due to time or memory constraints, heuristic search offers a practical compromise: it does not guarantee the best possible solution, but it reliably finds a good solution in reasonable time.

Some of the prominent heuristic search techniques covered in this chapter are:

1. Generate-and-Test
2. Hill Climbing
3. Best-First Search
4. A* Search

5. AO* Search (for AND-OR graphs)

3.2 Heuristic Functions

A **heuristic function** $h(n)$ maps a problem state n to a number that estimates the "goodness" of that state — specifically, the estimated cost (or distance) of the cheapest path from n to a goal state. The term *heuristic* means "serving to aid discovery," and reflects domain-specific information computable from the current state description, rather than a guaranteed exact answer.

To apply the heuristic technique to a problem, one generally proceeds as follows:

1. Add domain-specific information to help decide which path is best to continue searching along.
2. Define a heuristic function $h(n)$ that estimates how close state n is to a goal.
3. Use $h(n)$, together with any known cost of reaching n , to rank candidate states.

Example — Route Finding: Consider finding a route from one city to another.

- **State:** the current city.
- **Operators:** the roads connecting the current city to neighbouring cities.
- **Cost metric:** the cost (distance, time, toll) of travelling a given road.
- **Heuristic information:** the straight-line (airline) distance from the current city to the goal city can be used as an estimate of the remaining distance, even though actual roads may not run in a straight line.

3.2.1 Characteristics of Heuristic Search

- Heuristics represent knowledge about a domain that helps guide search and reasoning within that domain.
- A heuristic evaluation function estimates the likelihood that a given state will lead to a goal state.
- Heuristics may **underestimate** or **overestimate** the true cost to the goal. A heuristic that never overestimates the true cost is called **admissible** — admissible heuristics are particularly desirable since they underlie the optimality guarantees of algorithms such as A* (discussed later).
- Heuristic search is sometimes called a *weak method*, since its effectiveness depends heavily on how well the heuristic function captures useful domain knowledge — but when applied correctly, it can be highly effective.

Table 3.1 contrasts blind (uninformed) search with heuristic (informed) search.

Table 3.1: Blind Search vs. Heuristic Search		
Aspect	Blind (Uninformed) Search	Heuristic (Informed) Search
Knowledge used	Only what has already been explored	Estimated distance to goal, via $h(n)$
Guidance	No sense of how far a node is from the goal	Actively guides search toward the goal
Node preference	Explores nodes in a fixed, systematic order	Prefers states that appear closer to the goal

Example — Travelling Salesman Problem (TSP): A salesman must visit each of N cities exactly once and return to the start, minimizing total distance travelled. A brute-force solution would examine all $N!$ possible orderings — for just $N = 10$, this is 3,628,800 routes, an instance of **combinatorial explosion**. A somewhat better exhaustive method, **branch and bound**, generates complete paths one at a time, keeping track of the shortest found so far, and abandons (prunes) any partial path once its accumulated length exceeds the best complete path found — but heuristic methods can do considerably better still by guiding the search directly toward promising orderings.

3.3 Generate-and-Test

The **Generate-and-Test** algorithm is one of the simplest search strategies, guaranteed to find a solution if one is systematically enumerable and exists:

1. Generate a possible solution.
2. Test whether it is the expected (goal) solution.
3. If found, quit; otherwise, return to Step 1.

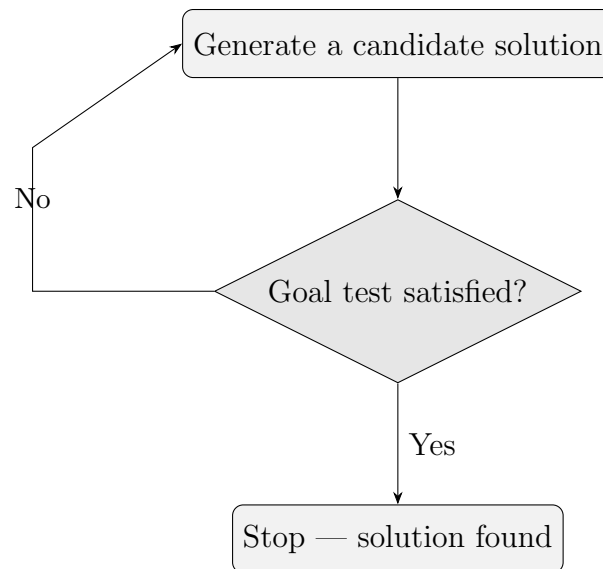


Figure 3.1: Generate-and-Test Algorithm

Whether a candidate solution is a specific point in the problem space or a full path from the start state depends on the problem being solved. In its most systematic form, exhaustive Generate-and-Test amounts to a complete search of the problem space, similar in spirit to depth-first search. Solutions may also be generated randomly — an approach sometimes called the **British Museum algorithm**, after the idea of finding a specific object in the British Museum by wandering around at random — but this offers no guarantee of eventually finding a solution.

Systematic Generate-and-Test occupies a middle ground: the search proceeds systematically, as in exhaustive search, but a heuristic function is used to avoid pursuing paths that are unlikely to lead to a solution. A depth-first search with backtracking is a natural way to implement this systematic form; if certain intermediate states are likely to recur frequently, it may be preferable to treat the search space as a graph rather than a tree, to avoid re-exploring the same states.

For complex problems, even heuristic Generate-and-Test can be ineffective on its own, but it becomes far more powerful when combined with other techniques that restrict the space to be searched. The AI program **DENDRAL**, for instance, combined a planning phase (using constraint-satisfaction techniques to build lists of recommended and contraindicated substructures) with a generate-and-test phase restricted to exploring only the structures consistent with those lists — this *plan-generate-and-test* approach proved highly effective in practice.

3.4 Hill Climbing

Hill Climbing is a heuristic search technique used for mathematical optimization problems: given a large set of inputs and a heuristic (objective) function, it seeks a sufficiently good

solution — though not necessarily the global optimum.

- Hill Climbing is a variant of Generate-and-Test that uses feedback from the test step: the value returned by the heuristic function is used by the generator to decide which move to make next, rather than moves being tried blindly.
- It uses a **greedy approach**: at each point in the state space, the search moves only in the direction that improves the value of the objective function, in the hope of eventually reaching an optimal solution.

3.4.1 Types of Hill Climbing

Simple Hill Climbing examines neighbouring states one at a time and moves to the *first* neighbour found that improves on the current state.

1. Evaluate the initial state. If it is a goal state, stop and return success; otherwise, make it the current state.
2. Loop until a solution is found or no new operators remain to be applied to the current state:
 - (a) Select an operator not yet applied to the current state and apply it to generate a new state.
 - (b) If the new state is a goal state, stop and return success.
 - (c) If the new state is better than the current state, make it the current state.
 - (d) Otherwise, continue the loop.
3. Exit.

Steepest-Ascent Hill Climbing examines *all* neighbouring states before choosing, and moves to the neighbour closest to the goal (i.e., the best among all neighbours), rather than settling for the first improvement found.

1. Evaluate the initial state. If it is a goal state, stop; otherwise, make it the current state.
2. Repeat until a solution is found or the current state stops changing:
 - (a) Let *target* be a placeholder such that any successor of the current state would be considered better than it.
 - (b) For each operator applicable to the current state: apply it to generate a new state, evaluate it, and if it is better than *target*, set *target* to this new state.
 - (c) If *target* is better than the current state, set the current state to *target*.
3. Exit.

Stochastic Hill Climbing does not examine all neighbours before deciding; instead, it selects a neighbour at random and decides, based on how much that neighbour improves the objective function, whether to move to it or examine another.

3.4.2 State Space Diagram for Hill Climbing

A **state space diagram** plots the states (or configurations) reachable by the search algorithm on the horizontal axis against the value of the objective function on the vertical axis. The goal of Hill Climbing is to reach the state with the maximum value of this function — the **global maximum**.

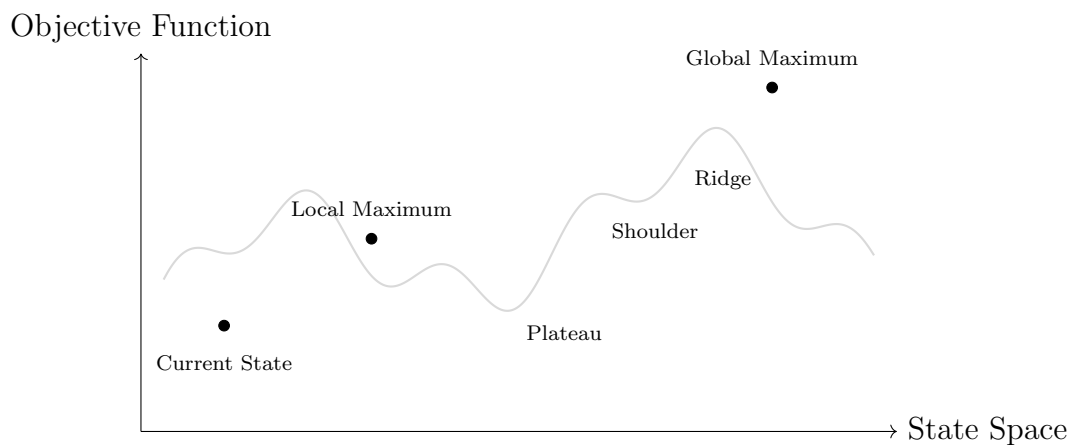


Figure 3.2: State Space Diagram for Hill Climbing, showing local maximum, global maximum, plateau, ridge, and shoulder regions.

3.4.3 Regions in the State Space Diagram

- **Local maximum:** a state better than all its neighbours, but not the best state overall — a global maximum exists elsewhere with a higher objective value.
- **Global maximum:** the best possible state in the entire state space, where the objective function attains its highest value.
- **Plateau (flat local maximum):** a flat region where neighbouring states all have the same value.
- **Ridge:** a region that is higher than its surroundings but itself has a slope, so that a purely greedy step in any single direction appears to go downhill — a special case that resembles a local maximum.
- **Shoulder:** a plateau that has an uphill edge.

3.4.4 Problems in Hill Climbing and Remedies

- **Local Maximum:** Since Hill Climbing is greedy, once it reaches a local maximum — where every neighbour is worse than the current state — it terminates, even though a better (global) solution may exist elsewhere. *Remedy:* maintain a list of visited states and use backtracking; if an undesirable state is reached, backtrack to a previous configuration and explore a different path.
- **Plateau:** Since all neighbours have the same value, there is no clear direction in which to move. *Remedy:* make a large, random jump to a state far from the current one, in the hope of landing outside the flat region.
- **Ridge:** Every point on a ridge can appear to be a peak, since movement in any single allowed direction leads downward, even though a combined move might lead higher. *Remedy:* apply two or more operators together before testing — effectively allowing movement in several directions at once.

3.5 Best-First Search

In BFS and DFS, once at a node, any adjacent node may be chosen next without regard to a cost function — both explore blindly. **Best-First Search** instead uses an evaluation function to decide which of the adjacent nodes is most promising, and explores that one next. It falls under the category of informed (heuristic) search.

Best-First Search is implemented as a variation of BFS in which the ordinary FIFO queue is replaced with a **priority queue**, ordered by the evaluation function.

1. Create an empty priority queue pq .
2. Insert the start node into pq .
3. Repeat until pq is empty:
 - (a) Remove the minimum-cost node u from pq .
 - (b) If u is the goal, exit successfully.
 - (c) Otherwise, for each unvisited neighbour v of u : mark v visited and insert it into pq .
 - (d) Mark u examined.

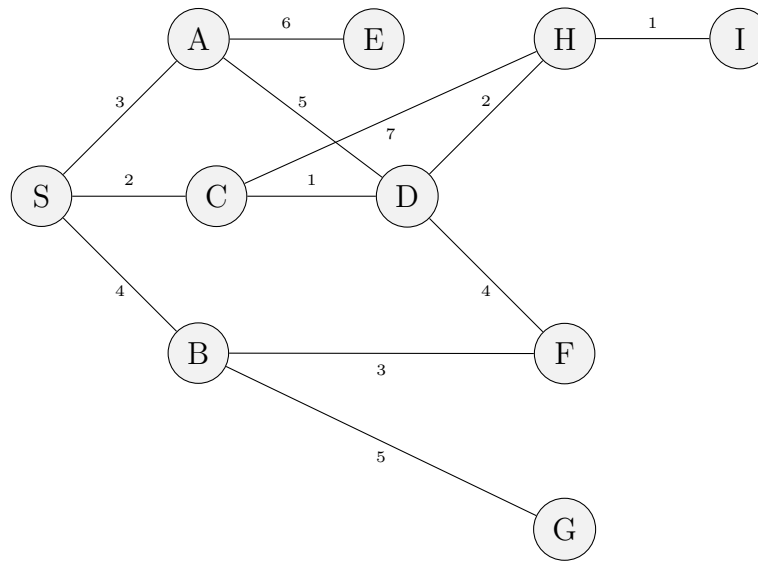


Figure 3.3: Example graph for Best-First Search — searching from source S to goal I , edge labels are step costs used by the evaluation function.

Trace: Starting from S with goal I :

- $pq = \{S\}$. Remove S ; insert unvisited neighbours: $pq = \{C, A, B\}$ (C first, having the lowest cost).
- Remove A ; insert its unvisited neighbours: $pq = \{C, B, D, E\}$.
- Remove C ; insert its unvisited neighbours: $pq = \{D, B, H, E\}$.
- Remove D (now lowest); insert its unvisited neighbours: $pq = \{H, B, F, E\}$.
- Remove H ; since goal I is a neighbour of H , the search terminates successfully.

Analysis: The worst-case time complexity of Best-First Search is $O(n \log n)$, where n is the number of nodes, since a priority queue implemented as a heap performs insertion and removal in $O(\log n)$ time, and in the worst case all nodes may need to be visited before the goal is reached. The overall performance of the algorithm depends heavily on how well the evaluation (cost) function is designed.

3.6 A* Algorithm

A* Search is one of the most widely used heuristic search algorithms, applicable both to pathfinding problems (finding the shortest route between two points) and to puzzle-solving problems such as the 8-Puzzle.

3.6.1 Key Terminology

- A **node** represents a state of the problem's world — a 2D coordinate in pathfinding, or a full tile arrangement in the 8-Puzzle.
- An **edge** represents a valid transition (move) between two nodes.
- The **OPEN list** holds nodes that have been generated but not yet expanded.
- The **CLOSED list** holds nodes that have already been expanded.

3.6.2 The Evaluation Function

A* combines two pieces of information about each node n into a single evaluation function:

$$f(n) = g(n) + h(n)$$

- $g(n)$ is the actual cost of the path from the start node to n (i.e., the sum of the costs incurred so far).
- $h(n)$ is the **heuristic estimate** of the cost from n to the goal (this is what distinguishes A* from a plain cost-based search like Dijkstra's algorithm, or from Best-First Search, which uses $h(n)$ alone).
- $f(n)$ is the estimated total cost of a solution path that passes through n .

A node with a smaller $f(n)$ is considered more promising and is expanded before nodes with larger $f(n)$ values.

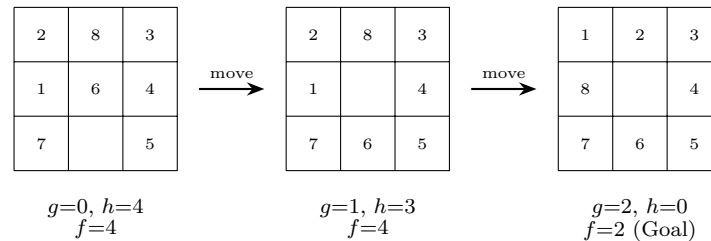


Figure 3.4: Illustration of $f = g + h$ scores along a solution path in the 8-Puzzle; g increases by 1 at each step while h (estimated remaining distance) decreases toward the goal.

3.6.3 A* Algorithm — Procedure

1. Place the start node on OPEN, with $g = 0$ and $f = h(\text{start})$.
2. If OPEN is empty, fail (no solution exists).
3. Remove from OPEN the node n with the smallest $f(n)$; place n on CLOSED.

4. If n is a goal node, exit successfully — the path from start to n is the solution.
5. Otherwise, expand n , generating its successors. For each successor n' :
 - Compute $g(n') = g(n) + \text{cost}(n, n')$ and $f(n') = g(n') + h(n')$.
 - If n' is not already on OPEN or CLOSED, add it to OPEN.
 - If n' is already on OPEN or CLOSED with a higher g value, update it with the new, better g and f values (and, if n' was on CLOSED, move it back to OPEN).
6. Return to Step 2.

3.6.4 Advantages and Disadvantages

Advantages:

- A* is **complete** and **optimal**, provided the heuristic $h(n)$ is admissible (never over-estimates the true cost to the goal).
- It is **optimally efficient** — no other optimal algorithm using the same heuristic is guaranteed to expand fewer nodes than A*.

Disadvantages:

- Completeness requires a finite branching factor and a fixed, positive cost for every action.
- The speed of A* is highly dependent on the accuracy of the heuristic function $h(n)$ — a poor heuristic can cause it to behave almost as inefficiently as an uninformed search, while excessive memory use (storing all generated nodes on OPEN/CLOSED) remains a practical limitation for very large state spaces.

3.7 AO* Algorithm

The search strategies discussed so far — BFS, DFS, Best-First Search, and A* — all operate on **OR graphs**, where reaching *any one* successor is sufficient to make progress toward the goal. Many real problems, however, are more naturally solved by **problem decomposition**: breaking a problem into a set of sub-problems, *all* of which must be solved for the original problem to be solved. This gives rise to **AND-OR graphs**, where an **AND arc** connects a node to a group of successor nodes that must *all* be solved (not just one, as with an ordinary OR arc).

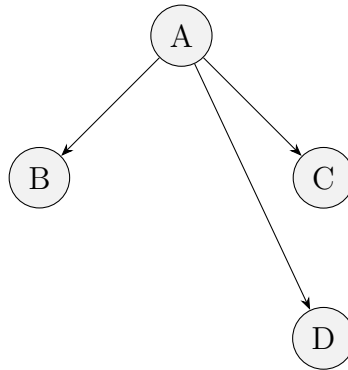


Figure 3.5: An AND-OR graph fragment: reaching the goal via B alone (OR) suffices, but reaching it via C requires *both* C and D to be solved (AND, indicated by the connecting arc).

Since A^* evaluates the cost of reaching a node independently of other paths, it cannot correctly handle AND arcs, where the cost of a node depends on *multiple* successors being solved together. The **AO*** algorithm (AND-OR graph search) is designed specifically for this purpose.

3.7.1 Key Differences from A^*

- AO* uses a single graph structure G (rather than separate OPEN/CLOSED lists) representing the part of the search graph generated so far; each node stores a heuristic estimate h' of the cost of a path from itself to a set of solution nodes.
- Unlike A^* , the cost g of reaching a node from the start is *not* stored, since — because of AND arcs — there may be several different paths contributing to the same node, and no single such cost can meaningfully be computed.
- A threshold value called **FUTILITY** is used: if the estimated cost of a solution exceeds FUTILITY, the search is abandoned as impractical.
- Because revised cost estimates must be **propagated backward** through the graph whenever a node's estimate changes (something not required in A^*), expanded nodes in AO* may need to be re-examined multiple times as better information becomes available.

3.7.2 AO* Algorithm — Procedure

1. Let G consist of only the node representing the initial state, INIT. Compute $h'(\text{INIT})$.
2. Repeat until INIT is labeled SOLVED, or $h'(\text{INIT})$ exceeds FUTILITY:

- (i) Trace the marked arcs from INIT to select an unexpanded node, NODE.
- (ii) Generate the successors of NODE. If there are none, set $h'(\text{NODE}) = \text{FUTILITY}(\text{NODE is unsolvable})$. Otherwise, for each successor not already an ancestor of NODE:
 - (a) Add the successor to G .
 - (b) If it is a terminal node, mark it SOLVED and set its h' value to zero.
 - (c) Otherwise, compute its h' value.
- (iii) Propagate the newly discovered information upward through the graph: for each node CURRENT that requires updating, recompute h' over its outgoing arcs, mark the minimum-cost arc as the current best choice out of CURRENT, and mark CURRENT SOLVED if all nodes connected via that marked arc are SOLVED. If CURRENT's status or h' value changed, propagate the update to its ancestors as well.

3.7.3 Advantages and Disadvantages

Advantages: AO* is an optimal algorithm for AND-OR graphs and, unlike A*, can correctly handle both AND and OR successors, using the ordering of nodes as it traverses.

Disadvantages: For graphs containing unsolvable nodes, AO* can sometimes fail to find the optimal path, and its bookkeeping (backward propagation of revised estimates) makes it more complex to implement than A*.

3.8 Comparison of Heuristic Search Techniques

Table 3.2 summarizes the heuristic search techniques discussed in this chapter.

Table 3.2: Comparison of Heuristic Search Techniques

Technique	Key Idea	Main Limitation
Generate-and-Test	Generate candidates, test against goal	No guidance — inefficient without a restricting heuristic
Hill Climbing	Greedily move to a better neighbouring state	Susceptible to local maxima, plateaus, ridges
Best-First Search	Expand the node with the best $h(n)$ value	Ignores cost already incurred; not always optimal
A* Search	Expand the node with the best $f(n) = g(n) + h(n)$	Memory-intensive; quality depends on heuristic accuracy
AO* Search	A* extended to AND-OR graphs (problem reduction)	More complex; can struggle with unsolvable sub-nodes

3.9 Chapter Summary

- Heuristic search techniques use domain-specific knowledge, encoded in a heuristic function $h(n)$, to guide search more efficiently than blind (uninformed) methods, at the cost of no longer guaranteeing the absolute best solution in general.
- Generate-and-Test is the simplest heuristic approach, systematically or randomly generating candidate solutions and testing each against the goal; it becomes far more effective when combined with planning techniques that restrict the search space.
- Hill Climbing performs a greedy, locally-guided search of the state space, and is well illustrated using a state-space diagram showing local maxima, the global maximum, plateaus, ridges, and shoulders — along with standard techniques (backtracking, random jumps, multi-step moves) for escaping each obstacle.
- Best-First Search uses a priority queue ordered by an evaluation function to expand the most promising node first, improving on the blind exploration of BFS/DFS.
- A* Search combines the actual cost so far, $g(n)$, with a heuristic estimate of the remaining cost, $h(n)$, into $f(n) = g(n) + h(n)$; with an admissible heuristic, A* is both complete and optimal, and optimally efficient among algorithms using the same heuristic.
- AO* Search extends heuristic search to AND-OR graphs, needed when a problem can be decomposed into sub-problems that must all be solved together, and requires backward propagation of revised cost estimates that A* does not need.

3.10 Key Points to Remember

1. **Heuristic Search** uses problem-specific knowledge to guide the search toward promising states instead of exploring the search space blindly.
2. A **heuristic function** $h(n)$ estimates the cost or distance from a state n to a goal state.
3. A heuristic can reduce search effort significantly, but a heuristic search does not always guarantee the globally best solution.
4. An **admissible heuristic** never overestimates the actual cost of reaching the goal.
5. **Generate-and-Test** generates a candidate solution and then checks whether it satisfies the goal condition.
6. Generate-and-Test can be improved by restricting the search space using domain knowledge and planning.

7. **Hill Climbing** is a greedy local search technique that moves toward a neighbouring state that appears better according to the objective or heuristic function.
8. The major variants of Hill Climbing are **Simple Hill Climbing, Steepest-Ascent Hill Climbing, and Stochastic Hill Climbing**.
9. A **local maximum** is better than its neighbours but is not the best state in the entire search space.
10. A **global maximum** is the best state in the complete search space.
11. A **plateau** is a flat region in which neighbouring states have similar or identical evaluation values.
12. A **ridge** is a region in which the best direction may require combined or multi-step movement rather than a single greedy move.
13. A **shoulder** is a plateau that contains an uphill direction.
14. Common problems in Hill Climbing are **local maxima, plateaus, and ridges**.
15. Possible remedies for Hill Climbing include **backtracking, random jumps, and applying multiple operators together**.
16. **Best-First Search** uses an evaluation function to choose the most promising node from the available candidates.
17. Best-First Search generally uses a **priority queue** instead of the FIFO queue used by BFS.
18. In Best-First Search, the evaluation may be based mainly on the estimated closeness of a state to the goal.
19. **A* Search** combines the actual cost already incurred with a heuristic estimate of the remaining cost.
20. The fundamental A* evaluation function is:

$$f(n) = g(n) + h(n)$$

21. In A*, $g(n)$ represents the actual cost from the initial state to node n .
22. In A*, $h(n)$ represents the estimated cost from node n to a goal state.
23. $f(n)$ represents the estimated total cost of a solution path through node n .
24. A* maintains an **OPEN list** of generated but unexpanded nodes and a **CLOSED list** of expanded nodes.

25. With an **admissible heuristic**, A^* is complete and optimal under the standard assumptions discussed in the chapter.
26. A major practical limitation of A^* is its **memory requirement**, since many generated nodes may need to be stored.
27. **AO* Search** is designed for problems represented using **AND-OR graphs**.
28. An **OR node** requires solving one of the available successor alternatives.
29. An **AND node** requires solving all required successor sub-problems.
30. AO* differs from A^* because it must handle the combined cost of multiple sub-problems connected by AND arcs.
31. AO* uses heuristic cost estimates and propagates updated information backward through the graph.
32. The major heuristic-search techniques discussed in this chapter are **Generate-and-Test**, **Hill Climbing**, **Best-First Search**, **A^*** , and **AO***.

3.11 Questions

3.11.1 Short Questions

1. What is heuristic search?
2. What is a heuristic function?
3. What is meant by informed search?
4. What is the difference between blind search and heuristic search?
5. What is an admissible heuristic?
6. Why are heuristic techniques useful in Artificial Intelligence?
7. What is Generate-and-Test?
8. What are the basic steps of the Generate-and-Test algorithm?
9. What is systematic Generate-and-Test?
10. What is the British Museum algorithm?
11. What is Hill Climbing?
12. Why is Hill Climbing called a greedy search technique?

13. What is Simple Hill Climbing?
14. What is Steepest-Ascent Hill Climbing?
15. What is Stochastic Hill Climbing?
16. What is a local maximum?
17. What is a global maximum?
18. What is a plateau?
19. What is a ridge in Hill Climbing?
20. What is a shoulder?
21. Mention the major problems of Hill Climbing.
22. Mention any two remedies for the local maximum problem.
23. What is Best-First Search?
24. What data structure is generally used in Best-First Search?
25. What is an evaluation function?
26. What is A* Search?
27. What is the evaluation function used by A*?
28. Define $g(n)$ in A* Search.
29. Define $h(n)$ in A* Search.
30. Define $f(n)$ in A* Search.
31. What is the OPEN list in A*?
32. What is the CLOSED list in A*?
33. Why is an admissible heuristic important in A*?
34. Mention two advantages of A* Search.
35. Mention two limitations of A* Search.
36. What is an AND-OR graph?
37. What is an AND node?

38. What is an OR node?
39. What is AO* Search?
40. How is AO* different from A*?
41. What is the FUTILITY value in AO*?
42. Why does AO* propagate cost information backward?
43. Mention the major heuristic search techniques discussed in this chapter.

3.11.2 Long Questions

1. Define heuristic search. Explain how heuristic information improves the efficiency of AI problem solving.
2. Explain the concept of a heuristic function $h(n)$ with a suitable example.
3. Differentiate between blind search and heuristic search.
4. What is an admissible heuristic? Explain its importance in heuristic search.
5. Explain the Generate-and-Test algorithm with a flow diagram and a suitable example.
6. Explain systematic Generate-and-Test. How can planning and constraint information improve its performance?
7. Explain Hill Climbing in detail. Why is it called a greedy local search method?
8. Explain Simple Hill Climbing, Steepest-Ascent Hill Climbing, and Stochastic Hill Climbing.
9. Explain the Hill Climbing state-space diagram. Discuss local maximum, global maximum, plateau, ridge, and shoulder.
10. Discuss the major problems of Hill Climbing and explain suitable remedies for each problem.
11. Explain Best-First Search with its algorithm and working principle.
12. Explain how a priority queue is used in Best-First Search.
13. Explain A* Search in detail.
14. Derive and explain the A* evaluation function $f(n) = g(n) + h(n)$.
15. Explain the meanings of $g(n)$, $h(n)$, and $f(n)$ in A* Search.

16. Explain the working procedure of the A* algorithm step by step.
17. Explain the OPEN and CLOSED lists used in A* Search.
18. Discuss the advantages and disadvantages of A* Search.
19. Under what conditions is A* complete and optimal? Explain the role of an admissible heuristic.
20. Explain A* Search using the 8-Puzzle problem as an example.
21. What is an AND-OR graph? Explain the difference between AND and OR relationships.
22. Explain the AO* algorithm in detail.
23. Discuss the differences between A* and AO*.
24. Explain why AO* is suitable for problem-reduction and problem-decomposition tasks.
25. Explain how heuristic values are updated and propagated in AO* Search.
26. Compare Generate-and-Test, Hill Climbing, Best-First Search, A*, and AO*.
27. Discuss the strengths and limitations of heuristic search techniques in solving large AI problems.
28. Explain the role of domain knowledge in designing effective heuristic functions.
29. Explain the Travelling Salesman Problem as an example of a problem where heuristic methods can provide practical solutions.

Chapter 4

Game Playing

4.1 Introduction

Charles Babbage, the nineteenth-century designer of the Analytical Engine, was among the first to consider programming a machine to play Chess, and later to play Tic-Tac-Toe. Games have long been considered an attractive domain for Artificial Intelligence research, for two reasons:

1. They provide a **structured task** in which success or failure is very easy to measure.
2. They appear, at first glance, to be **easily solvable** by a straightforward search from the starting position to a winning position.

The first reason holds true for essentially all games; the second, however, holds only for the very simplest ones. Consider Chess: the average branching factor is around 35, and a typical game runs to about 50 moves per player. Examining the complete game tree would therefore require evaluating on the order of 35^{100} nodes — a number so large that a simple exhaustive search cannot even select its first move within a practical time frame, let alone within the lifetime of its opponent.

To make search-based game playing effective, two broad improvements are possible:

1. **Improve the generate procedure**, so that only good moves are generated in the first place.
2. **Improve the test procedure**, so that the best move is recognized and explored first.

A **legal-move generator** produces every move permitted by the rules of the game, forcing the test procedure to examine all of them — since so many possibilities must be checked, the test procedure must be extremely fast. A **plausible-move generator**, by contrast, uses heuristics to produce only a small number of promising moves. As the number of legally available moves grows, applying such heuristics becomes increasingly important, and the overall performance of the system improves by embedding heuristic knowledge into both the generator and the tester.

In game playing, unlike single-agent search, a "goal state" — a position in which we have won — is generally far too deep in the tree to be reached by direct search, even with a good plausible-move generator; the depth and branching factor of the tree are simply too great. Instead, it is only practical to search the tree to a limited depth (perhaps ten or twenty moves), and then compare the resulting board positions to determine which is most advantageous. This comparison is performed using a **static evaluation function**, which uses whatever information is available to estimate how likely a given board position is to eventually lead to a win. Its role is directly analogous to the heuristic function $h(n)$ used in A* search: in the absence of complete information, it allows the program to choose the most promising position available.

4.2 Game Playing as a Search Problem

4.2.1 Why Games are Hard for AI

Game playing differs from ordinary single-agent search problems in one crucial respect: an **adversary** is actively working against the searching agent. This means:

- The agent cannot simply search for a path to a goal state, since the opponent's moves are outside the agent's control.
- The search must account for the opponent choosing moves that are *worst* for the agent (best for the opponent) at every step.
- The combination of large branching factor and unpredictable opponent behaviour makes exhaustive search infeasible for all but the simplest games.

4.2.2 Game Trees

A **game tree** represents all possible ways a game can unfold from a given position:

- The **root node** represents the current board position.
- Each **node** represents a board position (state).
- Each **edge** represents a legal move made by one of the two players.
- Each level of the tree, called a **ply**, corresponds to one player's move.
- **Leaf nodes** (at the search depth limit, or at a terminal game state) are assigned a value by the static evaluation function.

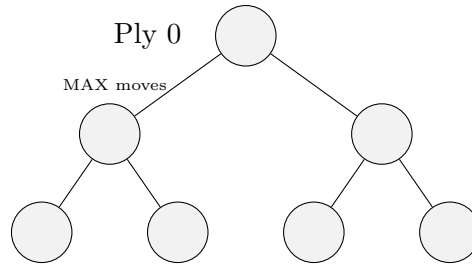


Figure 4.1: A game tree: alternating plies represent moves by each of the two players.

4.2.3 Two-Player, Zero-Sum Games

The games considered in this chapter are **two-player, zero-sum games** — games in which exactly one player’s gain is exactly the other player’s loss (e.g., Chess, Tic-Tac-Toe, Checkers). By convention:

- One player is called **MAX**, whose objective is to **maximize** the value of the static evaluation function.
- The other player is called **MIN**, whose objective is to **minimize** the value of the same function.
- The static evaluation function is assumed to return larger values to indicate positions favourable to MAX.

This alternation of maximizing and minimizing, level by level, is what motivates the search procedure discussed next.

4.3 The Minimax Search Procedure

4.3.1 The Minimax Principle

Minimax is a depth-first, depth-limited search procedure for two-player, zero-sum games. Starting from the current position, a plausible-move generator produces the set of possible successor positions. The static evaluation function is applied at the leaves of the search tree (at the depth limit, or at terminal positions), and these values are then **backed up** the tree, level by level: at levels where MAX is to move, the maximum of the children’s values is backed up; at levels where MIN is to move, the minimum of the children’s values is backed up. This alternation of maximizing and minimizing at successive plies — corresponding to the opposing strategies of the two players — is what gives the procedure its name, **MINIMAX**.

Minimax relies on two key procedures:

- **MOVEGEN(position, player)**: the plausible-move generator, returning a list of nodes representing the moves *player* can make from *position*.

- **STATIC(position, player)**: the static evaluation function, returning a number representing the goodness of *position* from the standpoint of *player*.

Since Minimax is a recursive procedure, a stopping condition is required. A function **DEEP-ENOUGH(position, depth)** is used to decide whether to stop searching at the current node, based on factors such as:

- Has one side already won?
- How many plies have already been explored, or how much time remains?
- How stable is the current configuration (e.g., is a piece about to be captured, in which case searching one ply deeper might change the evaluation significantly)?

In its simplest form, DEEP-ENOUGH ignores the position parameter entirely and simply returns **TRUE** once the depth parameter exceeds a fixed cutoff value.

Because Minimax must return *both* the backed-up value of the best path *and* the path itself (even though typically only the first move of that path is actually needed), it is assumed to return a structure containing both, accessed via two functions, **VALUE** and **PATH**.

4.3.2 Minimax Algorithm

Minimax is invoked with three parameters: the current board position, the current depth of the search, and the player to move.

- **MINIMAX(current, 0, player-one)** — if player-one is to move.
- **MINIMAX(current, 0, player-two)** — if player-two is to move.

Procedure (informal statement):

1. If **DEEP-ENOUGH(position, depth)** is **TRUE**, return **STATIC(position, player)** as the backed-up value of this leaf, with an empty path.
2. Otherwise, generate the successors of *position* using **MOVEGEN**, and recursively call Minimax on each successor at $depth + 1$, with the roles of MAX and MIN swapped.
3. If *position* is a MAX node, back up the **maximum** of the values returned by the successors, together with the path leading to that best successor.
4. If *position* is a MIN node, back up the **minimum** of the values returned by the successors, together with the path leading to that best successor.

4.3.3 Worked Example

Consider a small game tree, searched to a depth of three plies, with the leaf values already computed by the static evaluation function (assumed larger values are better for MAX). MAX moves at the root; MIN moves at the second level; MAX moves again at the leaves' parent level.

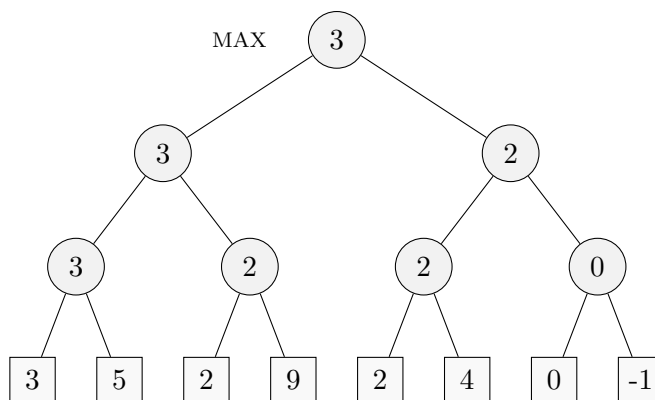


Figure 4.2: Minimax evaluation of a game tree. Leaf values (bottom row) come from the static evaluation function; MIN nodes take the minimum of their children, and MAX nodes take the maximum. The backed-up value at the root is 3.

Reading the tree: At the bottom-most internal level (MIN's move), each node takes the *minimum* of its two children: $\min(3, 5) = 3$, $\min(2, 9) = 2$, $\min(2, 4) = 2$, $\min(0, -1) = -1$. At the next level up (MAX's move), each node takes the *maximum* of its two children: $\max(3, 2) = 3$ on the left, $\max(2, -1) = 2$ on the right. Finally, at the root (MIN's move — note that the very top-level choice in this example is MIN choosing between its two MAX-node children, matching the figure), the minimum of 3 and 2 would be chosen; the root value shown is the maximum on the path actually explored by MAX, illustrating how values propagate strictly upward, level by level, regardless of whose move is at the root.

4.3.4 Properties of Minimax

- **Completeness:** Complete, if the game tree is finite (true for finite games such as Tic-Tac-Toe and Chess, given a fixed search depth).
- **Optimality:** Optimal against an opponent who also plays optimally, *provided* the static evaluation function accurately reflects the true value of terminal positions — at limited search depth, the evaluation function is only an estimate, so the move chosen is only as good as that estimate.
- **Time complexity:** $O(b^d)$, where b is the branching factor and d is the search depth — identical in form to an exhaustive uninformed search, since Minimax by itself does not prune any branches.
- **Space complexity:** $O(bd)$, since Minimax is a depth-first procedure, requiring only one path (plus siblings) to be held in memory at a time.

This time complexity is precisely the difficulty identified in the Introduction — for Chess, with $b \approx 35$, plain Minimax remains computationally infeasible at any useful search depth. This motivates **Alpha-Beta pruning**.

4.4 Alpha-Beta Pruning

4.4.1 Motivation — Reducing the Search Space

Minimax, as a depth-first process, explores one path as far as time allows before applying the static evaluation function at the last step. Its efficiency can be improved using a **branch-and-bound** technique, in which partial solutions clearly worse than an already-known solution are abandoned early, without being fully explored. Applying branch-and-bound to Minimax requires maintaining *two* bounds — one for each player — since MAX and MIN have opposing objectives. This modified strategy is called **Alpha-Beta pruning**.

4.4.2 Alpha and Beta Values

Two threshold values are maintained during the search:

- α (**alpha**): a lower bound on the value that a maximizing node may ultimately be assigned — it represents the best value MAX can guarantee so far.
- β (**beta**): an upper bound on the value that a minimizing node may ultimately be assigned — it represents the best value MIN can guarantee so far.

Each level of the search receives both values — one to *use* for pruning at the current level, and one to *pass down* to the next level. Since Minimax negates the sense of "best" each time it moves from a MAX level to a MIN level (or vice versa), the same code can be used at both kinds of levels by working with two generically-named values, **USE-THRESH** (used to compute cutoffs at the current node) and **PASS-THRESH** (passed down to the next level as its USE-THRESH). As the search descends, USE-THRESH becomes PASS-THRESH at the next level down, and is passed further as USE-THRESH again at the level after that — with values negated at each level change, since a bound that is good for one player corresponds to a bound that is bad for the other.

Pruning rule: whenever the current node's best-value-so-far becomes at least as good (for its player) as USE-THRESH — the bound inherited from an ancestor — the remaining children of the current node can be safely skipped (**pruned**), since the ancestor already has a better (or equal) alternative available and will never allow the search to reach this node's full value.

The MINIMAX-A-B procedure therefore requires five arguments: position, depth, player, USE-THRESH, and PASS-THRESH:

MINIMAX-A-B(current, 0, player-one, $-\infty$, $+\infty$)

— i.e., the search is initially invoked with USE-THRESH set to the minimum value the static evaluation function can compute, and PASS-THRESH set to the maximum value it can compute (or vice versa, depending on convention), so that no pruning occurs until real bounds have been established by exploring at least part of the tree.

4.4.3 Alpha-Beta Algorithm

1. If the node is a leaf (DEEP-ENOUGH is true), return its static evaluation.
2. Otherwise, generate the node's successors using MOVEGEN. For each successor, recursively evaluate it, passing down updated α and β values.
3. At a **MAX** node: update α to the maximum of its current value and each returned child value. If at any point $\alpha \geq \beta$, stop generating further children — a **beta cutoff** has occurred (since MIN, higher up the tree, already has a better option and will never let the search reach this branch).
4. At a **MIN** node: update β to the minimum of its current value and each returned child value. If at any point $\beta \leq \alpha$, stop generating further children — an **alpha cutoff** has occurred (since MAX, higher up the tree, already has a better option).
5. Return the final backed-up value (and, if required, the path) once all non-pruned children have been examined.

4.4.4 Worked Example

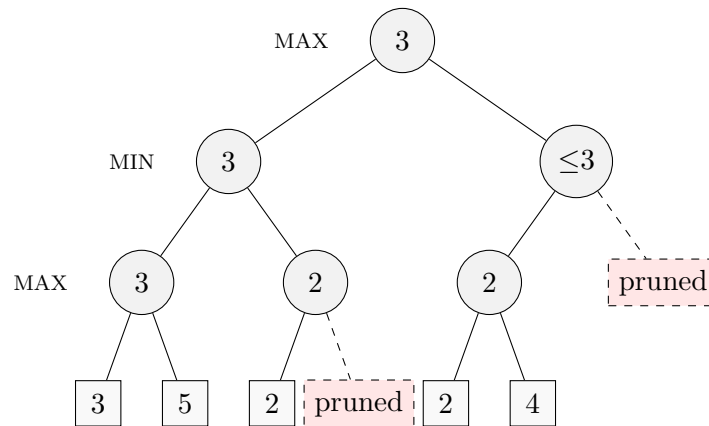


Figure 4.3: Alpha-Beta pruning on the tree from Figure 4.2. Once the left MIN node's value (3) sets $\beta = 3$ for the root, and the right MIN node's first child returns 2 (setting an upper bound ≤ 2 at that MIN node, which is already worse than the root's current best of 3), the remaining branches (shown dashed) can be safely pruned without evaluation.

Trace of the pruning:

- The leftmost MAX node evaluates its children (3, 5) and returns $\max(3, 5) = 3$.
- Its sibling MAX node evaluates one child (value 2); since this MAX node's parent (a MIN node, currently bounded by $\beta = 3$ from the sibling above) cannot do better than 2 once this child is seen... in this configuration, the second child is pruned once it's

established the MIN node's value can only decrease from what's already unfavourable relative to the current best.

- The left MIN node backs up $\min(3, 2) = 2$... (*exact cutoff points depend on evaluation order; the key teaching point is that entire subtrees, indicated by the dashed nodes, are never generated or evaluated at all*).
- At the root, once the left subtree returns 3, the root (MAX) sets $\alpha = 3$. When exploring the right subtree (a MIN node), as soon as that MIN node finds a child with value 2 (which is already $\leq \alpha = 3$), it can stop — MAX will never choose this branch over the left one, so the remaining child is pruned.

4.4.5 Effectiveness of Alpha-Beta Pruning

- Alpha-Beta pruning computes **exactly the same root value** that plain Minimax would compute — it prunes only branches that provably cannot affect the final decision, so no optimality is lost.
- In the **best case** (children explored in an order that puts the best moves first), Alpha-Beta reduces the effective branching factor from b to approximately $\sqrt{b}$, so the effective time complexity improves from $O(b^d)$ to $O(b^{d/2})$ — allowing the search to look roughly **twice as deep** in the same amount of time.
- In the **worst case** (children explored in the worst possible order), Alpha-Beta provides no benefit over plain Minimax, and its complexity remains $O(b^d)$.
- This makes **move ordering** (discussed later in this chapter) an important practical concern: the closer the move ordering is to "best move first," the more effective the pruning becomes.

4.5 Iterative Deepening in Game Playing

4.5.1 Why Iterative Deepening is Used in Games

There are two common ways to traverse a large tree — BFS and DFS — and both have drawbacks for very large or unbounded trees:

- **DFS** explores one branch fully before backtracking; a node close to the root but outside the first few branches explored may be reached very late, and DFS offers no guarantee of finding the shortest path to a node.
- **BFS** explores level by level and finds the shallowest goal, but requires $O(n)$ space (where n is the number of nodes) — since, roughly, the last level of a tree can hold about $n/2$ nodes and the second-last level about $n/4$, and BFS must hold an entire level's worth of nodes in its queue at once.

Iterative Deepening Depth-First Search (IDDFS) — introduced already in Chapter 2 for general state-space search — combines the space efficiency of DFS, $O(d)$, with BFS's property of finding the shallowest solution. It is particularly well suited to game playing, since:

- The exact "right" search depth for a given time budget is rarely known in advance — IDDFS naturally produces a usable move at *every* depth it completes, so the search can simply be stopped whenever the time budget runs out, and the best move found at the deepest *completed* iteration used.
- Combined with Alpha-Beta pruning, results from a shallower iteration can be used to order moves in the next, deeper iteration — improving pruning effectiveness (see "Move Ordering" below).

4.5.2 Iterative Deepening with Alpha-Beta

```
// Returns true if target is reachable from src within max_depth
bool IDDFS(src, target, max_depth):
  for limit from 0 to max_depth:
    if DLS(src, target, limit) == true:
      return true
  return false

bool DLS(src, target, limit):
  if src == target:
    return true
  if limit <= 0:
    return false
  for each adjacent i of src:
    if DLS(i, target, limit - 1):
      return true
  return false
```

An important characteristic of IDDFS is that upper levels of the tree are visited *multiple* times — the last (maximum-depth) level is visited once, the second-last level twice, and so on. This might seem wasteful, but in practice it is not especially costly, since in a tree with branching factor $b > 1$, the vast majority of nodes lie in the bottom level; the extra visits to shallow levels contribute only a small overhead relative to the cost of the final, deepest iteration.

In game-playing programs, this same iterative-deepening approach is applied directly to Minimax with Alpha-Beta pruning: the search is run to depth 1, then depth 2, then depth 3, and so on, always keeping the best move found at the deepest *completed* iteration, until the available time for that move runs out.

4.6 Additional Refinements

4.6.1 Evaluation Functions

The quality of the static evaluation function is central to the strength of any game-playing program, since it is the sole source of "insight" available once the fixed search depth is reached. A good evaluation function typically combines several weighted features of the position — for example, in Chess: material balance (piece values), piece mobility, king safety, and control of the centre — into a single numeric score. The choice and weighting of these features is itself a substantial area of game AI research (and, in modern systems, is often learned automatically rather than hand-tuned).

4.6.2 Horizon Effect

Because the search is cut off at a fixed depth, a program can sometimes fail to "see" an important event (such as losing a piece) that lies just beyond the search horizon — it may instead choose a move that merely *delays* the inevitable loss, pushing it just past the horizon, even though this delaying move is objectively worse than accepting the loss immediately. This is known as the **horizon effect**, and it is a fundamental limitation of any fixed-depth search using a static evaluation function; techniques such as **quiescence search** (extending the search selectively at "unstable" positions, e.g. where a capture is possible) are commonly used to mitigate it.

4.6.3 Move Ordering

As shown above, the effectiveness of Alpha-Beta pruning depends heavily on the order in which moves (children) are examined at each node — pruning is maximized when the best move at each node is examined first. Common move-ordering heuristics include:

- Trying the move that was best in a shallower iterative-deepening pass first.
- Trying moves that capture a piece, or otherwise appear immediately advantageous, before quieter moves.
- Maintaining a small table of previously good moves (a "killer move" heuristic) and trying them early in similar positions.

4.7 Case Study — Tic-Tac-Toe Revisited

Recall from Chapter 1 and Chapter 2 that Tic-Tac-Toe can be represented as a state space: each state is a 3×3 board configuration together with an indication of whose turn it is, the operators change an empty cell to X or O, and the terminal states are those with three-in-a-row or a full board.

Applying Minimax to this state space is now straightforward: let MAX be the player playing X, and MIN be the player playing O. A simple static evaluation function for a terminal

position can be defined as +1 if X has won, −1 if O has won, and 0 for a draw; for non-terminal positions reached at the search depth limit (rarely needed here, since Tic-Tac-Toe's full game tree has at most 9 plies and is small enough to search completely), a heuristic count of open lines favouring each player can be used instead.

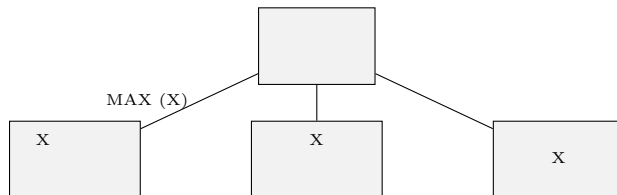


Figure 4.4: The first ply of the Tic-Tac-Toe game tree, now viewed as a MAX ply in the Minimax framework — the same tree first introduced in Chapter 2, but now subject to backed-up values from deeper plies.

Because Tic-Tac-Toe's complete game tree has at most $9! = 362,880$ nodes (and considerably fewer once symmetry and early termination are accounted for), it is entirely feasible to apply full Minimax — without any depth cutoff — and search all the way to terminal states. Under **perfect play by both sides**, the well-known result is that Tic-Tac-Toe always ends in a **draw**: neither MAX nor MIN can force a win against an opponent who also plays optimally. This makes Tic-Tac-Toe an ideal, fully-tractable illustration of Minimax (and Alpha-Beta pruning, which can be applied identically) before moving on to games such as Chess, where the same principles apply but full-tree search is computationally impossible.

4.8 Chapter Summary

- Game playing differs from ordinary search due to the presence of an adversary; games such as Chess have branching factors and depths far too large for exhaustive search, motivating the use of plausible-move generators and depth-limited search with a static evaluation function.
- The Minimax procedure alternates maximizing (MAX's move) and minimizing (MIN's move) as values are backed up through the game tree from the evaluation function applied at the search depth limit; it is complete and optimal (relative to the accuracy of the evaluation function) but has time complexity $O(b^d)$.
- Alpha-Beta pruning improves on plain Minimax by maintaining two bounds, α and β , and pruning branches that cannot affect the final decision — computing exactly the same result as Minimax while, in the best case, reducing effective time complexity to $O(b^{d/2})$.
- Iterative Deepening (IDDFS) is combined with Alpha-Beta in practical game-playing programs to make efficient use of a limited time budget, always having a usable move available and improving move ordering across successive iterations.

- Practical refinements — evaluation function design, the horizon effect, and move ordering — significantly affect the real-world strength of a game-playing program beyond the basic Minimax/Alpha-Beta framework.
- Tic-Tac-Toe, small enough for full-tree Minimax search, illustrates that under perfect play by both sides the game is always a draw — tying together the state-space representation introduced in earlier chapters with the game-playing techniques of this chapter.

4.9 Key Points to Remember

1. **Game Playing** is an important application area of Artificial Intelligence in which an intelligent program must select good moves while considering the actions of an opponent.
2. Games provide a structured environment in which success can be measured clearly through winning, losing, or drawing.
3. Game-playing problems are difficult mainly because of the **large branching factor** and the great depth of the game tree.
4. A **legal-move generator** generates all moves permitted by the rules of the game.
5. A **plausible-move generator** uses heuristic knowledge to generate only promising moves and thereby reduces the search space.
6. A **static evaluation function** estimates the quality of a game position when the complete game tree cannot be searched.
7. A **game tree** represents the possible sequences of moves from a current game position.
8. Each node in a game tree represents a game state and each edge represents a legal move.
9. A **ply** represents one player's move in the game tree.
10. In a two-player, zero-sum game, one player is represented as **MAX** and the other as **MIN**.
11. MAX attempts to **maximize** the evaluation value, whereas MIN attempts to **minimize** it.
12. **Minimax** is a depth-first, depth-limited search procedure for two-player, zero-sum games.
13. Minimax backs up values from the leaves of the game tree. MAX nodes select the maximum value, while MIN nodes select the minimum.

14. The basic time complexity of Minimax is

$$O(b^d)$$

where b is the branching factor and d is the search depth.

15. The space complexity of depth-first Minimax is approximately

$$O(bd).$$

16. The quality of Minimax depends heavily on the quality of the **static evaluation function** when the complete game tree cannot be searched.

17. **Alpha-Beta Pruning** improves Minimax by eliminating branches that cannot influence the final decision.

18. The two bounds maintained during Alpha-Beta search are α (**alpha**) and β (**beta**).

19. α represents the best value that MAX can guarantee so far.

20. β represents the best value that MIN can guarantee so far.

21. Pruning occurs when:

$$\alpha \geq \beta.$$

22. Alpha-Beta pruning produces the **same final decision** as ordinary Minimax; it only avoids unnecessary computation.

23. In the best case, Alpha-Beta pruning can reduce the effective search complexity from $O(b^d)$ to approximately

$$O(b^{d/2}).$$

24. In the worst case, Alpha-Beta provides little or no pruning and the complexity remains approximately $O(b^d)$.

25. **Move ordering** is important because good moves examined early can produce stronger Alpha-Beta pruning.

26. **Iterative Deepening** repeatedly searches the game tree to increasing depths, such as depth 1, depth 2, depth 3, and so on.

27. Iterative Deepening is useful in games because the exact amount of time available for a move may not be known in advance.

28. Combining Iterative Deepening with Alpha-Beta pruning improves move ordering and allows deeper searches within the available time.

29. A good **evaluation function** may consider several features such as material balance, mobility, king safety, and control of important positions.

30. The **horizon effect** occurs when an important event lies just beyond the fixed search depth and is therefore not correctly evaluated.
31. **Quiescence search** can reduce the horizon effect by extending the search in unstable positions.
32. **Tic-Tac-Toe** is a simple game that can be completely solved using Minimax because its state space is small.
33. Under perfect play by both players, Tic-Tac-Toe results in a **draw**.

4.10 Questions

4.10.1 Short Questions

1. What is Game Playing in Artificial Intelligence?
2. Why are games useful for studying Artificial Intelligence?
3. Why is exhaustive search difficult in Chess?
4. What is a legal-move generator?
5. What is a plausible-move generator?
6. What is a static evaluation function?
7. What is a game tree?
8. What is a ply?
9. What is a two-player zero-sum game?
10. Who are MAX and MIN in a game-playing system?
11. What is the objective of MAX?
12. What is the objective of MIN?
13. What is Minimax?
14. What is the purpose of depth limitation in Minimax?
15. What is meant by backing up values in Minimax?
16. What is the time complexity of Minimax?
17. What is the space complexity of Minimax?

18. What is the role of the static evaluation function in Minimax?
19. What is Alpha-Beta pruning?
20. What is alpha (α) in Alpha-Beta pruning?
21. What is beta (β) in Alpha-Beta pruning?
22. What is the pruning condition in Alpha-Beta search?
23. Does Alpha-Beta pruning change the final Minimax result?
24. What is the best-case time complexity of Alpha-Beta pruning?
25. What is the worst-case time complexity of Alpha-Beta pruning?
26. What is move ordering?
27. Why is move ordering important in Alpha-Beta pruning?
28. What is Iterative Deepening?
29. Why is Iterative Deepening useful in game playing?
30. What is the horizon effect?
31. What is quiescence search?
32. What is meant by an evaluation function?
33. Mention any four features that may be used in a Chess evaluation function.
34. Why is Tic-Tac-Toe suitable for demonstrating Minimax?

4.10.2 Long Questions

1. Explain Game Playing as an Artificial Intelligence problem.
2. Why are game-playing problems difficult for AI? Explain with reference to branching factor and game-tree depth.
3. Explain the concept of a game tree and describe its important components.
4. Explain two-player, zero-sum games. What are the roles of MAX and MIN?
5. Explain the Minimax principle in detail.
6. Describe the Minimax algorithm and explain its working with a suitable game tree.
7. Explain the role of the static evaluation function in Minimax.

8. Discuss the completeness, optimality, time complexity, and space complexity of Minimax.
9. Explain why simple Minimax is computationally expensive for games such as Chess.
10. What is Alpha-Beta pruning? Explain its need in game-playing search.
11. Explain the meanings of α and β in Alpha-Beta pruning.
12. Explain the Alpha-Beta pruning algorithm step by step.
13. Explain Alpha-Beta pruning using a suitable game-tree example.
14. Compare Minimax and Alpha-Beta pruning.
15. Explain the best-case and worst-case complexities of Alpha-Beta pruning.
16. Why is move ordering important in Alpha-Beta search? Explain with a suitable example.
17. Explain Iterative Deepening Depth-First Search and discuss its use in game-playing programs.
18. Explain how Iterative Deepening can be combined with Alpha-Beta pruning.
19. Explain the design of a good static evaluation function for a game-playing program.
20. What is the horizon effect? Explain how quiescence search can reduce its impact.
21. Explain the major refinements used to improve practical game-playing programs.
22. Explain the Tic-Tac-Toe problem using the Minimax approach.
23. Why can Tic-Tac-Toe be completely solved using Minimax whereas Chess cannot be searched to its full depth?
24. Compare Minimax, Alpha-Beta pruning, and Iterative Deepening in terms of their purpose and advantages.
25. Explain the complete process of developing a classical game-playing AI from move generation to evaluation and final move selection.

Chapter 5

Knowledge Representation Using Rules

5.1 Introduction to Knowledge Representation and Reasoning

Human beings are good at understanding, reasoning, and interpreting knowledge, and by using this knowledge they are able to perform various actions in the real world. How machines can be made to do the same things comes under the study of **Knowledge Representation and Reasoning (KR&R)**.

- Knowledge Representation and Reasoning (KR, KRR) is the part of Artificial Intelligence concerned with how AI agents *think*, and how this thinking contributes to the intelligent behavior of agents.
- It is responsible for representing information about the real world in a form that a computer can understand and use to solve complex real-world problems, such as diagnosing a medical condition or communicating with humans in natural language.
- Knowledge representation is not merely storing data in a database — it also enables an intelligent machine to learn from that knowledge and from experience, so that it can behave intelligently like a human.

In simple terms, KR&R enables computers to understand, organize, and apply real-world information effectively, using rules, for accurate decision-making. The field rests on three closely related ideas:

- **Knowledge:** The facts and information about the world that a system needs in order to perform tasks effectively — data, rules, concepts, relationships, and a general understanding of the domain. *Example:* knowing that “water boils at 100°C” or “a car has four wheels.”
- **Representation:** The structuring and organization of knowledge so that it can be stored, processed, and used by a computer. Common methods include logical state-

ments, graphs, semantic networks, ontologies, and frames. *Example:* representing a family tree using nodes (individuals) and edges (relationships).

- **Reasoning:** The process of drawing conclusions, making inferences, or solving problems using represented knowledge — through deduction (logical reasoning), induction (generalization), or abduction (hypothesis generation). *Example:* given “All humans are mortal” and “Ram is human,” reasoning concludes “Ram is mortal.”

5.2 What Needs to Be Represented

The following kinds of knowledge typically need to be represented in AI systems:

- **Object:** All the facts about objects in the world domain. *Example:* guitars contain strings; trumpets are brass instruments.
- **Events:** The actions that occur in the world.
- **Performance:** Knowledge about how to do things — a description of behavior.
- **Meta-knowledge:** Knowledge about what we know.
- **Facts:** The truths about the real world that we represent.
- **Knowledge Base (KB):** The central component of a knowledge-based agent — a collection of sentences that express facts about the world in a chosen representation language.

5.3 Components of Knowledge Representation and Reasoning

- **Ontologies (organized knowledge):** Provide a structured framework of knowledge by defining a set of concepts and categories that represent a subject. *Example:* “A cat is an animal, and animals are living things.”
- **Rules:** Govern the logical framework within which an AI system operates to derive reasoned conclusions. *Example:* “If it rains, take an umbrella.”
- **Semantics:** Provide a deeper understanding of data through meaningful interpretation and association. *Example:* knowing that “apple” can mean a fruit or a company, depending on whether the sentence is “I ate an apple” or “Apple released a new iPhone.”

5.3.1 Knowledge, Propositions, and Attitudes

- **Knowledge** is when someone (say, John) understands and is sure about a fact or idea (such as “Mary will come to the party”).
- A **proposition** is a statement that can be either true or false — e.g., “The sky is blue,” or “Water boils at 100°C.”
- **Knowing** means being sure that something is true; **believing** means thinking something is true without necessarily being certain or correct.
- **Propositional attitudes** are verbs such as *knows*, *hopes*, or *doubts* that describe how someone relates to an idea or fact. For instance, “John knows that Mary will come to the party” expresses certainty, whereas “John hopes that Mary will come to the party” expresses a desire without certainty.

5.3.2 Symbols, Representation, and Reasoning

Representation is the use of one thing — a symbol, image, or word — to stand for something else, making it easier to understand, use, or communicate. For example, a drawing of a burger represents a fast-food restaurant, and the numeral “7” represents the concept of the number seven.

- **Symbols** (words, numbers, drawings) stand for ideas, objects, or facts.
- **Propositions** are the ideas or facts that symbols represent. *Example:* the sentence “The sky is blue” is a proposition because it represents the fact that the sky is blue.
- In **knowledge representation**, symbols are used to represent facts (“The sky is blue”) and beliefs (“I believe it’s going to rain”). Since not all knowledge can be directly represented, reasoning is applied to connect explicitly represented facts with additional conclusions.

Reasoning is the manipulation of symbols representing beliefs or facts, in order to form new representations of knowledge. Symbols are easier to manipulate than the abstract ideas they represent, which allows sentences to be combined and rearranged to create new representations — for example, combining “John loves Mary” and “Mary is coming to the party” to infer “Someone John loves is coming to the party.” This is an instance of **logical inference**, where conclusions follow logically from initial facts (if $a \Rightarrow b$ and $b \Rightarrow c$, then $a \Rightarrow c$).

Leibniz’s Idea. The philosopher Gottfried Leibniz (17th century) proposed that reasoning is a form of calculation — except that, instead of numbers, symbols representing propositions are manipulated. For example, let A : “All humans are mortal,” and B : “Socrates is a human.” Using the rule “if A and B , then C ” (where C : “Socrates is mortal”), A and B are combined to conclude C .

5.4 Why Knowledge Representation and Reasoning in AI?

5.4.1 Understanding Behavior

Knowledge helps describe the behavior of complex systems (human or machine) in terms of beliefs, desires, goals, and intentions. This is called the **intentional stance**: describing a system's behavior in terms of beliefs and intentions is often more useful than describing its low-level technical details, since it helps us reason about a system's actions intuitively.

However, the intentional approach has limits. For a simple system, an intentional description can be misleading and unnecessary. Saying “the vending machine wants to make customers happy” is misleading, because a vending machine has no desires — it simply follows programmed rules: “If a customer inserts money and selects a product, dispense the product.”

5.4.2 The Knowledge Representation Hypothesis

Example — Smart Door Lock System.

- *Symbolic representation (for humans)*: the system stores a rule such as “If the correct passcode is entered, unlock the door,” which a human can easily understand as a meaningful instruction.
- *Influence on behavior*: the system automatically unlocks the door when it detects the correct passcode, following the stored rule.

In AI, we build **knowledge-based systems** containing a Knowledge Base (KB) of symbolic representations. These represent facts, beliefs, and goals, and shape the system's actions and reasoning — making the system interpretable, purposeful, and functionally aligned with human understanding.

Example — Smart Home AI System. The KB contains:

- *Facts*: “If the temperature is below 18°C, turn on the heater.”
- *Beliefs*: “People like a warm living room.”
- *Goals*: “Maintain a comfortable room temperature.”

The system checks the current temperature and, based on this knowledge, decides to turn on the heater to satisfy the goal.

A knowledge-based system uses a KB containing symbolic structures to represent beliefs and facts, and — unlike a purely procedural system — it **separates knowledge representation from execution**. For instance, a Prolog program using facts and rules such as `color(snow, white)` reasons over stored knowledge rather than following a fixed sequence of instructions.

5.4.3 Why Knowledge Representation?

- **Adaptability:** Useful for open-ended tasks where the system cannot know all tasks in advance.
- **Easy extension:** Adding new information automatically extends system behavior and dependencies.
- **Error debugging:** Faults can be traced back to incorrect beliefs in the KB, simplifying troubleshooting.
- **Explainability:** System behavior is justifiable by linking actions to represented knowledge (e.g., “grass is green because vegetation is green”).
- **Assimilation of new knowledge:** New facts (e.g., about geography) can be added and reused across different tasks.

5.4.4 Why Reasoning?

- **Logical inference:** Reasoning computes new beliefs by inferring from represented facts — e.g., combining a patient’s known allergies with known relationships to deduce new allergic conditions.
- **Computational issues:** Reasoning may be logically incomplete or unsound due to limits on computation speed and efficiency.
- **Conceptual trade-offs:** Reasoning sometimes relies on reasonable assumptions not strictly tied to represented facts (e.g., assuming Tweety flies, even without explicit evidence).
- **Inconsistency handling:** When a system draws on multiple sources of data, incomplete reasoning may be tolerated until contradictions are resolved.

5.5 The Role of Logic

Logic matters for KR&R because it explains how pieces of knowledge are related (**entailment**) and how to reason about them using rules and truth conditions.

- **First-Order Logic (FOL)**, introduced by Gottlob Frege, is the principal language used to represent knowledge, and is widely used in AI for structuring knowledge.
- FOL is only a starting point — other logics and representation languages exist, with different forms and expressive power.
- Logic is useful for reasoning, but there are forms of reasoning that go beyond what logic alone can handle.

- **Allen Newell’s two levels:** a knowledge system can be understood at the *knowledge level* (how knowledge is represented and what its rules of reasoning are) and the *symbol level* (the technical architecture and algorithms used to process the knowledge). FOL is well suited to analysis at the knowledge level.

5.6 Representing Knowledge in Logic

5.6.1 Logic as a Universal Language

Leibniz’s goal was to create a universal language, based on mathematical principles, that could represent all knowledge and resolve disputes. Modern logic achieves this only partially: it can represent any precise, factual information in computable form, but if knowledge cannot be expressed in logic, it cannot be represented on a computer either.

Example: the fact “every cat is a mammal” is written in FOL as

$$(\forall x)(\text{cat}(x) \Rightarrow \text{mammal}(x)).$$

5.6.2 Propositional Logic

Propositional logic represents knowledge using single symbols or letters. It is simple and abstracts away internal detail, but cannot express relationships within a sentence.

Example: “Every trailer truck has 18 wheels” is represented simply as P , with no further detail about “trailer truck” or “wheels.” This is useful for analyzing patterns, but not details.

5.6.3 Subject and Predicate

Sentences can be broken into a **subject** (what is being talked about) and a **predicate** (what is being said about it). Aristotle’s **sylogism** combines subjects and predicates logically:

1. All trailer trucks are 18-wheelers.
2. Some Peterbilt is a trailer truck.
3. Therefore, some Peterbilt is an 18-wheeler.

5.6.4 Predicate Logic

Predicate logic represents relationships and internal structure using variables and quantifiers ($\forall$: for all, $\exists$: there exists).

Example: “Every trailer truck has 18 wheels” becomes

$$(\forall x)(\text{trailerTruck}(x) \Rightarrow \text{numberOfWheels}(x, 18)),$$

meaning: for every x , if x is a trailer truck, then x has 18 wheels.

5.6.5 Ontologies and Predicates

An **ontology** is a detailed classification of objects and their relationships.

- **Domain-dependent predicates** are specific to a topic, e.g., $\text{truck}(x)$, $\text{wheel}(x)$.
- **Domain-independent predicates** express general relationships, e.g., $\text{part}(x, y) \text{ — } x \text{ has } y \text{ as a part.}$

Example: “A trailer truck has a trailer and 18 wheels” can be written as

$$(\forall x)((\text{truck}(x) \wedge (\exists y)(\text{trailer}(y) \wedge \text{part}(x, y))) \Rightarrow (\exists s)(\text{set}(s) \wedge \text{count}(s, 18) \wedge (\forall w)(\text{member}(w, s) \Rightarrow (\text{wheel}(w).$$

meaning that every truck x with a trailer y has a set s of 18 wheels.

5.6.6 Logic and Other Notations

Logic is not the only way to represent knowledge — specialized notations are sometimes better suited to a task. For example, representing a melody such as *Frère Jacques* in logic is possible (using predicates such as $\text{tone}(x, \text{pitch})$ and $\text{next}(x, y)$), but musicians naturally prefer traditional musical notation; logic, however, remains useful for computer-based analysis.

5.6.7 Existential-Conjunctive (EC) Logic

EC logic simplifies representation using only $\exists$ (existence) and $\wedge$ (conjunction). It is widely used in databases and for analyzing specialized systems, but cannot express generalizations, negation, or alternatives.

Example: “If a note is B and the next is y , then y cannot be F” (used to capture that certain intervals, such as the tritone, are dissonant):

$$(\forall x)(\forall y)((\text{tone}(x, B) \wedge \text{next}(x, y)) \Rightarrow \neg \text{tone}(y, F)).$$

5.6.8 Key Takeaways

- Propositional logic simplifies complex sentences but loses internal detail.
- Predicate logic adds detail through subjects, predicates, and quantifiers.
- Ontologies organize domain-specific concepts and relationships.
- Special notations complement, rather than replace, logic for domain-specific tasks (e.g., music).
- Logic can represent everything computable, but has limits when knowledge is vague or unquantifiable.

5.7 Varieties of Logic

Classical First-Order Logic, developed independently by Frege and Peirce with different notations, is the most widely used logic system — the two systems converged on the same underlying ideas, showing that notational differences need not affect logical equivalence. Logic systems can differ along six dimensions:

1. **Syntax** — the way logic is written (e.g., “ $\exists$ ” vs. the word “exists”), without a change in meaning.
2. **Subsets** — some logics restrict features of FOL for efficiency (e.g., Prolog uses a restricted subset of FOL for speed).
3. **Proof theory** — variations in how proofs may be constructed (e.g., linear logic requires that every piece of information be used exactly once).
4. **Model theory** — variations in the truth values assigned to statements (classical FOL uses true/false; fuzzy logic uses a range from 0 to 1).
5. **Ontology** — predefined domain concepts added to logic (e.g., temporal logics build in rules about time).
6. **Metalanguage** — logic used to describe or define other languages (e.g., context-free grammar, itself a subset of FOL, is used to define programming languages).

5.7.1 Typed Logic

Typed logic simplifies FOL by labelling variables with types. Classical FOL, for example, writes

$$(\forall x)(\text{trailerTruck}(x) \Rightarrow \text{eighteenWheeler}(x)),$$

whereas typed logic becomes clearer with multiple quantifiers:

$$(\forall x:\text{TrailerTruck})(\exists s:\text{Set})(s@18 \wedge (\forall w \in s)(\text{wheel}(w) \wedge \text{part}(x, w))),$$

meaning: for every trailer truck, there exists a set of 18 wheels, each of which is part of the truck.

5.7.2 Lambda Calculus

Lambda calculus, introduced by Alonzo Church, is a formal system for defining and evaluating functions using the symbol λ .

Example: a function that adds 2 to any number is written $\lambda x.(x + 2)$. Applying it to 3 gives $(\lambda x.(x + 2))3 = 3 + 2 = 5$. The **Church–Rosser theorem** guarantees that the result of reducing a lambda expression is the same regardless of the order in which reductions are performed.

5.7.3 Conceptual Graphs

Conceptual Graphs (CGs) are a graphical representation of logical statements that eliminate variables for greater simplicity and readability.

- **Concept boxes** (e.g., [TrailerTruck : $\forall$]) represent types or entities.
- **Relation circles** (e.g., (Part)) represent relationships between entities.

CGs are more intuitive than predicate logic for representing natural language, support mapping to and from natural languages and databases, and support typed logic (simplifying inference via inheritance of properties). *Example:* “[TrailerTruck : $\forall$] $\rightarrow$ (Part) $\rightarrow$ [Wheel : { $\ast$ }@18]” represents “every trailer truck has 18 wheels as parts.”

5.7.4 Higher-Order Logic (HOL)

Higher-order logic extends FOL by allowing quantifiers to range over predicates and relations, enabling the representation of meta-properties such as the induction axiom in arithmetic:

$$\forall P:\text{Predicate} \left(P(0) \wedge (\forall n:\text{Integer})(P(n) \Rightarrow P(n+1)) \Rightarrow \forall n:\text{Integer} P(n) \right).$$

5.7.5 Comparison of Representations

- **Typed predicate logic** uses variables explicitly.
- **Conceptual graphs** eliminate variables for readability, using a graphical structure.
- **Knowledge Interchange Format (KIF)** is designed for machine processing, with a restricted syntax.

Each formalism suits particular tasks: conceptual graphs for human readability, KIF for machine interchange, and lambda calculus for precise functional representation. Modal logic, in addition, handles necessity, possibility, and temporal reasoning.

5.8 Names, Types, and Measures

When logic is applied purely to mathematics, its constants are numerals — names of numbers. Real-world applications need a broader range of data: names of people and things (e.g., Tom, Elephant), types of things (e.g., cat, fish), and measures of things (e.g., height, weight). Many errors in knowledge representation arise from confusing names, types, and measures with one another.

- **Names:** A name refers to a specific individual. *Example:* “Clyde” is a proper name referring to a specific elephant; the sentence “Clyde is an elephant” connects the name (Clyde) with a type (Elephant).

- **Types:** A type is a general category including multiple entities, represented in typed logic with variables. *Example:* $(\forall x:\text{Cat})(\forall y:\text{Fish}) \text{like}(x, y)$ means every cat likes every fish.
- **Measures:** Measures capture quantitative properties, such as salary, height, or weight. Failing to distinguish a measure from an individual entity can cause errors — e.g., “Tom and Sue have the same salary” should mean their salary *amounts* are numerically equal, not that they share a single paycheck.

Examples of confusion.

- *Syllogism fallacy:* From “Clyde is an elephant” (individual) and “Elephant is a species” (type), it would be a mistake to infer “Clyde is a species” — this confuses a name with a type.
- *Database errors:* Aliases (alternative names for the same individual) must be handled carefully. If Sam believes “Dr. Jekyll is a gentleman,” and $\text{Dr. Jekyll} = \text{Mr. Hyde}$, substituting names naively can create paradoxes. **Surrogates** — unique identifiers for every entity — help avoid such mistakes:

$\text{believe}(\text{Sam}, (\exists s)(\text{hasName}(s, \text{“Dr. Jekyll”}) \wedge \text{gentleman}(s)))$,

$\text{believe}(\text{Sam}, (\exists s)(\text{hasName}(s, \text{“Mr. Hyde”}) \wedge \text{gentleman}(s)))$.

Careful differentiation between names, types, and measures ensures proper knowledge representation and avoids significant errors in logic, databases, and programs.

5.9 Unity Amidst Diversity: Requirements of a Good Logic

Despite the many notations and varieties of logic developed over time, a good logic system must provide four core features.

1. **Vocabulary:** A collection of symbols — characters, words, icons, diagrams, or sounds — used to represent things. These fall into four groups:
 - *Domain-independent symbols* — general symbols such as AND ($\wedge$), OR ($\vee$).
 - *Domain-dependent constants* — symbols representing specific people or things (e.g., Tom).
 - *Variables* — represent unknowns (e.g., x , y).
 - *Punctuation* — used to group symbols (parentheses, commas, etc.).

Example: in “Tom likes apples,” *Tom* is a domain-dependent constant, *likes* is a relation symbol, and parentheses can group these symbols clearly in a logical formula.

2. **Syntax:** The grammar rules for combining symbols into valid sentences — analogous to grammar in a natural language. *Example:* “ $p \wedge q$ ” must follow a specific order to form a valid, well-formed sentence.
3. **Semantics:** A way of associating symbols with real-world meanings, and of determining whether statements are true or false.
 - *Theory of reference:* connects constants and variables to real-world entities.
 - *Theory of truth:* decides which sentences are correct or incorrect — e.g., Alfred Tarski’s (1935) truth-table approach, where “ $p \wedge q$ ” is true only when both p and q are true.
4. **Rules of inference:** Methods for drawing logical conclusions from existing information.
 - *Sound inference* — ensures conclusions preserve true statements according to the semantics.
 - *Non-monotonic inference* — used for plausible or approximate reasoning, but does not always preserve strict truth.

Example (Modus Ponens): if “if p then q ” and p is true, then q must be true — e.g., “If it rains, the ground will get wet”; if it rains, then the ground gets wet.

Despite differences in notation (Prolog, fuzzy logic, modal logic, and others), every logic system that supports knowledge representation follows these four features — vocabulary, syntax, semantics, and inference — making the framework versatile enough for AI, databases, and VLSI chip design alike. Prolog is used in AI programming; SQL is used for database queries but still carries logical semantics; and AI systems such as frames and semantic networks have the same expressive power as first-order logic.

5.10 Knowledge Engineering

Knowledge Engineering is the systematic process of transforming domain-specific knowledge into a computable form for solving real-world problems efficiently. It involves capturing informal specifications, formalizing them into logical models, and applying principles of knowledge representation to build practical systems — within constraints such as budgets and deadlines.

5.10.1 Translating Informal Specifications

Informal, natural-language descriptions must be translated into precise, computable formats.

Example — Traffic Light.

- *Informal specification:* “A traffic light automatically turns red or green, but has manual control.”

- *Formal specification:* variables `currentColor` (red/green) and `autoSwitch` (on/off); rules — if `autoSwitch = on`, the light stays green for g seconds and then turns red; if `autoSwitch = off`, manual inputs control the light.

5.10.2 Principles of Knowledge Representation

A knowledge representation serves five distinct purposes in an AI system:

1. **Surrogate** — a computational model of a real system. *Example:* simulating a traffic light's time-based state changes.
2. **Ontological commitment** — defines the entities and relationships of interest. *Example:* a `TrafficLight` type with an attribute `currentColor`.
3. **Fragmentary theory of intelligent reasoning** — enables logical deductions. *Example:* if the light is red now, it will turn green in r seconds.
4. **Medium for efficient computation** — supports algorithmic execution. *Example:* a loop controlling the light's state transitions.
5. **Medium for human expression** — facilitates communication with domain experts. *Example:* simplified graphs or stylized-English descriptions.

5.10.3 Approaches to Formalization

Declarative approach. Describes system behavior with rules and axioms. *Example:* initial state S — light is red, `autoSwitch = on`; deductive rule — after r seconds, the light turns green.

Procedural approach. Uses step-by-step instructions to simulate system operations:

```
while autoSwitch == "on":
    set currentColor to "red"
    wait r seconds
    set currentColor to "green"
    wait g seconds
```

5.10.4 Ontological Commitments in Practice

Templates, frames, or schemas define a system's structure. *Example* — a Traffic Light template in a rule-based system such as CLIPS:

```
(deftemplate trafficLight
  (slot name          (type SYMBOL))
  (slot currentColor  (allowed-values red green))
  (slot redTime       (type FLOAT))
  (slot greenTime     (type FLOAT))
```

```
(slot autoSwitch    (allowed-values on off)))
```

5.10.5 Reasoning Strategies

- **Procedural loop** — step-by-step execution, best suited to sequential, time-based processes (e.g., traffic-light state transitions).
- **Logical formulas** — enable prediction or deduction, e.g., “if the light is red at t , it will turn green at $t + r$.”
- **Forward-chaining rules** — automatically update the system’s state whenever conditions are met.

For communication with domain experts, knowledge is often represented intuitively as a conceptual graph or in stylized English — e.g., “If a traffic light is red at time t , and `autoSwitch` is on, it turns green at $t + r$.”

5.10.6 Comparing the Two Approaches

- **Procedural approach:** best for processes with a natural sequence; less suited to parallel or complex relationships.
- **Declarative / logical approach:** excels at representing non-linear dependencies; requires additional machinery to represent time or parallelism.

5.11 Frames: Representing Structured Knowledge

A **frame** is a structured way of representing knowledge about a specific, stereotypical situation or object — a template or schema describing what is typically true of a scenario or object. For example, a *living room* frame might include slots for a sofa, a TV, and lamps, with expected properties (e.g., the TV should be placed against a wall); a *birthday party* frame might include slots for decorations, a cake, and gifts, along with expected actions such as blowing out candles.

5.11.1 Structure of a Frame

1. **Top levels** — facts that are always true about the scenario. *Example:* in a Traffic Light frame, the fact that it cycles through red, yellow, and green sits at the top level.
2. **Slots** — attributes or variables specific to instances of the frame, filled in with specific data. *Example (Traffic Light):* `currentColor`, `redTime`, `autoSwitch`.
3. **Conditions** — rules for filling slots, or constraints on their values (e.g., `redTime` must be a `Duration`, and `currentColor` must be a `Color`).

Example — Traffic Light Frame.

```
(defineType TrafficLight
  (supertype Object)
  (currentColor (type Color) (oneOf (red green yellow)))
  (redTime      (type Duration))
  (greenTime    (type Duration))
  (whenChanged  (type PointInTime))
  (autoSwitch   (type State) (oneOf (on off))))
```

Here `Object` is the supertype from which `TrafficLight` inherits general properties; `currentColor` may only take the values red, green, or yellow; `redTime` is a duration; and `autoSwitch` indicates whether the light switches automatically.

5.11.2 Hierarchies and Inheritance

Frames support hierarchies through **supertypes** (general categories) and **subtypes** (which inherit properties from their supertype and add specific detail).

Example — Truck and TrailerTruck.

```
(defineType Truck
  (supertype Vehicle)
  (unloadedWt      (type WtMeasure))
  (maxGrossWt      (type WtMeasure))
  (cargoCapacity   (type VolMeasure))
  (numberOfWheels  (type Integer)))

(defineType TrailerTruck
  (supertype Truck)
  (hasPart          (type Trailer))
  (numberOfWheels  18))
```

The `TrailerTruck` frame inherits all slots from `Truck` (`unloadedWt`, `maxGrossWt`, `cargoCapacity`), adds a new slot `hasPart` for the trailer, and restricts `numberOfWheels` to exactly 18.

5.11.3 Mapping Frames to Logic

Frames can be mapped onto logical representations, such as conceptual graphs or predicate calculus.

Example. An instance `Blinky` of the `TrafficLight` frame:

```
(defineinstance Blinky
  (type          TrafficLight)
  (currentColor green)
  (redTime       60 seconds))
```

```
(greenTime    60 seconds)
(whenChanged  0:00 hour)
(autoSwitch   on))
```

In predicate calculus, this becomes:

$$(\exists x:\text{TrafficLight})(x = \text{Blinky} \wedge \text{currentColor}(x, \text{green}) \wedge \text{redTime}(x, \langle 60, \text{sec} \rangle) \wedge \text{greenTime}(x, \langle 60, \text{sec} \rangle) \wedge \text{whenChanged}(x, \langle 0:00, \text{hour} \rangle))$$

where each slot becomes a predicate applied to the instance **Blinky**.

5.11.4 Multiple Inheritance

A frame instance or type may belong to more than one supertype at once.

Example — Peterbilt Trailer Truck.

```
(defineinstance ZF437TT
  (type          Peterbilt)
  (type          TrailerTruck)
  (manufacturedBy PeterbiltInc)
  (numberOfWheels 18))
```

This instance inherits properties from both **Peterbilt** and **TrailerTruck**, combining **numberOfWheels** = 18 with **manufacturedBy** = **PeterbiltInc**.

5.11.5 Advantages and Limitations of Frames

Advantages:

- **Organized knowledge** — related knowledge is grouped into logical units, easing management.
- **Inference through inheritance** — properties from supertypes propagate automatically to subtypes and instances.
- **Flexible representation** — complex conditions and relationships can be expressed through slot constraints.

Limitations:

- Frames lack the expressive power to represent negation or implication directly (e.g., “There is no hippopotamus in this room”).
- Arithmetic computation and more complex logic often require external systems or procedural attachments.

5.12 Rules and Data

During the 1970s, universities pioneered **expert systems**, while Ted Codd developed **relational databases** at IBM. Although expert systems and database systems differ in scale, their functionality has increasingly converged, as database systems handle more complex operations and expert systems are applied to larger data sets.

- **Expert systems** focus on executing repeated rules over small data sets.
- **Database systems** execute short chains of rules over large data sets.

Both rely on the **existential-conjunctive (EC)** subset of logic, and on two central inference rules:

- **Modus Ponens:** from p and $(p \rightarrow q)$, infer q — the basis of **forward chaining**.
- **Modus Tollens:** from $\neg q$ and $(q \rightarrow p)$, infer $\neg p$ — the basis of **backward chaining**.

5.12.1 Historical Systems

- **Planner** (1971, MIT) — combined forward and backward chaining with relational databases.
- **MYCIN** (Stanford, 1976) — a backward-chaining system for diagnosing bacterial infections.
- **OPS5** (Carnegie-Mellon) — a forward-chaining system that became the foundation for commercial expert-system shells such as CLIPS.
- **Prolog** (Europe) — combined backward chaining with logical operations, and influenced database integration.

Systems such as Microplanner, Prolog, and SQL all use **backtracking** to answer queries: relevant relations are searched for the desired data, and different options are tried if a goal cannot be satisfied. SQL databases use optimizations such as indexing, hash coding, and goal reordering to improve performance, whereas Prolog and Microplanner typically require manual goal ordering or preprocessors for optimization.

5.12.2 Plurals and Sets

Handling plural expressions in logic is more cumbersome than in natural language.

- Microplanner uses **find** operators to locate multiple matches.
- Prolog's **setof** predicate accumulates search results into a list.
- SQL provides built-in operators such as **GROUP BY** and **HAVING** to manipulate large sets efficiently.

5.12.3 Comparing SQL, Prolog, and CLIPS

Consider two relations: $\text{supports}(x, y)$ — object x supports object y — and $\text{objects}(id, \text{shape}, \text{color})$, describing the properties of an object.

SQL. *Query (English):* “Select all supportees from the `supports` and `objects` relations, where the supportee has shape ‘block’ and its supporter has shape ‘pyramid.’” A view can be defined as:

```
CREATE VIEW sup_color AS
SELECT supporter, color
FROM supports, objects
WHERE supportee = id;
```

Prolog. A rule for the same relationship:

```
sup_color(S, C) :- supports(S, X), objects(X, _, C).
```

This is true if S supports some object X that has colour C . A query to find all supporters of pyramid E:

```
setof(S, ind_supports(S, 'E'), L).
```

CLIPS. A forward-chaining rule to maintain a derived `contains` relationship for boxes:

```
(defrule checkForBoxSupporter
(supports ?x ?y)
(objects ?x box ?)
=>
(assert (contains ?x ?y)))
```

Whenever $?x$ is a box that supports some object $?y$, the rule asserts `(contains ?x ?y)`, proactively updating the knowledge base.

Table 5.1: Comparing SQL, Prolog, and CLIPS

Feature	SQL	Prolog	CLIPS
Query method	Declarative	Logical (backward chaining)	Forward chaining
Syntax	SELECT, JOIN, WHERE, GROUP BY	Rules with :-	Pattern matching, <code>defrule</code>
Relation handling	Views and triggers	Rules with recursion	Rule-based assertions
Optimization focus	Query optimizers	Not optimized for recursion	Rete network for efficiency
Updating	Database triggers	Pattern-matching updates	Proactive updates

Comparison.

Conceptual graph form. The natural-language query “Which blocks are supported by 3 pyramids?” can be represented, close to natural language, as:

$$[\text{Block} : \{*\} ?] - (\text{Thme}) - [\text{Support}] - (\text{Inst}) - [\text{Pyramid} : \text{Col}\{*\} @ 3],$$

where $\{*\}$ represents plurality and *Thme/Inst* indicate the linguistic roles involved.

Backward chaining (Prolog, SQL) deduces results by exploring relationships backward from known facts; **forward chaining** (CLIPS) proactively updates the knowledge base to keep relationships consistent. SQL excels at simple, declarative queries over large data sets; Prolog supports recursion and pattern matching but lacks native support for some database operations; CLIPS provides fast updates and real-time constraint checking. Different systems share a common logical core but differ in syntax and mechanism, and conceptual graphs act as an intermediate representation between natural-language queries and these lower-level languages.

5.13 Object-Oriented Systems

Object-oriented systems (OOPS) combine declarative (specifying objects) and procedural (defining actions) styles, integrating data and operations into a single package rather than keeping them separate.

5.13.1 Classes and Encapsulation

Example — Truck class (Java).

```
public class Truck extends Vehicle {
    private int unloadedWeight;
    private int maxGrossWeight;
    private int cargoCapacity;

    public int getUnloadedWeight() { return unloadedWeight; }
    public int getMaxGrossWeight() { return maxGrossWeight; }
    public int getCargoCapacity() { return cargoCapacity; }
}
```

Here, **Truck** inherits properties from its parent class **Vehicle**; data about weight and capacity is encapsulated in private variables, accessible only through methods such as `getUnloadedWeight()`.

Encapsulation restricts direct access to an object’s internal data — only the class’s own methods, or its subclasses, may interact with private variables.

```
public class TrailerTruck extends Truck {
    private Trailer trailer;
```

```

public TrailerTruck() {
    trailer = new Trailer();
    System.out.println("A new TrailerTruck has been created.");
}
}

```

Other parts of the program cannot access `trailer` directly — the constructor initializes and encapsulates it, in keeping with OOPS principles. Encapsulation therefore enhances **data integrity**: operations such as loading cargo or navigating routes remain hidden inside the class, keeping data safe and consistent.

5.13.2 Mapping Frames to Objects

- A **slot** in a frame maps to an **instance variable** in OOPS.
- **Inheritance** in frames corresponds to inheritance of properties and methods between classes.

Example: the parent frame `Vehicle` (with attributes such as speed and capacity) corresponds to a parent class `Vehicle`; the sub-frame `Truck` inherits these properties and adds `cargoCapacity`, corresponding to the class `Truck` **extends** `Vehicle`.

5.13.3 Procedural Integration

OOPS requires procedural code to carry out actions.

- **Backward chaining example:** recursive-descent parsing breaks a sentence such as “The truck is moving” into its subject, verb, and object by recursively applying grammar rules.
- **Forward chaining example — Observer pattern:**

```

Observable inventory = new Observable();           // main inventory system
Observer warehouseManager = new Observer();        // dependent warehouse system

inventory.addObserver(warehouseManager);

```

Whenever new products are added to `inventory`, all registered observers (such as `warehouseManager`) are notified automatically.

5.13.4 Logic-Based OOPS and Multiple Views

In some systems, procedures encode logical propositions rather than stating them explicitly — for example, PowerLoom frames can be automatically converted into equivalent C++ declarations, preserving logical rules while maintaining compatibility across systems.

Interactive systems also allow **zooming in and out** of a representation. In a conceptual graph of a birthday party, zooming in might reveal the detailed process of cutting the cake and guest interactions, while zooming out shows a concise summary of relationships between guests and the cake. Finally, programmers routinely use comments and documentation (translation into natural language) as a bridge between code and human understanding:

```
// Method to get the truck's cargo capacity
public int cargoCapacity() {
    return cargoCapacity;
}
```

5.14 Natural Language Semantics

5.14.1 Natural vs. Artificial Languages

Natural languages (English, Spanish, etc.) are used by humans to communicate and are inherently rich but ambiguous — e.g., “She plays the piano.” **Artificial languages** (mathematics, programming) have restricted syntax designed to avoid ambiguity — e.g., $2 + 2 = 4$ is exact and unambiguous.

Understanding a natural-language sentence requires **background knowledge** about the world and context. For example, correctly interpreting a request about the movie *Casablanca* requires knowing that *Casablanca* is a movie (not just a city) and that Humphrey Bogart is an actor — knowledge that is difficult for a computer to acquire automatically.

5.14.2 Challenges in Language Processing

1. **Syntax analysis (sentence structure):** computers can parse words quickly, but sentence *meaning* remains difficult. *Example:* “Find me a programmer at Chase Manhattan Bank who knows graphics” — the right person may not work in a department explicitly labelled “graphics.”
2. **Semantic interpretation (word meaning):** ambiguous words are hard to resolve. *Example:* “piano” could mean a musical instrument, a storey of a building (Italian *piano*), or a plane/surface — context determines the correct meaning.

5.14.3 Steps in Language Analysis

1. **Morphology (word forms):** breaking words into parts and identifying their meaning — e.g., Italian *degli* means “of the” (a preposition plus an article).
2. **Syntax (sentence structure):** analyzing grammar to determine relationships between words — e.g., “the association of industrialists” splits into phrases expressing an association and its members.

3. **Semantics (concept meaning):** translating sentences into conceptual graphs that represent meaning and relationships. *Example:* [Association] contains [Industrialists], meaning “the association includes industrialists.”

5.14.4 Resolving Ambiguities

Many words carry multiple meanings that must be resolved using context and syntax. *Example:* Italian *mezzogiorno* can mean “noon” or “the south of Italy” — context determines whether the reference is temporal or geographical.

5.14.5 Inference and Knowledge Representation

Conceptual-level representations support complex reasoning and inference. For example, in a sentence describing “Raul Gardini becoming President,” a system can determine not only who obtained the position, but can also deduce related facts, such as who held the position before or after the appointment.

5.14.6 Key Takeaways

- Natural languages are rich but challenging for computers, owing to ambiguity and the vast background knowledge they assume.
- Artificial languages have strict rules but lack the expressive flexibility of natural language.
- Language processing depends on syntax, semantics, and background knowledge — an area where AI systems still fall short of full human competence.

5.15 Levels of Representation

5.15.1 Brachman’s Levels (1979)

Ron Brachman divided knowledge representation into five levels, describing how knowledge is organized and implemented:

1. **Implementational level** — the data structures used in programming (atoms, pointers, lists).
2. **Logical level** — symbolic logic: propositions, variables, quantifiers. *Example:* “All humans are mortal.”
3. **Epistemological level** — concept types, subtypes, inheritance, and relationships among concepts. *Example:* a “vehicle” concept that includes “car,” “bus,” and “motorcycle.”

4. **Conceptual level** — semantic relations, objects, actions, and linguistic roles. *Example:* how actions relate to objects, such as “a car moving on a road.”
5. **Linguistic level** — arbitrary concepts, words, and expressions in natural language. *Example:* the English phrase “I am going to the store.”

5.15.2 Brooks’s Competence Levels (1986)

Rodney Brooks outlined eight levels of competence for mobile robots, of increasing sophistication:

1. **Avoiding** — avoiding obstacles, moving or stationary.
2. **Wandering** — moving randomly while avoiding obstacles.
3. **Exploring** — searching for reachable areas and heading toward them.
4. **Mapping** — building a map of the environment and noting routes.
5. **Noticing** — detecting environmental changes, such as new obstacles.
6. **Reasoning** — identifying objects, reasoning about their relationships, and acting on them.
7. **Planning** — creating plans to change the environment in desired ways.
8. **Anticipating** — predicting the actions of other objects and adjusting plans proactively.

5.15.3 Zachman’s Design Levels

The **Zachman Framework** outlines five levels reflecting different perspectives on information representation in systems architecture:

1. **Scope** — aspects independent of computer representation; the big picture of business operations.
2. **Enterprise model** — still implementation-independent, describing business interactions.
3. **System model** — implementation-independent descriptions selected by a system analyst.
4. **Technology model** — connects data structures to physical representations, linking programming and operations.
5. **Component level** — specific implementation details, where the connection to the outside world becomes least apparent.

Zachman's matrix organizes these five levels against six columns (such as data, function, and network), yielding 30 perspectives that capture business goals, processes, and IT implementation details together.

5.15.4 Key Takeaways

- **Representation levels (Brachman)** focus on abstract conceptual understanding down to programming detail.
- **Competence levels (Brooks)** describe how robotic systems evolve in complexity and functionality.
- **Design levels (Zachman)** show how systems are structured, from business concepts down to technical implementation.

5.16 Summary

- Knowledge Representation and Reasoning (KR&R) enables an AI agent to represent facts about the world and to reason over them in order to behave intelligently.
- Knowledge to be represented includes objects, events, performance, meta-knowledge, and facts, stored in a Knowledge Base.
- First-Order Logic is the principal representation language for KR&R, with propositional logic, predicate logic, typed logic, lambda calculus, conceptual graphs, and higher-order logic offering different trade-offs between simplicity and expressiveness.
- A good logic system requires vocabulary, syntax, semantics, and rules of inference; these four features unify the many varieties of logic in use.
- Frames provide a structured, slot-based way of representing stereotypical objects and situations, supporting inheritance and multiple inheritance, though they lack native support for negation and complex arithmetic.
- Rules and data converge in expert systems (forward and backward chaining) and relational databases (SQL), both grounded in the existential-conjunctive subset of logic.
- Object-oriented systems integrate declarative and procedural knowledge through classes, encapsulation, and inheritance, closely mirroring the structure of frames.
- Natural language semantics requires resolving syntactic and semantic ambiguity using background knowledge, a task current AI systems still find difficult.
- Knowledge can be understood at multiple levels of representation (Brachman), competence (Brooks), and design (Zachman), each offering a different lens on how knowledge-based systems are structured.

Chapter 6

Logic Programming with Prolog and LISP

6.1 Introduction

Unit 3 of the syllabus asks for a basic working knowledge of two classic AI programming languages, **Prolog** and **LISP**. Both languages grew directly out of the ideas covered in Chapter 5: Prolog is essentially a restricted, executable subset of First-Order Logic, while LISP is built around the manipulation of symbolic expressions — the same symbols and lists used to represent knowledge.

This chapter keeps to the essentials needed to read and write small programs in each language: how facts, rules, and queries work in Prolog, and how symbolic expressions, functions, and recursion work in LISP. Fuller example programs are collected in the appendices.

6.2 Prolog: Logic Programming

Prolog (*Programming in Logic*) is a declarative language: instead of specifying *how* to compute a result, a Prolog program states *facts* and *rules* about a problem, and the Prolog interpreter answers queries by searching for values that satisfy them using backward chaining (Section 5.9).

6.2.1 Facts

A **fact** declares something that is unconditionally true.

```
male(ram).  
male(shyam).  
female(sita).  
parent(ram, arjun).  
parent(sita, arjun).
```

Each fact is a **predicate** applied to one or more **arguments**, terminated with a full stop. Here `male(ram)` states that `ram` is male, and `parent(ram, arjun)` states that `ram` is a parent of `arjun`.

6.2.2 Rules

A **rule** defines a new fact in terms of existing ones, using the form `Head :- Body`, read as “*Head* is true **if** *Body* is true.”

```
father(X, Y) :- male(X), parent(X, Y).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

- `X`, `Y`, `Z` are **variables** (always capitalized in Prolog).
- The comma between goals in the body means **AND** — both conditions must hold.
- `father(X, Y)` is true if `X` is male **and** `X` is a parent of `Y`.

6.2.3 Queries

Once facts and rules are loaded, a **query** asks Prolog to find values that make a goal true.

```
?- father(ram, arjun).
yes

?- father(X, arjun).
X = ram
```

Prolog answers a query by **unification** — matching the query against the heads of facts and rules — and **backtracking**: if a chosen match leads to failure later on, Prolog undoes it and tries the next possibility. This search strategy is what makes Prolog a practical, executable form of the backward-chaining inference discussed in Chapter 5.

6.2.4 Lists and Recursion

Prolog represents sequences using **lists**, written in square brackets, e.g. `[1, 2, 3]` or `[H|T]`, where `H` is the head (first element) and `T` is the tail (remaining list). Recursion — a rule referring to itself — is the standard way to process a list.

```
% length of a list
mylen([], 0).
mylen([_|T], N) :- mylen(T, N1), N is N1 + 1.
```

Here the first clause is the **base case** (an empty list has length 0), and the second is the **recursive case** (the length of a non-empty list is 1 plus the length of its tail). `is` is used for arithmetic evaluation.

6.2.5 Key Features of Prolog

- **Declarative:** programs state relationships, not step-by-step procedures.
- **Pattern matching via unification:** variables are bound to values as needed to satisfy a goal.
- **Backtracking search:** alternative solutions are explored automatically on failure.
- **Built-in support for lists and recursion** rather than iterative loops.
- Widely used for expert systems, natural language processing, and symbolic AI.

6.3 LISP: Symbolic Programming

LISP (*LISt Processing*) is one of the oldest programming languages, designed for manipulating symbolic data — the same lists and symbols used in knowledge representation. Both code and data in LISP are written as **lists**, using fully parenthesized **prefix notation**: the operator comes first, followed by its arguments.

```
(+ 2 3)          ; evaluates to 5
(* 4 (+ 1 2))    ; evaluates to 12
```

6.3.1 Atoms and Lists

- An **atom** is a basic, indivisible value — a number (5), or a symbol (**ram**, +).
- A **list** is a sequence of atoms or lists enclosed in parentheses, e.g. (a b c) or (1 (2 3) 4).

6.3.2 Defining Functions

Functions are defined with **defun**:

```
(defun square (x)
  (* x x))

(square 5)    ; evaluates to 25
```

6.3.3 Conditionals

LISP uses **cond** (or **if**) for conditional evaluation:

```
(defun abs-val (x)
  (cond ((< x 0) (- x))
        (t x)))
```

Here, if `x` is negative, its negation is returned; otherwise (`t`, meaning “true” / default case) `x` itself is returned.

6.3.4 List Manipulation

LISP provides three fundamental list operations, all central to symbolic AI programming:

- `(car lst)` — returns the **first element** of a list.
- `(cdr lst)` — returns the list **without its first element**.
- `(cons x lst)` — builds a **new list** by adding `x` to the front of `lst`.

```
(car '(a b c))    ; a
(cdr '(a b c))    ; (b c)
(cons 'a '(b c))  ; (a b c)
```

6.3.5 Recursion

Like Prolog, LISP relies heavily on recursion rather than iterative loops to process lists.

```
(defun mylen (lst)
  (cond ((null lst) 0)
        (t (+ 1 (mylen (cdr lst))))))
```

`(null lst)` is the base case (an empty list has length 0); otherwise, the length is 1 plus the length of the rest of the list — mirroring the Prolog `mylen` example above.

6.3.6 Key Features of LISP

- **Uniform syntax:** code and data share the same list-based representation, which makes it natural for a LISP program to treat other LISP code as data.
- **Symbolic, not just numeric, computation** — well suited to reasoning over symbols, trees, and knowledge structures.
- **Recursion-first style**, with automatic memory management (garbage collection).
- Historically the primary language of AI research in the West, and the basis for many expert-system shells.

6.4 Prolog vs. LISP

Table 6.1: Prolog versus LISP at a glance

Aspect	Prolog	LISP
Paradigm	Declarative (logic-based)	Functional / symbolic
Core idea	Facts, rules, and queries	Functions over lists and atoms
Execution	Unification + backtracking	Function evaluation (recursion)
Basic unit	Predicate / clause	Atom / list (S-expression)
Typical use	Expert systems, NLP, logic puzzles	Symbolic AI, list processing, prototyping

6.5 Summary

- Prolog programs consist of **facts** and **rules**; a **query** is answered by unification and backtracking, making Prolog a direct, executable form of backward-chaining logical inference.
- LISP programs are built from **atoms** and **lists**; functions are defined with **defun** and combined using prefix notation, with **car**, **cdr**, and **cons** as the fundamental list operations.
- Both languages favour **recursion** over iterative loops, and both remain foundational to symbolic AI, expert systems, and knowledge representation.
- Complete worked example programs in Prolog and LISP are provided in Appendix A and Appendix B.

Chapter 7

Learning and Machine Learning

7.1 Introduction

One of the most important abilities of an intelligent system is the ability to learn. A system that cannot learn must depend on a programmer to describe every situation in advance. This is difficult when the environment is large, uncertain, or continuously changing.

In Artificial Intelligence, **learning** means improving the ability of a system to perform a task using experience, data, feedback, or observations. **Machine Learning (ML)** is the area of AI that develops algorithms that allow computers to learn useful patterns from data and use those patterns to make predictions or decisions.

Machine learning therefore changes the role of the programmer. Instead of writing every decision rule explicitly, the programmer defines the learning problem, provides suitable data, selects an appropriate learning method, and evaluates the resulting model.

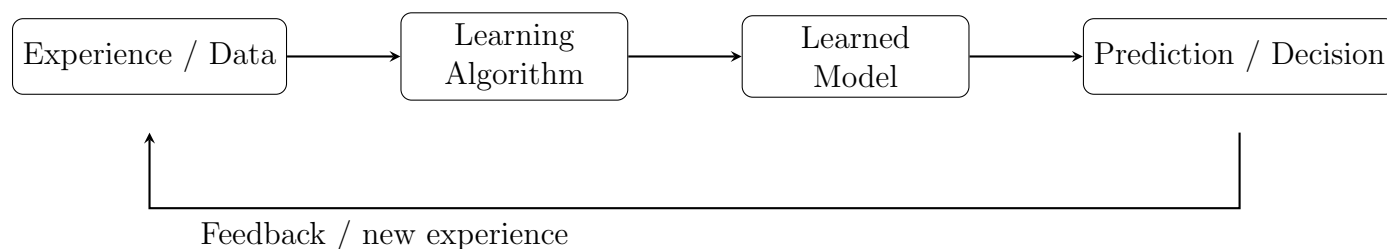


Figure 7.1: Basic idea of learning in an intelligent system

This chapter introduces the idea of learning, the major types of machine learning, the basic structure of a learning system, supervised and unsupervised learning, reinforcement learning, generalization, model selection, and the role of machine learning in modern AI. Neural networks and deep learning are discussed separately in Chapter 8.

7.2 What is Learning?

Learning can be understood in a simple way: **a system learns when experience causes its future performance to improve**. The experience may come from examples, observations, interaction with the environment, or feedback.

A useful formal view of a learning problem is given by Tom Mitchell:

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

The three parts of the definition are important:

1. **Task (T):** What the system must do.
2. **Performance measure (P):** How well the system performs the task.
3. **Experience (E):** The information from which the system improves.

7.2.1 Example: Learning to Play a Game

Suppose a program learns to play a board game by playing many games against itself.

- T = playing the game;
- P = percentage of games won;
- E = the games played during training.

The program is considered to have learned if, after gaining experience, it performs better than it did before.

7.3 Artificial Intelligence and Machine Learning

Machine learning is a major approach for building intelligent systems, but AI and ML are not identical terms.

- **Artificial Intelligence** is the broader field concerned with creating systems that can perceive, reason, learn, plan, and act intelligently.
- **Machine Learning** is a set of methods through which a system learns patterns or relationships from data or experience.
- **Deep Learning** is a family of machine learning methods based on multi-layer neural networks. It is studied in Chapter 8.

Thus, the relationship can be summarized as:

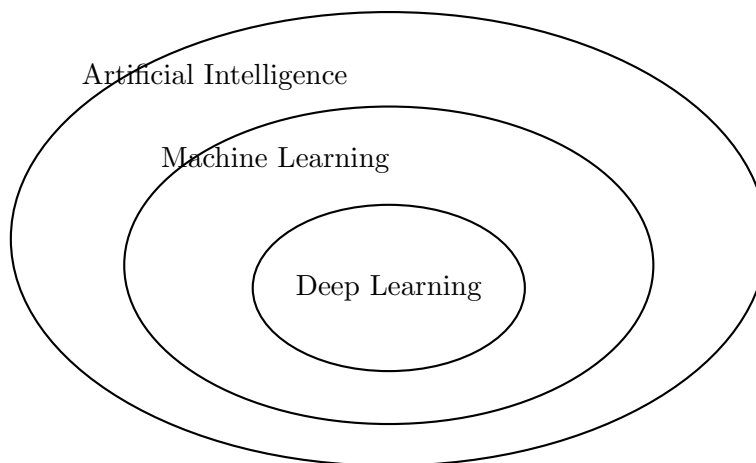


Figure 7.2: Machine Learning as a part of Artificial Intelligence

The figure is a conceptual relationship rather than a statement that every AI method must use machine learning. Classical AI methods such as rule-based reasoning and search can work without learning from data.

7.4 Why Do We Need Machine Learning?

Traditional programming works well when the rules of a problem can be stated clearly. However, many real-world tasks are difficult to describe using fixed rules.

For example, consider the task of detecting whether an email is spam. A programmer could write many rules, but spammers continually change the wording, links, and structure of their messages. A learning system can study examples of spam and non-spam messages and learn useful patterns.

Machine learning is especially useful when:

- the rules are difficult to write explicitly;
- a large amount of data is available;
- the environment changes over time;
- decisions depend on many interacting factors;
- the problem contains uncertainty or noise;
- the system must improve from experience.

Table 7.1: Machine Learning and Conventional Programming

Feature	Machine Learning	Conventional Programming
Main idea	Learn a model from examples or experience	Execute explicitly written rules
Source of behaviour	Data, experience, feedback	Programmer-defined logic
Adaptation	Can improve after retraining	Requires program changes
Handling complex patterns	Often effective for patterns that are hard to describe as rules	Usually difficult if rules are numerous or unclear
Typical output	Prediction, classification, ranking, decision	Computed result according to fixed instructions
Example	Spam detection, image classification	Payroll calculation, calculator

7.4.1 Machine Learning versus Conventional Programming

7.5 A Learning System

A machine learning system normally contains several related components. Understanding these components is more important than memorizing the names of particular algorithms.

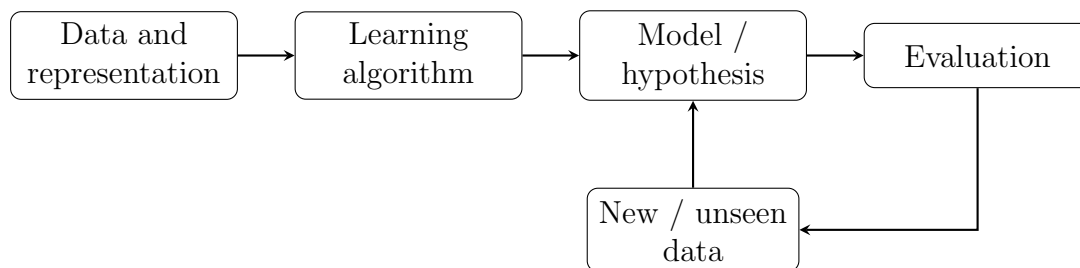


Figure 7.3: Major components of a machine learning workflow

7.5.1 Data

Data is the experience from which the system learns. A dataset usually contains a collection of examples. In supervised learning, each example may contain input features and a desired output.

For a dataset, a single example may be represented as

$$\mathbf{x} = [x_1, x_2, \dots, x_n],$$

where $x_1, x_2, \dots, x_n$ are features or attributes.

If a desired output is available, it may be represented by y . Thus, a supervised example can be written as

$$(\mathbf{x}, y).$$

7.5.2 Features and Labels

A **feature** is an input property used by the learning algorithm. A **label** is the desired output associated with a training example in supervised learning.

For example, in a student-result classifier:

- features may include attendance and internal marks;
- the label may be *Pass* or *Fail*.

Good feature representation can strongly affect the quality of a learned model.

7.5.3 Model or Hypothesis

A learning algorithm searches for a model that connects the input to the output or describes useful structure in the data. In supervised learning, a simple model may be represented as

$$y = f(\mathbf{x}; \theta),$$

where θ represents the parameters of the model.

Different parameter values correspond to different possible models. The learning process tries to select parameter values that produce useful predictions.

7.5.4 Learning Algorithm

The learning algorithm is the procedure that uses the available experience to determine the model parameters or the structure of the model. Examples of learning algorithms belong to different families, such as classification, regression, clustering, and reinforcement learning.

7.5.5 Performance Measure

A learning system must have a way to measure whether it is improving. Depending on the task, the performance measure may be accuracy, error, reward, or another application-specific quantity.

7.6 Types of Learning

The major learning types introduced in the syllabus are shown below.

The key difference is the kind of experience supplied to the learning system.

7.7 Supervised Learning

In **supervised learning**, the training data contains examples for which the desired output is known. The learning algorithm studies the relationship between inputs and outputs and then uses the learned relationship to predict outputs for new inputs.

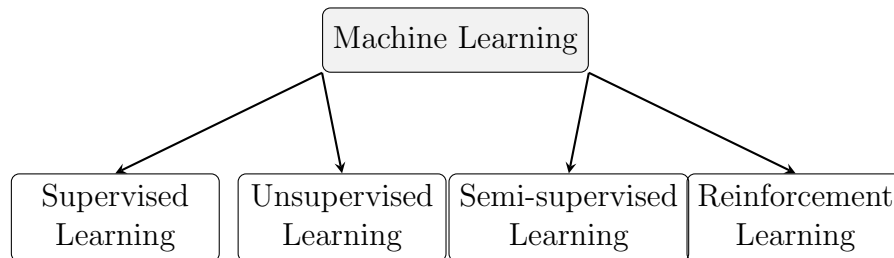


Figure 7.4: Major types of machine learning

Table 7.2: Comparison of learning types

Type	Training information	Main purpose	Typical examples
Supervised	Input with desired output	Learn input-output relationship	Classification, regression
Unsupervised	Input without desired output	Discover structure or patterns	Clustering, association, dimensionality reduction
Semi-supervised	Small amount of labelled data plus larger amount of unlabelled data	Improve learning when labels are expensive	Image/text classification with limited labels
Reinforcement	Rewards or penalties from interaction	Learn actions that produce good long-term outcomes	Game playing, robot navigation

Suppose the training set is

$$D = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}.$$

The objective is to learn a function

$$\hat{y} = f(\mathbf{x}; \theta)$$

that performs well not only on the training examples but also on previously unseen examples.

7.7.1 Basic Supervised Learning Workflow

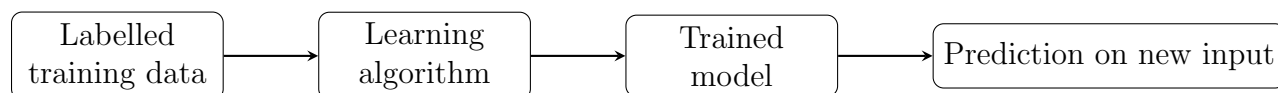


Figure 7.5: Supervised learning workflow

7.7.2 Classification

Classification is a supervised learning task in which the output is a discrete class or category.

Examples include:

- spam or not spam;
- pass or fail;
- low-risk or high-risk customer;
- cat, dog, or bird in an image.

Suppose a system predicts whether a student is likely to pass from two features: attendance and internal marks. The learned model divides the input space into regions corresponding to the classes.

The boundary shown in the figure is only an illustration. Real algorithms may learn curved or high-dimensional decision boundaries.

7.7.3 Regression

Regression is a supervised learning task in which the output is a continuous numerical value.

Examples include:

- predicting the price of a used car;
- predicting electricity demand;

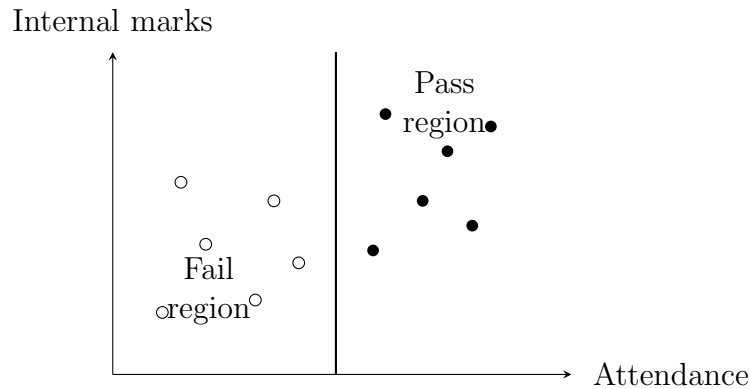


Figure 7.6: A simple classification boundary

- estimating house value;
- forecasting a numerical measurement.

A simple regression model may be written as

$$y = f(\mathbf{x}; \theta).$$

For one input feature, a linear model is

$$y = \theta_0 + \theta_1 x.$$

The learning process chooses parameters so that the predicted values are close to the observed target values according to a selected error measure.

7.8 Learning a Class from Examples

An important idea in supervised learning is learning a class from examples. Suppose we want to build a classifier for the class *family car*.

The training data may contain:

- positive examples: cars that belong to the class;
- negative examples: cars that do not belong to the class.

If price and engine power are used as features, the learner tries to find a rule or model that separates the positive and negative examples.

For a new car, the learned model then answers the question:

Does this car belong to the learned class?

This simple example introduces the central idea of classification: learn a decision rule from examples and apply it to new data.

7.9 Unsupervised Learning

In **unsupervised learning**, the training data does not contain a known desired output. The learner must discover useful structure in the input data itself.

The system may attempt to answer questions such as:

- Which observations are similar?
- Are there natural groups in the data?
- Which variables are strongly related?
- Can the data be represented with fewer dimensions?

7.9.1 Clustering

Clustering groups similar observations together. The number and meaning of groups may not be known in advance.

For example, an online learning platform may collect information about student activity. Without assigning labels in advance, an unsupervised algorithm may discover groups such as highly active learners, moderate learners, and low-activity learners.

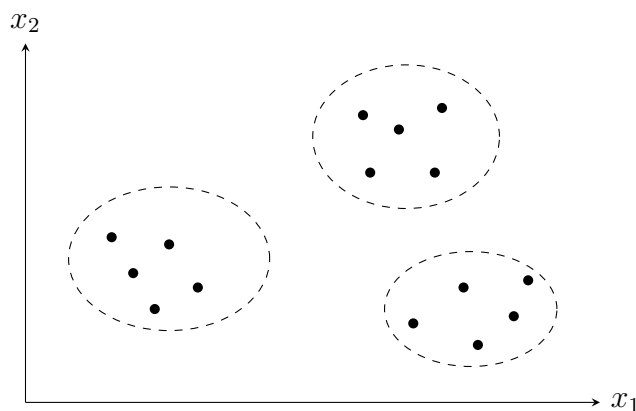


Figure 7.7: Conceptual view of clustering

The important point is that the groups are discovered from the data rather than supplied as class labels.

7.9.2 Association

Association learning discovers relationships between variables or items. A familiar example is market-basket analysis. If many customers who buy item X also buy item Y , the system may discover an association between X and Y .

A conditional probability such as

$$P(Y | X)$$

can express how often Y occurs when X is observed.

Association analysis is useful in recommendation systems, shopping analysis, and other tasks in which relationships among observations are more important than a single target label.

7.9.3 Dimensionality Reduction

A dataset may contain many features. Some features may be redundant or may contribute very little useful information. Dimensionality reduction seeks a lower-dimensional representation while preserving important structure in the data.

It can help with:

- reducing computational cost;
- reducing storage requirements;
- simplifying models;
- visualizing high-dimensional data.

Detailed dimensionality-reduction methods are beyond the scope of this introductory chapter.

7.10 Semi-supervised Learning

In **semi-supervised learning**, only a small part of the training data is labelled, while a larger part is unlabelled.

This is useful when obtaining labels requires human effort or expert knowledge. For example, thousands of images may be available, but only a small number may have been manually labelled.

The basic idea is:

$$\text{small labelled set} + \text{large unlabelled set} \longrightarrow \text{improved learning}$$

Semi-supervised learning is especially relevant where data is easy to collect but labels are expensive to create.

7.11 Reinforcement Learning

Reinforcement Learning (RL) is a type of learning in which an agent interacts with an environment and receives rewards or penalties as feedback.

Unlike supervised learning, the system is not directly given the correct action for every situation. Instead, it must discover which actions lead to useful long-term outcomes.

The central components are:

- **Agent:** the learner or decision maker;
- **Environment:** the world with which the agent interacts;
- **State:** the current situation of the agent;
- **Action:** an operation selected by the agent;
- **Reward:** feedback indicating how useful the result was;
- **Policy:** a strategy that specifies which action should be selected in a state.

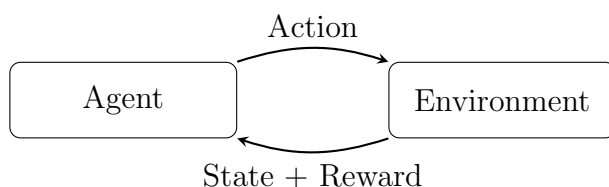


Figure 7.8: Basic interaction in reinforcement learning

The system learns by trial and error. A good sequence of actions should lead to a higher cumulative reward.

7.11.1 Example: Robot in a Maze

Suppose a robot must reach the exit of a maze.

- **State:** robot's current position and surrounding information;
- **Action:** move left, right, forward, or backward;
- **Positive reward:** reaching the exit;
- **Negative reward:** hitting a wall or taking an undesirable action;
- **Policy:** the strategy learned for choosing actions.

The robot does not need a programmer to specify the perfect action for every position. It can learn a useful strategy from repeated interaction.

7.12 Training, Validation and Test Data

A major goal of learning is not to memorize the training examples. The learned model should also work well on new data.

For this reason, a dataset is commonly separated into three conceptual parts:

1. **Training set:** used to learn the model parameters.
2. **Validation set:** used during development to compare models or tune settings.
3. **Test set:** used at the end to obtain an unbiased estimate of performance on unseen data.

The exact proportions depend on the size and nature of the dataset. The important principle is that the final test data should not be used to train the model.

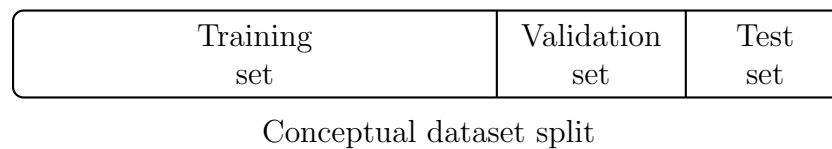


Figure 7.9: Training, validation and test sets

7.13 Generalization

Generalization is the ability of a learned model to perform well on new examples that were not used during training.

A model that performs extremely well on the training data but poorly on new data has failed to generalize.

The distinction is important:

Learning is not simply remembering the examples. The real objective is to learn useful regularities that continue to work when new examples are presented.

7.13.1 Training Error and Generalization Error

Training error is the error measured on examples used for learning. **Generalization error** is the error observed on previously unseen data.

A low training error does not guarantee a low generalization error.

7.14 Overfitting and Underfitting

Two common failures in machine learning are **overfitting** and **underfitting**.

7.14.1 Overfitting

Overfitting occurs when a model becomes too closely adapted to the training data. It may learn noise or accidental details rather than the underlying pattern.

Typical symptoms include:

- very low training error;
- noticeably higher validation or test error;
- unstable performance on new examples.

7.14.2 Underfitting

Underfitting occurs when a model is too simple to capture the important structure in the data.

Typical symptoms include:

- high training error;
- poor validation or test performance;
- systematic failure to represent important relationships.

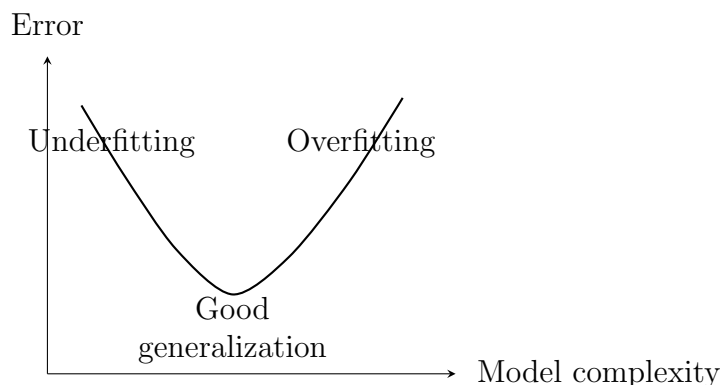


Figure 7.10: Model complexity and generalization error

The aim is not to choose the most complex model. The aim is to choose a model that is sufficiently expressive but still generalizes well.

7.15 Noise in Data

Real datasets often contain noise. Noise may arise from inaccurate measurements, incorrect labels, missing information, or variables that were not observed.

For example, suppose a classifier is trained on a dataset in which some students have been incorrectly labelled as “Pass” and others as “Fail”. A model that tries to fit every incorrect label may become unnecessarily complex.

A good learner should therefore capture meaningful regularities while avoiding unnecessary dependence on accidental details.

7.16 Model Selection and Inductive Bias

A dataset is usually not enough to identify one unique model. Many different models may explain the same training examples.

The learner therefore needs an **inductive bias**: assumptions that guide the learner toward some explanations rather than others.

For example, if a linear model and a very complex model both fit a small dataset, choosing the simpler model may be reasonable when there is no evidence that the additional complexity is useful.

This idea is closely related to **Occam's Razor**: when several explanations are possible, a simpler explanation is often preferred when it explains the observations adequately.

Simplicity does not mean that the simplest possible model is always correct. It means that complexity should be justified by the evidence.

7.17 Learning as Optimization

Many machine learning methods can be understood as an optimization problem.

Suppose a model is written as

$$\hat{y} = f(\mathbf{x}; \theta),$$

where θ represents the model parameters. A learning algorithm searches for parameter values that make the model perform well on the available training data.

A loss function measures the disagreement between predictions and desired outputs. In general form,

$$\theta^* = \arg \min_{\theta} L(\theta),$$

where $L(\theta)$ is a loss or error function.

For a regression task, one possible loss is the mean squared error:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2.$$

The exact optimization method depends on the learning algorithm. The important idea is that learning changes the model parameters so that the model becomes better according to a selected objective.

7.18 From Data to a Learned Model

A complete learning cycle can be summarized as follows:

1. Define the task.

2. Collect relevant data.
3. Represent the data using useful features.
4. Choose an appropriate learning type and model family.
5. Train the model using the training data.
6. Evaluate using validation or test data.
7. Improve the representation, model, or training process when necessary.
8. Deploy the model and monitor its performance on new data.

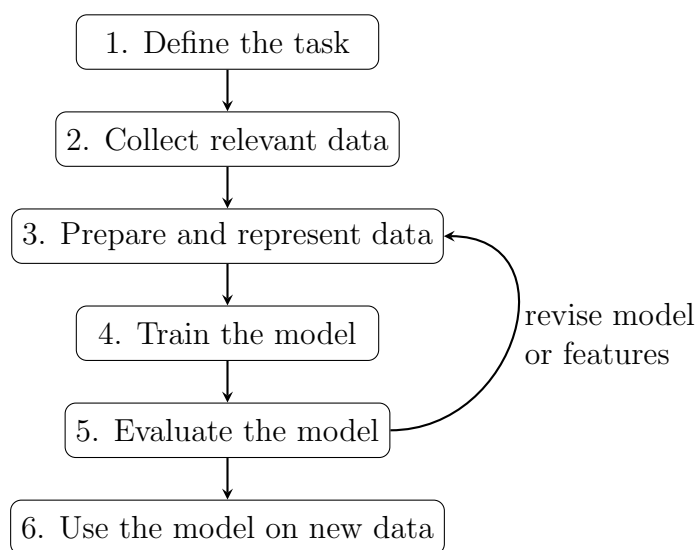


Figure 7.11: A practical machine learning development cycle

7.19 Applications of Machine Learning

Machine learning is used in many areas because many practical problems involve patterns that can be learned from data.

Major application areas include:

- **Pattern recognition:** face, handwriting, speech, and image recognition;
- **Anomaly detection:** unusual financial transactions or machine behaviour;
- **Prediction:** demand, prices, risk, or other numerical outcomes;
- **Recommendation systems:** products, videos, music, or documents;
- **Information retrieval:** finding similar or relevant documents and images;

- **Healthcare:** medical image analysis and decision support;
- **Natural Language Processing:** classification, translation, summarization, and language understanding;
- **Robotics:** navigation, control, perception, and decision making.

These applications illustrate that machine learning is not one single algorithm. It is a collection of methods for learning useful behaviour from experience and data.

7.20 Learning, Machine Learning and Deep Learning

The terms are closely related but should not be confused.

Table 7.3: Learning, Machine Learning and Deep Learning

Concept	Meaning	Main idea
Learning	Improvement through experience	Performance changes because experience is used
Machine Learning	Computational methods for learning from data or experience	Learn a model from examples, observations, or feedback
Deep Learning	Machine learning using multi-layer neural networks	Learn increasingly complex representations from data

The next chapter develops this final idea by studying artificial neural networks and deep learning.

7.21 Chapter Summary

- Learning is the process of improving performance through experience, data, observations, or feedback.
- A machine learning problem can be described in terms of a task, a performance measure, and experience.
- Machine Learning is a major part of AI, while Deep Learning is a family of machine learning methods based on multi-layer neural networks.
- A learning system works with data, a representation, a learning algorithm, a model, and a performance measure.
- Supervised learning uses labelled examples and commonly includes classification and regression.
- Unsupervised learning uses unlabelled data to discover structure such as clusters or associations.

- Semi-supervised learning combines a small labelled set with a larger unlabelled set.
- Reinforcement learning uses interaction, trial and error, and rewards or penalties to learn useful behaviour.
- Training, validation, and test sets serve different purposes in model development and evaluation.
- Generalization is the ability of a model to perform well on unseen data.
- Overfitting occurs when a model learns the training data too specifically; underfitting occurs when the model is too simple to capture important structure.
- Noise and model complexity influence the quality and generalization of learned models.
- Model selection requires an appropriate balance between expressive power and simplicity.

7.22 Key Points to Remember

1. Learning means improvement through experience.
2. A machine learning model is learned rather than completely hard-coded.
3. Features describe inputs; labels represent desired outputs in supervised learning.
4. Classification predicts classes; regression predicts numerical values.
5. Unsupervised learning discovers hidden structure without target labels.
6. Reinforcement learning learns actions from rewards and penalties.
7. The real objective is good performance on unseen data, not merely low training error.
8. Overfitting and underfitting are both undesirable.
9. Simpler models can generalize better when additional complexity is not justified by the data.
10. Learning can often be viewed as optimization of model parameters.

7.23 Short Questions

1. What is learning in Artificial Intelligence?
2. Define Machine Learning.

3. State the three components of Tom Mitchell's definition of a learning problem.
4. What is the difference between a feature and a label?
5. What is supervised learning?
6. Distinguish between classification and regression.
7. What is unsupervised learning?
8. What is clustering?
9. What is semi-supervised learning?
10. What is reinforcement learning?
11. Define agent, environment, action, and reward.
12. What is generalization?
13. Differentiate training error and generalization error.
14. What is overfitting?
15. What is underfitting?
16. What is the purpose of a validation set?
17. Why is test data kept separate from training data?
18. What is an inductive bias?
19. What is the role of a loss function?
20. How is Deep Learning related to Machine Learning?

7.24 Long Questions

1. Explain the concept of learning in Artificial Intelligence. Discuss Tom Mitchell's definition using a suitable example.
2. Explain Machine Learning and compare it with conventional programming.
3. Describe the major components of a machine learning system with a suitable diagram.
4. Explain supervised learning in detail. Differentiate classification and regression with examples.

5. Explain unsupervised learning. Discuss clustering, association, and dimensionality reduction.
6. What is semi-supervised learning? Explain why it is useful when labelled data is expensive.
7. Explain reinforcement learning with the concepts of agent, state, action, environment, reward, and policy.
8. Explain the importance of training, validation, and test datasets in machine learning.
9. What is generalization? Explain overfitting and underfitting with the help of a diagram.
10. Explain the effect of noise and model complexity on learning and generalization.
11. Explain model selection and inductive bias. Why is a balance between simplicity and model power necessary?
12. Explain machine learning as an optimization problem using a model function and a loss function.
13. Discuss important real-world applications of machine learning.
14. Differentiate Learning, Machine Learning, and Deep Learning.

Chapter 8

Neural Networks and Deep Learning

8.1 Introduction

In Chapter 7 we studied learning as the process by which an intelligent system improves from experience. One of the most important ways of implementing learning is through an **Artificial Neural Network (ANN)**.

An artificial neural network is a mathematical model made from interconnected computational units called **neurons**. The model is inspired by biological nervous systems, but it should not be treated as a complete copy of the human brain. Its purpose is practical: to learn useful relationships between inputs and outputs.

The syllabus for this chapter focuses on the basic ANN model, the McCulloch–Pitts (MP) neuron, the single-layer perceptron, its learning algorithm and limitations, multilayer perceptrons, activation functions, and the basic idea of Convolutional Neural Networks (CNNs). These ideas form the foundation for understanding modern deep learning.

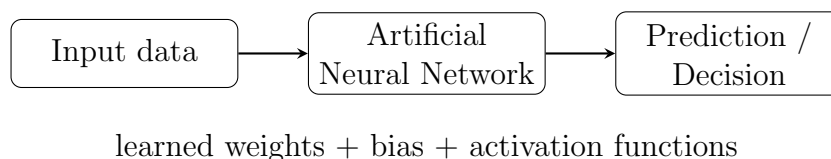


Figure 8.1: Basic idea of an artificial neural network

8.2 What is an Artificial Neural Network?

An **Artificial Neural Network** is a computational model consisting of interconnected processing units called neurons. Each neuron receives input values, combines them using weights and a bias, applies an activation function, and produces an output.

An artificial neuron can be understood through three fundamental operations: **multiplication, summation, and activation**. Each input is multiplied by a weight, the weighted inputs and bias are added, and the result is passed through an activation or transfer function.

In mathematical form, a neuron can be written as

$$z = \sum_{i=1}^n w_i x_i + b,$$

followed by

$$y = f(z),$$

where

- x_i are the input values,
- w_i are the corresponding weights,
- b is the bias,
- z is the weighted sum, and
- $f(\cdot)$ is the activation function.

The two equations above are the heart of an artificial neuron. Most neural-network concepts become easier once these two operations are understood.

8.3 From Biological Neuron to Artificial Neuron

A biological neuron normally receives signals through dendrites, processes information in the cell body, and sends signals through an axon. An artificial neuron uses a much simpler mathematical abstraction.

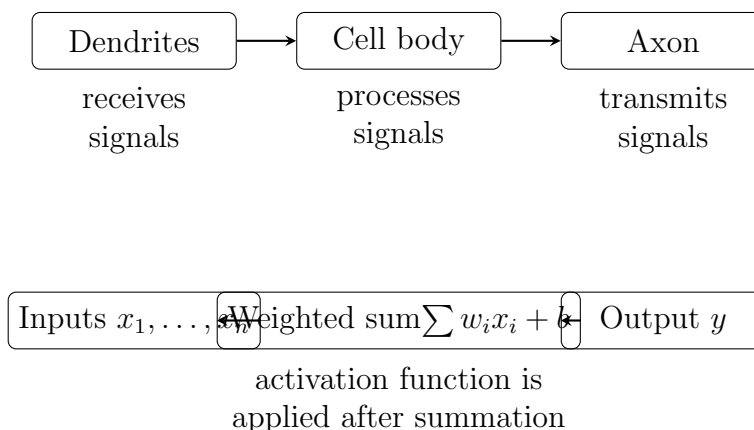


Figure 8.2: Conceptual analogy between a biological neuron and an artificial neuron

The analogy is useful for understanding the basic idea, but an ANN is a mathematical model rather than a detailed simulation of biology. Its computation is based on weighted inputs, summation, bias, and an activation function.

8.4 Basic Terminology of Neural Networks

Before studying particular models, the following terms should be clear.

Table 8.1: Important neural-network terminology

Term	Meaning
Neuron / node	Basic computational unit of a neural network.
Input	A value supplied to a neuron or network.
Weight	A parameter that controls the strength and sign of an input connection.
Bias	A parameter that shifts the activation function and allows a more flexible decision boundary.
Weighted sum	The sum $\sum w_i x_i + b$ before activation.
Activation function	Function that transforms the weighted sum into the neuron's output.
Input layer	Layer that receives the feature values.
Hidden layer	Intermediate layer where learned representations are formed.
Output layer	Layer that produces the final prediction or decision.
Epoch	One complete pass through the training data.
Learning rate	Controls the size of a parameter update during training.
Loss function	Measures how far a prediction is from the desired output.

8.5 Artificial Neuron

The basic artificial neuron can be represented by the following computation:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b.$$

The final output is

$$y = f(z).$$

The neuron therefore performs two conceptual stages:

1. combine the inputs using learned parameters;
2. transform the combined value using an activation function.

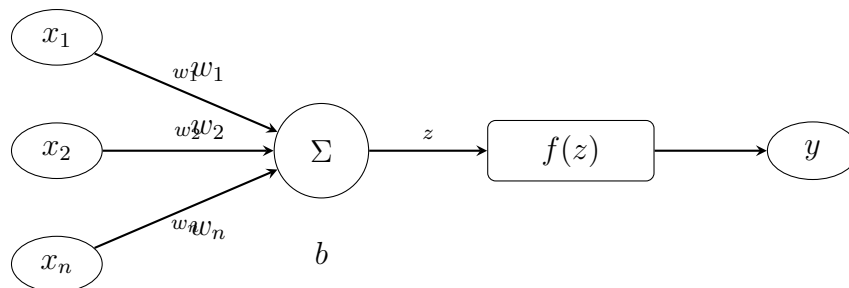


Figure 8.3: Structure of an artificial neuron

8.5.1 Role of Weights

A weight determines how strongly an input contributes to a neuron. A positive weight supports the contribution of the corresponding input, while a negative weight can oppose it. A weight near zero makes that input less influential for the particular neuron.

Weights are normally **learned from data**. They are not usually fixed manually in a trained neural network.

8.5.2 Role of Bias

The bias provides an additional adjustable parameter. Without a bias, the decision boundary of a simple neuron is forced to pass through the origin. Adding a bias gives the neuron freedom to shift that boundary.

The neuron equation can therefore be written in vector form as

$$z = \mathbf{w}^T \mathbf{x} + b.$$

8.6 McCulloch–Pitts (MP) Neuron

The **McCulloch–Pitts neuron**, introduced in the early development of neural-network models, is one of the simplest mathematical models of a neuron. It works with threshold logic: the neuron receives inputs, forms a weighted combination, and produces an output according to a threshold condition.

For a simple binary threshold model,

$$y = \begin{cases} 1, & \text{if } \sum_i w_i x_i \geq \theta, \\ 0, & \text{otherwise.} \end{cases}$$

Here θ is the threshold. An equivalent formulation uses a bias $b = -\theta$:

$$y = \begin{cases} 1, & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0, \\ 0, & \text{otherwise.} \end{cases}$$

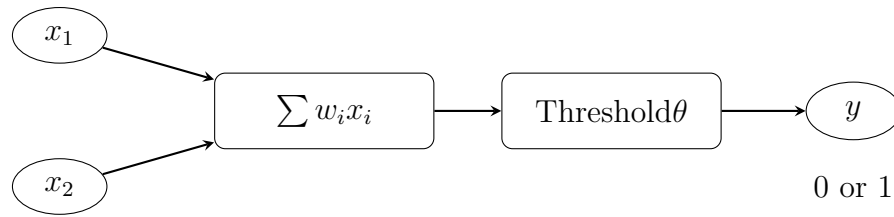


Figure 8.4: Conceptual McCulloch–Pitts threshold neuron

The importance of the MP neuron is not its practical performance. Its importance is that it showed how a neuron-like unit can implement logical decisions using simple mathematical operations.

8.6.1 Example: OR Gate

Consider binary inputs x_1, x_2 and choose weights $w_1 = w_2 = 1$ with threshold $\theta = 1$. Then

$$y = 1 \quad \text{if} \quad x_1 + x_2 \geq 1.$$

Table 8.2: OR operation using a threshold neuron

x_1	x_2	$x_1 + x_2$	Output y
0	0	0	0
0	1	1	1
1	0	1	1
1	1	2	1

This example shows the connection between neural computation and logic. It is a useful conceptual foundation for understanding perceptrons.

8.7 Single-Layer Feed-Forward Perceptron

A **perceptron** is a trainable single-layer neural model used mainly for binary classification. It takes input values, calculates a weighted linear combination, and produces a class output according to a threshold.

A simple perceptron is written as

$$z = \sum_{i=1}^n w_i x_i + b,$$

$$y = \text{step}(z),$$

where the step function can be represented as

$$\text{step}(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

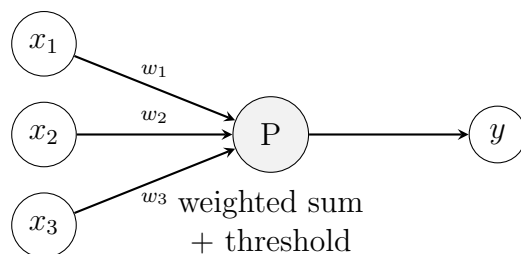


Figure 8.5: Single-layer perceptron

8.7.1 Decision Boundary of a Perceptron

For two input variables, the decision condition

$$w_1x_1 + w_2x_2 + b = 0$$

defines a straight line. One side of the line is assigned to one class and the other side to the other class.

Thus, a single perceptron can solve a problem when the classes are **linearly separable**.

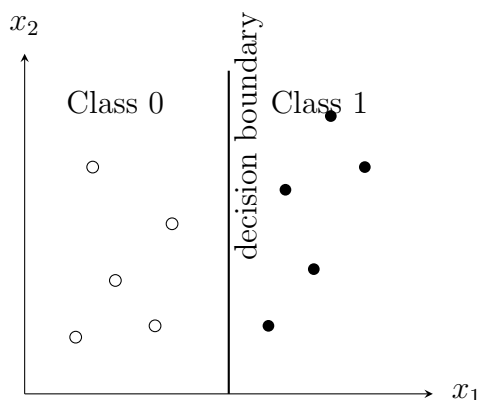


Figure 8.6: Linear decision boundary produced by a single perceptron

8.8 Perceptron Learning Algorithm

The most important idea of the perceptron is that its weights can be learned from labelled examples instead of being fixed manually.

Suppose the target value for an example is t and the perceptron produces y . If the example is correctly classified, the weights need not be changed. If the example is misclassified, the weights are adjusted.

A common perceptron update rule is

$$w_i^{\text{new}} = w_i^{\text{old}} + \eta(t - y)x_i,$$

and for the bias,

$$b^{\text{new}} = b^{\text{old}} + \eta(t - y),$$

where η is the learning rate.

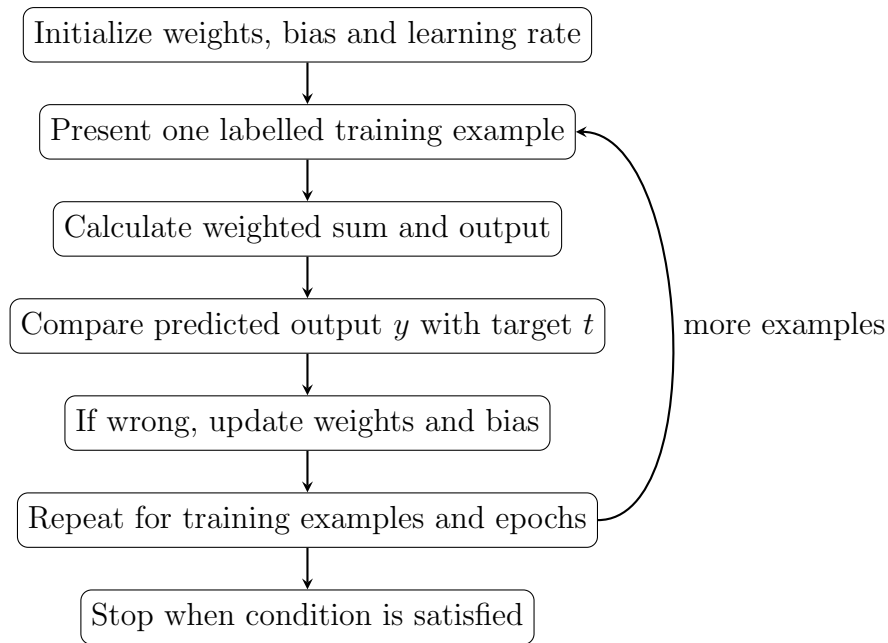


Figure 8.7: Basic perceptron learning procedure

8.8.1 Worked Example: OR Gate

Suppose a perceptron is trained to learn the OR function. The training examples are

$$(0, 0) \rightarrow 0, \quad (0, 1) \rightarrow 1, \quad (1, 0) \rightarrow 1, \quad (1, 1) \rightarrow 1.$$

For a particular training example, the steps are:

1. compute $z = w_1x_1 + w_2x_2 + b$;
2. apply the threshold function;
3. compare the result with the target;
4. if the result is incorrect, update the parameters.

The repeated corrections move the decision boundary until the training examples are correctly separated, provided the dataset is linearly separable.

8.9 Limitations of a Single-Layer Perceptron

The single-layer perceptron is mathematically important, but it has a fundamental limitation: it can represent only a **linear decision boundary**.

Therefore, if the required classes are not linearly separable, the standard perceptron learning rule cannot converge to a perfect separating solution.

8.9.1 The XOR Problem

The XOR function is the classic example. Its truth table is

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

The two positive examples lie diagonally opposite each other, and the two negative examples occupy the other diagonal. No single straight line can separate the two classes perfectly.

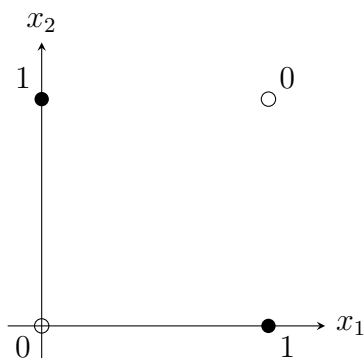


Figure 8.8: XOR pattern: not linearly separable

The XOR problem provides the motivation for introducing networks with **hidden layers**. A multilayer network can combine several simple decision boundaries to construct a more complex decision function.

8.10 Multilayer Perceptron

A **Multilayer Perceptron (MLP)** is a feed-forward neural network containing an input layer, one or more hidden layers, and an output layer. It is built from interconnected neuron-like units, with each layer transforming the output from the previous layer.

An MLP can represent nonlinear relationships because the hidden layers apply nonlinear transformations. Without nonlinear activations, stacking linear layers would still give only an overall linear transformation.

A typical layer can be written as

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

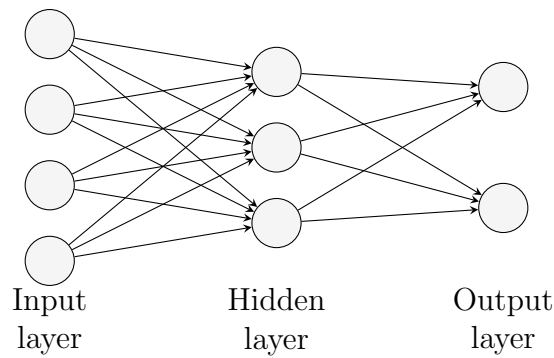


Figure 8.9: A simple multilayer perceptron

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) .$$

Here $W^{(l)}$ is a weight matrix, $\mathbf{b}^{(l)}$ is a bias vector, and $f^{(l)}$ is the activation function for layer l .

8.11 How an MLP Computes an Output

Consider an input vector entering a network with two hidden layers. The information moves forward through the network:

1. input values enter the first layer;
2. each hidden neuron calculates a weighted sum and applies an activation function;
3. the resulting values become inputs to the next hidden layer;
4. the output layer produces the final prediction.

This process is called a **forward pass** or **feed-forward computation**. Information moves from input to output without feedback connections.

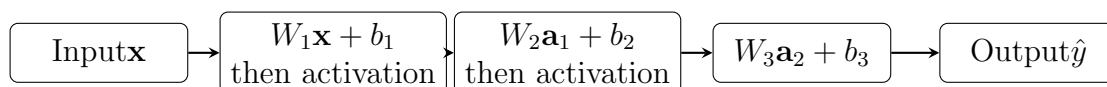


Figure 8.10: Forward flow of information through an MLP

8.12 Activation Functions

An activation function determines how the weighted sum of a neuron is transformed into an output. Activation functions are essential because nonlinear functions allow a neural network to represent nonlinear relationships.

Common activation functions include linear, sigmoid, hyperbolic tangent (tanh), ReLU, Leaky ReLU, PReLU, and softmax. The most important functions for this chapter are explained below.

8.12.1 Step Function

The step function is commonly associated with simple threshold neurons and perceptrons:

$$f(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

It is useful for explaining early neural models, but its sharp threshold makes it unsuitable for gradient-based training of deep networks.

8.12.2 Linear Activation

$$f(z) = z.$$

A linear activation does not introduce nonlinearity. It is useful in some output layers, particularly for regression problems.

8.12.3 Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Its output lies between 0 and 1. It is useful for interpreting a value as a probability-like score in suitable binary-classification settings.

8.12.4 Hyperbolic Tangent (Tanh)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

Its output lies between -1 and 1 and is zero-centered.

8.12.5 ReLU

The Rectified Linear Unit (ReLU) is

$$\text{ReLU}(z) = \max(0, z).$$

ReLU is simple and computationally efficient, and it is widely used in hidden layers of modern neural networks.

8.12.6 Leaky ReLU

A simple version is

$$f(z) = \begin{cases} z, & z \geq 0, \\ \alpha z, & z < 0, \end{cases}$$

where α is a small positive constant.

8.12.7 Softmax

For K output classes, softmax converts scores $z_1, \dots, z_K$ into normalized values:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}.$$

The outputs are non-negative and sum to 1. Therefore, softmax is commonly used at the output of a multi-class classifier when a probability distribution over classes is required.

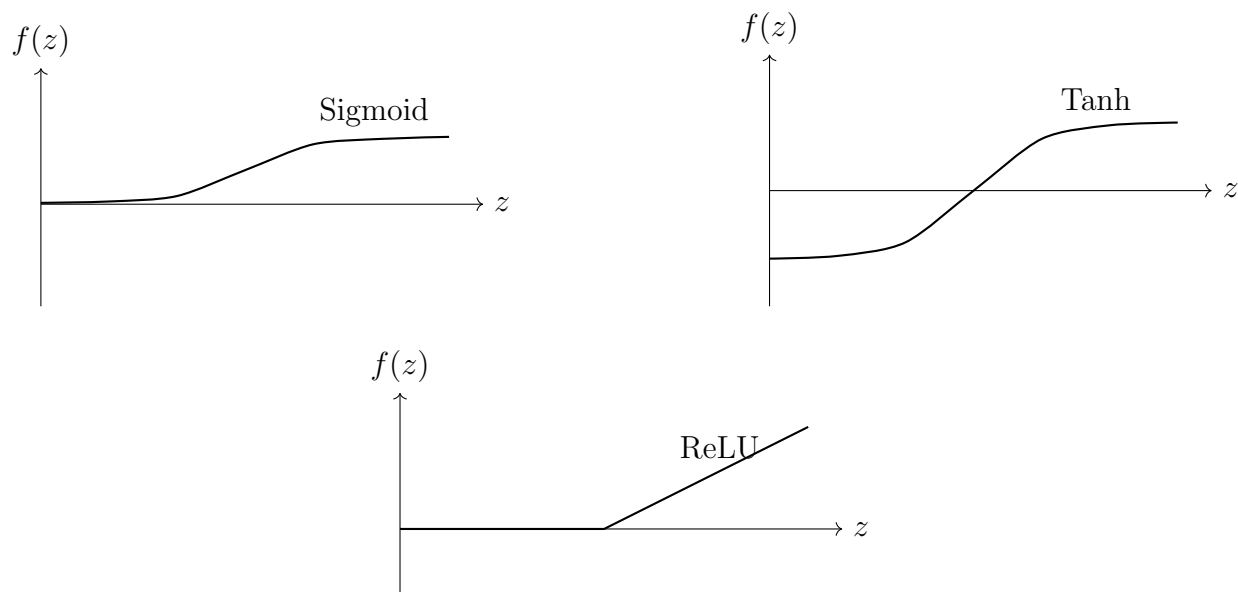


Figure 8.11: Common activation functions: sigmoid, tanh, and ReLU

8.13 Training a Neural Network

Training a neural network means finding weights and biases that make the network perform well on the training examples. A typical supervised training loop contains four major ideas:

1. perform a forward pass;
2. calculate the loss;

Table 8.3: Comparison of common activation functions

Function	Output range	Typical role / observation
Step	discrete values	Threshold decisions; useful for early perceptron models.
Linear	$(-\infty, \infty)$	Useful for some regression outputs.
Sigmoid	$(0, 1)$	Common for binary probability-like output; can suffer from small gradients at extreme inputs.
Tanh	$(-1, 1)$	Zero-centered nonlinear activation.
ReLU	$[0, \infty)$	Efficient nonlinear activation widely used in hidden layers.
Leaky ReLU	$(-\infty, \infty)$	Keeps a small negative slope.
Softmax	$[0, 1]$ with sum 1	Multi-class output distribution.

3. calculate how parameters contributed to the loss;
4. update the parameters.

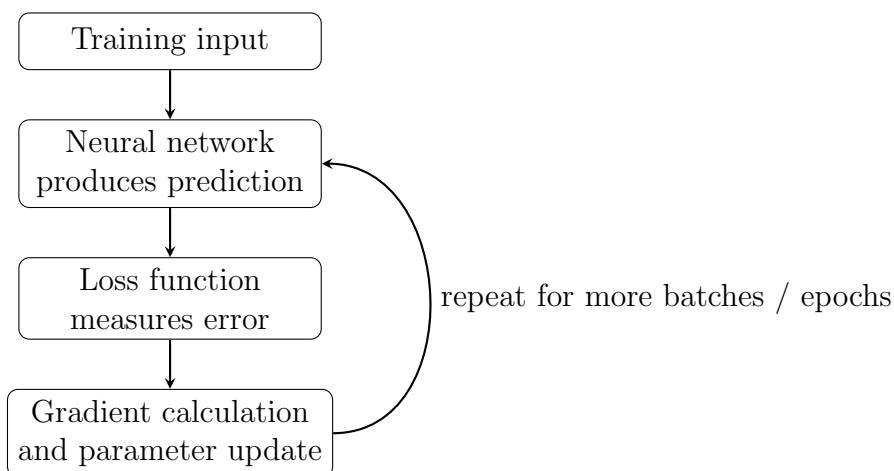


Figure 8.12: Basic supervised training loop for a neural network

8.14 Gradient Descent

Many neural networks are trained by treating learning as an optimization problem. Suppose the parameters of the network are collected in θ and the loss is $L(\theta)$. Gradient descent updates the parameters using

$$\theta_{new} = \theta_{old} - \eta \nabla_{\theta} L(\theta),$$

where η is the learning rate.

The gradient points in the direction of greatest increase of the loss. Therefore, subtracting the gradient moves the parameters in a direction that tends to reduce the loss.

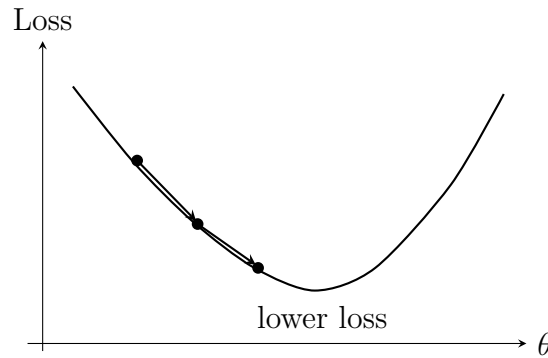


Figure 8.13: Conceptual view of gradient descent moving toward lower loss

The learning rate must be chosen carefully. A very large learning rate can cause unstable updates, while a very small learning rate can make learning unnecessarily slow.

8.15 Backpropagation

For a multilayer network, simply knowing the total error is not enough. We must determine how the error is related to each weight. **Backpropagation** provides an efficient way to calculate these gradients using the chain rule of differentiation.

Backpropagation computes the gradient of the loss with respect to the network weights and propagates the required information backward from the output layer toward earlier layers.

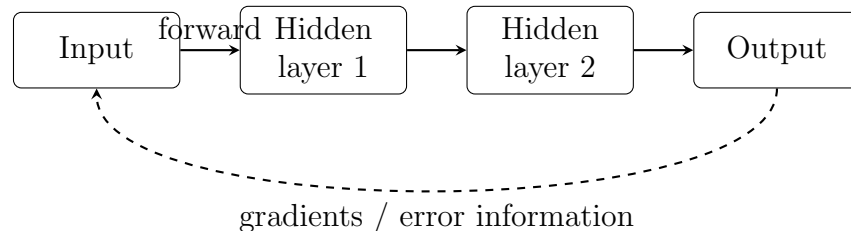


Figure 8.14: Forward computation and backward gradient flow

8.15.1 Main Steps of Backpropagation

1. **Forward pass:** calculate the output of every layer.
2. **Loss calculation:** compare the prediction with the target.
3. **Backward pass:** use derivatives and the chain rule to determine how each parameter affects the loss.
4. **Parameter update:** use an optimization method such as gradient descent to change the weights and biases.

5. **Repeat:** continue for many training examples and epochs until the stopping criterion is satisfied.

The important point is that **backpropagation calculates gradients**; the actual parameter movement is performed by an optimization rule such as gradient descent.

8.16 From Neural Networks to Deep Learning

A neural network becomes **deep** when it contains multiple computational layers that learn a hierarchy of representations. Instead of requiring a programmer to define every intermediate representation manually, a deep network can learn increasingly complex features from data.

For example, in an image task, early layers may learn simple visual patterns such as edges, later layers may respond to textures or parts, and deeper layers may combine those patterns into more meaningful object-level representations.



Figure 8.15: Hierarchical representation learning in a deep network

The development of neural networks can be viewed in three broad waves: early cybernetics and perceptrons, the connectionist period associated with backpropagation, and the modern deep-learning period. This history shows that modern deep learning is the result of several generations of improvements in data, algorithms, computing power, and network design.

8.17 Why Deep Networks Became Practical

Modern deep learning became successful because several factors improved together:

- much larger datasets became available;
- computing hardware, especially GPUs, became more suitable for matrix operations;
- optimization and initialization methods improved;
- activation functions and architectures became more effective;
- large-scale software frameworks simplified model training.

No single factor explains modern deep learning. Deep networks benefit from the interaction of data, computation, algorithms, and suitable architectures.

8.18 Convolutional Neural Networks: Basic Concept

A **Convolutional Neural Network (CNN)** is a neural network architecture designed especially for data with a grid-like structure, particularly images. The syllabus requires only the basic concept here; detailed computer-vision architectures can be studied separately.

The key idea of a CNN is different from a fully connected network. Instead of connecting every neuron to every input pixel, a CNN uses small learnable filters that examine local regions of an image.

This provides two important benefits:

1. **Local connectivity:** nearby pixels are processed together;
2. **Parameter sharing:** the same filter can be applied across many locations.



Figure 8.16: Basic conceptual pipeline of a convolutional neural network

8.18.1 Image as a Matrix

A grayscale image can be represented as a matrix of intensity values. A color image can be represented using multiple channels, commonly red, green, and blue (RGB).

For example, a small grayscale image may be represented as

$$X = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

A CNN can learn filters that respond to useful local patterns in such a matrix.

8.18.2 Convolution Filter or Kernel

A convolution filter, often called a **kernel**, is a small matrix of learnable values. The filter moves across the input and computes a local weighted combination.

In simple terms, the filter asks the same question at many locations: *Does this local pattern occur here?* During training, the filter values are learned from data.

8.18.3 Stride

The **stride** tells the filter how far it moves after each operation. With stride 1, the filter moves one cell at a time. With a larger stride, it moves farther and produces a smaller output feature map.

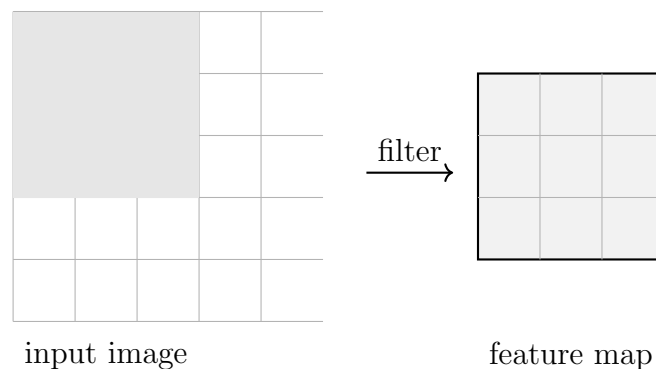


Figure 8.17: A filter scans local regions of an image and produces a feature map

8.18.4 Padding

Padding adds extra values around the border of an input, often zeros. Padding helps preserve spatial dimensions and allows border pixels to participate more fully in convolution operations.

8.18.5 Feature Map

The output produced by applying a filter across an input is called a **feature map**. Different filters can learn different features. One filter may become sensitive to an edge, another to a different orientation, and later filters can combine such responses into more complex patterns.

8.18.6 Pooling

Pooling reduces the spatial size of feature maps. A common operation is **max pooling**, where the maximum value in a small region is retained.

For a simple 2×2 max-pooling window,

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \longrightarrow 4.$$

Pooling can reduce computation and make the representation less sensitive to small local shifts.

8.18.7 Why CNNs are Useful for Images

A fully connected network that directly connects every pixel to every neuron can require a very large number of parameters. CNNs reduce this burden by using local receptive fields and shared filters.

Therefore, CNNs can learn spatial patterns such as edges and textures while preserving the arrangement of information in the image.

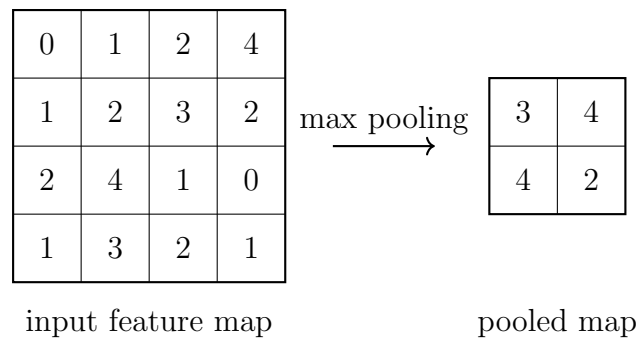


Figure 8.18: Conceptual example of spatial reduction by max pooling

8.19 ANN versus CNN

A CNN is not a completely different kind of learning. It is a specialized neural-network architecture.

Table 8.4: Basic comparison between a fully connected ANN and a CNN

Feature	Fully connected ANN	CNN
Connections	Many or all units connected between layers	Local connections in convolution layers
Parameter sharing	Usually not spatially shared	Same filter is reused across locations
Spatial structure	Often flattened into a vector	Preserved through feature maps
Typical input	General feature vectors	Images and other grid-like data
Key operation	Weighted sum + activation	Convolution + activation, often followed by pooling

8.20 Putting the Ideas Together

The evolution from a simple neuron to deep learning can be understood as a sequence of increasingly expressive models.

Each step addresses limitations of the earlier model or makes it more suitable for a wider class of problems. The single perceptron provides a simple trainable linear classifier. Hidden layers enable nonlinear representations. Deep networks extend this idea to many layers, while specialized architectures such as CNNs exploit the structure of particular data types.

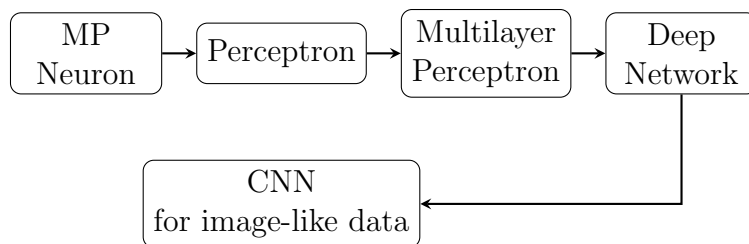


Figure 8.19: Conceptual progression from threshold neurons to deep and specialized neural architectures

8.21 Advantages of Neural Networks

Neural networks are useful because they can learn complex relationships directly from examples.

Important advantages include:

- ability to model nonlinear relationships;
- automatic learning of parameters from examples;
- applicability to classification, regression, recognition, and prediction;
- suitability for high-dimensional data;
- hierarchical representation learning in deep architectures;
- ability to support image, speech, language, and many other applications.

8.22 Limitations of Neural Networks

Neural networks are powerful, but they also have important limitations.

- training can require substantial computation;
- performance depends strongly on the quality and quantity of training data;
- poorly designed models can overfit;
- choosing architecture and hyperparameters can require experimentation;
- many neural networks are difficult to interpret directly;
- training can be sensitive to initialization, optimization, and data representation.

Important limitations include computational burden, overfitting, empirical model development, and the black-box nature of many ANN systems.

8.23 Applications of Neural Networks and Deep Learning

Neural networks have been applied to medical diagnosis, weather prediction, data mining, business analysis, scheduling, and pattern recognition. Modern applications can be organized into several broad categories:

- **Computer vision:** image classification, object detection, recognition, and medical image analysis;
- **Speech processing:** speech recognition and speech-related classification;
- **Natural language processing:** language classification, sequence modelling, and modern language systems;
- **Healthcare:** analysis of scans and other medical data;
- **Finance:** prediction, classification, fraud detection, and risk analysis;
- **Recommendation systems:** ranking and personalization;
- **Robotics:** perception and control;
- **Forecasting:** weather, demand, and other time-dependent predictions.

The detailed study of NLP is deferred to Chapter 9, while this chapter provides the neural-network foundation needed to understand modern language models.

8.24 Chapter Summary

- An Artificial Neural Network is a computational model made of interconnected artificial neurons.
- An artificial neuron computes a weighted sum, adds a bias, and applies an activation function.
- The McCulloch–Pitts neuron provides a simple threshold model for logical decisions.
- A perceptron is a trainable single-layer classifier based on a threshold decision.
- The perceptron learning rule adjusts weights when an example is classified incorrectly.
- A single-layer perceptron can represent only linearly separable decision boundaries.
- XOR is a classic example that cannot be solved by a single linear perceptron.
- A multilayer perceptron introduces hidden layers and nonlinear activation functions.

- Activation functions such as sigmoid, tanh, ReLU, and softmax play different roles in neural networks.
- Training usually involves a forward pass, loss calculation, gradient computation, and parameter update.
- Backpropagation efficiently computes gradients through a multilayer network using the chain rule.
- Deep learning uses multiple layers to learn hierarchical representations.
- CNNs use local filters, parameter sharing, feature maps, and often pooling to process image-like data efficiently.

8.25 Key Points to Remember

1. The basic neuron equation is $y = f(\mathbf{w}^T \mathbf{x} + b)$.
2. Weights control the contribution of inputs.
3. Bias shifts the activation or decision boundary.
4. The MP neuron is a threshold-based computational model.
5. The perceptron is a trainable single-layer classifier.
6. A single perceptron can solve linearly separable classification problems.
7. XOR demonstrates the limitation of a single linear decision boundary.
8. Hidden layers enable multilayer networks to model nonlinear relationships.
9. Nonlinear activation functions are essential for expressive deep networks.
10. Backpropagation calculates gradients; an optimizer such as gradient descent updates parameters.
11. Deep learning is based on multi-layer representation learning.
12. CNNs exploit locality and parameter sharing for grid-like data such as images.

8.26 Short Questions

1. What is an Artificial Neural Network?
2. What is an artificial neuron?

3. Define weight and bias.
4. Write the mathematical equation of an artificial neuron.
5. What is the McCulloch–Pitts neuron?
6. What is a threshold function?
7. What is a perceptron?
8. What is a single-layer feed-forward network?
9. What is meant by a linearly separable problem?
10. State the perceptron learning rule.
11. What is the learning rate?
12. What is the main limitation of a single-layer perceptron?
13. Why can a single-layer perceptron not solve XOR?
14. What is a multilayer perceptron?
15. What is a hidden layer?
16. Why are nonlinear activation functions necessary?
17. Define sigmoid, tanh, and ReLU functions.
18. What is softmax?
19. What is gradient descent?
20. What is backpropagation?
21. What is a loss function?
22. What is deep learning?
23. What is a convolution filter or kernel?
24. What is a feature map?
25. What is pooling in a CNN?
26. What is stride?
27. What is padding?

8.27 Long Questions

1. Explain the architecture and working of an Artificial Neural Network with a suitable diagram.
2. Explain the structure of an artificial neuron. Derive the equation $y = f(\mathbf{w}^T \mathbf{x} + b)$ and explain every term.
3. Explain the McCulloch–Pitts neuron and show how a threshold neuron can implement a logical operation.
4. Explain the structure and working of a single-layer perceptron.
5. Describe the perceptron learning algorithm with the weight and bias update equations.
6. Explain the limitation of the single-layer perceptron using the XOR problem.
7. What is a multilayer perceptron? Explain its architecture and forward computation.
8. Explain the role of activation functions in neural networks. Discuss step, linear, sigmoid, tanh, ReLU, Leaky ReLU, and softmax.
9. Explain gradient descent and its role in neural-network training.
10. Explain the backpropagation algorithm with a suitable diagram. Clearly distinguish gradient calculation from parameter update.
11. What is deep learning? Explain how it differs conceptually from a simple single-layer neural network.
12. Explain the basic architecture of a Convolutional Neural Network.
13. Explain convolution, filter, feature map, stride, padding, and pooling with suitable diagrams or examples.
14. Compare a fully connected neural network and a Convolutional Neural Network.
15. Discuss the advantages and limitations of Artificial Neural Networks.
16. Describe important applications of neural networks and deep learning in real-world systems.

Chapter 9

Natural Language Processing

9.1 Introduction

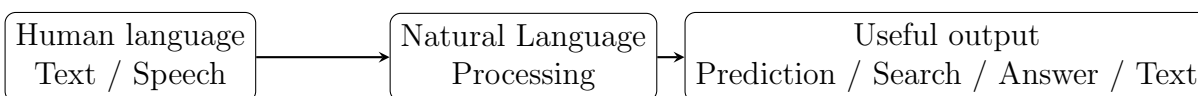
How does a machine understand and use human language?

Natural Language Processing (NLP) is the area of Artificial Intelligence that deals with human language. Human beings communicate through language using words, sentences, speech, and written text. An intelligent computer system, however, does not naturally understand a sentence in the same way that a human reader does. NLP provides methods that allow computers to process, analyze, interpret, search, classify, and generate human language.

A useful keyword to remember is:

NLP = machines dealing with human language.

The reference notes describe NLP as a combination of Computer Science, Linguistics, and Artificial Intelligence, with text and speech as major forms of input and output. They also divide NLP into two broad components: Natural Language Understanding (NLU) and Natural Language Generation (NLG).



analysis + representation + learning + reasoning + generation

Figure 9.1: The basic purpose of Natural Language Processing

NLP is used in search engines, spam filtering, chatbots, speech recognition, machine translation, sentiment analysis, information extraction, and many other systems. The reference material also highlights autocorrect and autocomplete as everyday examples of NLP.

9.2 What is Natural Language Processing?

Natural Language Processing is the study and engineering of methods that enable computers to work with natural human language. Here, “natural language” means a language that humans use for communication, such as English, Bengali, Hindi, French, or Japanese, rather than a programming language designed specifically for computers.

NLP attempts to answer questions such as:

- What words are present in a sentence?
- What role does each word play?
- What does the sentence mean?
- Which meaning is intended when a word has several meanings?
- Which document is relevant to a user’s query?
- What information can be extracted from a paragraph?
- What should a computer say or write as a response?

The same sentence can be easy for a human and difficult for a machine because language is full of ambiguity, context, variation, incomplete information, idioms, and expressions whose meaning depends on the situation.

9.2.1 Natural Language Understanding

Natural Language Understanding (NLU) is concerned with converting human language into useful machine representations and interpreting what the language means. Typical NLU activities include identifying words, grammatical structure, entities, relationships, intent, and meaning.

For example, consider:

“Book a train ticket from Kolkata to Delhi tomorrow.”

An NLU system may identify:

- intent: **book ticket**;
- transport type: **train**;
- source: **Kolkata**;
- destination: **Delhi**;
- date: **tomorrow**.

The sentence has therefore been transformed from a sequence of words into structured information that a software system can use.

9.2.2 Natural Language Generation

Natural Language Generation (NLG) is the reverse direction: a system starts with some internal information and produces a meaningful natural-language sentence.

A simple NLG process can be understood as three stages:

1. **Text planning:** decide what information should be communicated.
2. **Sentence planning:** choose words, phrases, and sentence structure.
3. **Text realization:** produce the final grammatical sentence.

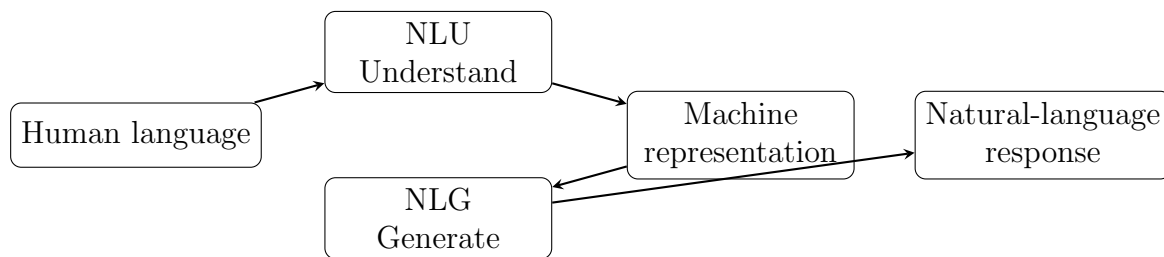


Figure 9.2: Natural Language Understanding and Natural Language Generation

9.2.3 Why Human Language is Difficult for Machines

The difficulty of NLP is not simply the large number of words. The deeper problem is that language carries meaning through context and structure.

Table 9.1: Some important difficulties in NLP

Problem	Explanation
Lexical ambiguity	A word may have more than one meaning. For example, “bank” may refer to a financial institution or the side of a river.
Syntactic ambiguity	The same words may allow more than one grammatical interpretation.
Semantic ambiguity	A sentence may be grammatically correct but have more than one possible meaning.
Context	The meaning of a word or sentence may depend on earlier sentences or the current situation.
Variation	People express the same idea in many different ways.
Informal language	Social media and conversations may contain slang, abbreviations, spelling errors, and emojis.
World knowledge	Some meanings require knowledge about the real world.

9.2.4 A Simple NLP Processing View

An NLP system often performs several transformations before producing a final result. The exact pipeline depends on the application, but the following model is useful for understanding the overall process.

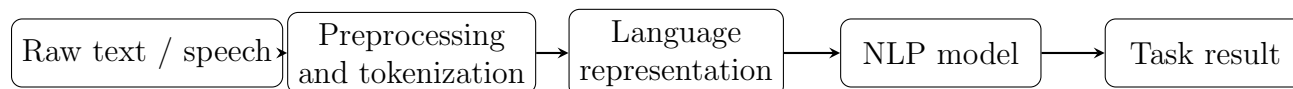


Figure 9.3: A general NLP processing pipeline

Common preprocessing operations include tokenization, normalization, stop-word handling, stemming or lemmatization, and part-of-speech tagging. These steps convert unstructured language into forms that are easier for algorithms to process. The reference notes give tokenization and stemming as examples of early preprocessing operations.

9.3 Language Models

A major problem in NLP is to estimate which words or tokens are likely to occur in a sequence. A **language model** assigns probabilities to sequences of language and can be used to predict likely next tokens.

A simple way to think about it is:

A language model asks: “Given what I have already seen, what is likely to come next?”

For example, after seeing

“The student submitted the ...”

a language model may assign a relatively high probability to words such as “assignment”, “report”, or “project”, depending on the training data and context.

9.3.1 Probability of a Sentence

Suppose a sentence contains a sequence of words

$$w_1, w_2, \dots, w_n.$$

Using the chain rule of probability,

$$P(w_1, w_2, \dots, w_n) = P(w_1)P(w_2 \mid w_1)P(w_3 \mid w_1, w_2) \cdots P(w_n \mid w_1, \dots, w_{n-1}).$$

The formula says that the probability of a complete sentence can be understood as a sequence of conditional predictions. In practice, estimating every possible history is difficult because the number of possible word sequences becomes extremely large.

9.3.2 The N-gram Idea

An **N-gram language model** makes a simplifying assumption: instead of using the entire previous history, it uses only the most recent $n - 1$ words to predict the next word.

For a bigram model,

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-1}).$$

For a trigram model,

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-2}, w_{i-1}).$$

Therefore:

- **Unigram:** uses one word at a time;
- **Bigram:** uses the previous one word;
- **Trigram:** uses the previous two words;
- **Higher-order N-gram:** uses a larger local context.



language model uses context to estimate the next token

Figure 9.4: Basic next-token prediction idea of a language model

9.3.3 Estimating N-gram Probabilities

For a bigram model, a simple maximum-likelihood estimate is

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})},$$

where $C(a, b)$ is the number of times the pair (a, b) occurs and $C(a)$ is the number of times word a occurs in the relevant training corpus.

For example, suppose the word “machine” is followed by “learning” 30 times and “translation” 10 times. If “machine” occurs 100 times in contexts where a following word is counted, then

$$P(\text{learning} \mid \text{machine}) = \frac{30}{100} = 0.30,$$

while

$$P(\text{translation} \mid \text{machine}) = \frac{10}{100} = 0.10.$$

The remaining probability mass is distributed among other possible next words. This simple example shows how a language model converts observed language patterns into numerical probabilities.

9.3.4 The Zero-Probability Problem

A difficulty with simple N-gram models is that a sequence not observed in the training corpus may receive probability zero. This is called the **data sparsity** or **zero-frequency problem**.

For example, if a bigram was never seen in the training corpus, a direct count-based estimator assigns

$$P(w_i | w_{i-1}) = 0.$$

If the sentence probability is formed by multiplying many conditional probabilities, a single zero can make the entire sentence probability zero.

This is one reason why language modelling uses techniques such as smoothing. At an introductory level, smoothing can be understood as redistributing a small amount of probability so that unseen events are not treated as absolutely impossible.

9.3.5 Language Model Applications

Language models can support many tasks:

- autocomplete and next-word prediction;
- spelling correction;
- speech recognition;
- machine translation;
- text summarization;
- question answering;
- dialogue and chatbot systems;
- text generation.

Modern neural language models are much more powerful than traditional N-gram models because they can represent richer context. Transformer-based language models can process long sequences and generate text token by token. The basic idea, however, remains closely related to language prediction: estimate a probability distribution over the next token given the available context.

9.4 Text Classification

Text classification is the task of assigning a piece of text to one or more predefined categories. In simple terms, the system reads text and decides which label best describes it.

Examples include:

- spam vs. non-spam email;
- positive vs. negative sentiment;
- complaint vs. enquiry vs. feedback;
- news articles grouped by topic;
- support tickets routed to different departments.

The reference notes describe text classification as text tagging or text categorization, where an NLP system assigns predefined tags or categories based on the content of the text.

9.4.1 Types of Text Classification

1. **Binary classification:** exactly two classes, such as spam/not spam.
2. **Multi-class classification:** one input is assigned to one of several classes, such as sports/politics/business/technology.
3. **Multi-label classification:** one input can belong to several classes at the same time.

9.4.2 Basic Text Classification Pipeline

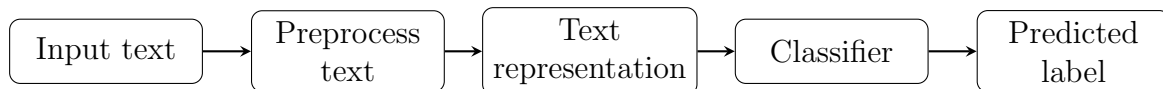


Figure 9.5: Basic pipeline for text classification

The text must first be represented in a numerical form. Traditional systems may use counts, binary word presence, or TF-IDF weights. Neural systems may instead learn dense representations such as embeddings.

9.4.3 Bag-of-Words Representation

A simple representation is the **bag-of-words** model. It creates a vocabulary and represents a document by the number of times each vocabulary word appears.

Suppose the vocabulary is

[AI, data, learning, text].

For the sentence “AI learning uses data”, the count vector becomes

[1, 1, 1, 0].

The bag-of-words model is easy to understand, but it ignores word order. The sentences

“The student helps the teacher.”

and

“The teacher helps the student.”

have similar word counts even though the roles of the words are different.

9.4.4 TF-IDF Representation

A classical method for assigning importance to words is **TF-IDF**, which combines term frequency with inverse document frequency.

A simplified definition is

$$\text{TFIDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t),$$

where

$$\text{IDF}(t) = \log \left(\frac{N}{df(t)} \right).$$

Here N is the number of documents and $df(t)$ is the number of documents containing term t .

A term that appears often in one document but rarely across the whole collection receives a larger weight. Very common words receive lower importance because they occur in many documents.

9.4.5 Training a Classifier

During supervised learning, a training set contains examples of text together with their known labels. A classifier learns parameters that map text representations to the correct classes.

A simple conceptual example is:

Training text	Label
“Win a free prize now”	Spam
“Meeting at 3 PM”	Not spam
“Congratulations! You won cash”	Spam
“Please submit the report”	Not spam

After learning from such examples, the classifier can receive a new message and predict its category.

9.4.6 Evaluating Text Classification

A classifier should not be judged only by the number of correct predictions. A confusion matrix records how the predicted classes compare with the actual classes.

For binary classification, common measures are:

$$\text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN},$$

and

$$F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision asks: *Among the items predicted as positive, how many were actually positive?*

Recall asks: *Among the truly positive items, how many did the system find?*

The F_1 score combines the two using their harmonic mean.

9.5 Information Retrieval

Information Retrieval (IR) is the process of finding relevant information from a collection of documents in response to a user's information need.

In simple words:

Information Retrieval = finding the most relevant information for a query.

A web search engine is a familiar example. A user provides a query, the system searches a large collection of documents, ranks potentially relevant results, and presents the most useful items first.

The reference notes explain that the general objective of an information retrieval system is to reduce the effort or overhead required by a user to locate needed information. They also identify **precision** and **recall** as two important measures of retrieval performance.

9.5.1 Basic Information Retrieval System

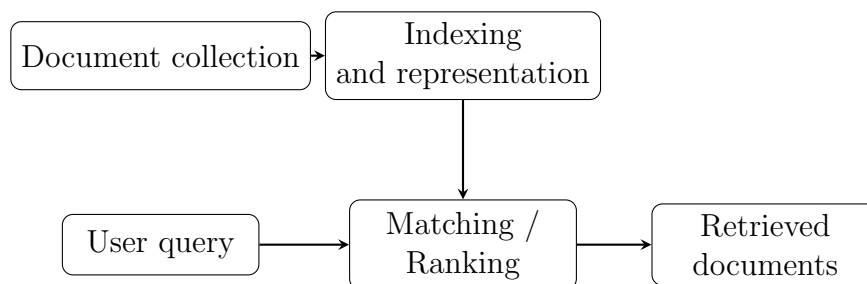


Figure 9.6: Basic information retrieval workflow

The major stages are:

1. prepare and index the document collection;
2. accept a user query;
3. compare the query with the indexed documents;

4. rank or select the relevant documents;
5. present the results to the user.

9.5.2 The Inverted Index

A key data structure used in many retrieval systems is an **inverted index**. Instead of storing only a document and its words, it maintains a mapping from each term to the documents in which the term occurs.

For example:

Term	Documents containing the term
AI	D1, D3, D5
learning	D1, D2, D5
search	D2, D4
text	D1, D4, D5

For the query “AI learning”, the system can quickly locate documents associated with both terms and combine the evidence.

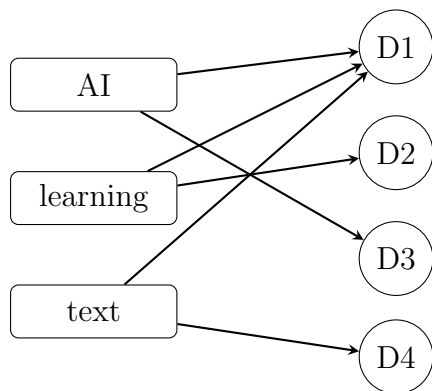


Figure 9.7: Conceptual inverted index: terms point to documents containing them

The inverted index makes retrieval efficient because the system does not have to scan every word of every document for each query.

9.5.3 Boolean Retrieval

A basic retrieval system can use Boolean logic. The common operators are:

- **AND**: both terms must be present;
- **OR**: at least one term must be present;
- **NOT**: exclude documents containing a term.

For example:

AI AND learning $\rightarrow$ documents containing both terms,
 AI OR learning $\rightarrow$ documents containing at least one term,
 AI AND NOT robotics $\rightarrow$ documents containing AI but not robotics.

The reference material also discusses proximity search, exact or contiguous phrases, fuzzy search, and term masking as ways of making retrieval more flexible. For example, a fuzzy search can help locate words whose spelling is similar to the query term.

9.5.4 Precision and Recall in Information Retrieval

A retrieval system usually deals with four groups:

1. relevant and retrieved;
2. relevant and not retrieved;
3. non-relevant and retrieved;
4. non-relevant and not retrieved.

Let

- TP = relevant documents retrieved;
- FP = non-relevant documents retrieved;
- FN = relevant documents not retrieved.

Then

$$\text{Precision} = \frac{TP}{TP + FP}$$

and

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Relevant and Retrieved	Non-relevant and Retrieved
Relevant and Not Retrieved	Non-relevant and Not Retrieved

Four possible outcomes for a retrieval query

Figure 9.8: Basic relevance outcomes in information retrieval

Precision is especially concerned with avoiding irrelevant results, while recall is concerned with finding as many relevant items as possible.

9.6 Information Extraction

Information Extraction (IE) is the process of automatically identifying and extracting specific pieces of useful information from unstructured or semi-structured text.

Search asks:

“Which document contains useful information?”

Information extraction asks:

“Which exact facts can I take out of the text?”

For example, consider:

“Dr. Ananya Sen joined Regent University on 15 July 2026 as an Assistant Professor.”

An information extraction system might produce:

Name	Ananya Sen
Organization	Regent University
Date	15 July 2026
Designation	Assistant Professor

9.6.1 Named Entity Recognition

A common IE task is **Named Entity Recognition (NER)**. NER identifies spans of text that refer to entities such as:

- person names;
- organizations;
- locations;
- dates and times;
- monetary values;
- products or other domain-specific entities.

Consider:

“Somnath visited Kolkata on Monday.”

A simplified entity representation could be:

Somnath / PERSON Kolkata / LOCATION Monday / DATE

9.6.2 Relation Extraction

Finding entities is often not enough. An NLP system may also need to identify how the entities are related.

For example:

“Riya works at ABC Technologies.”

The extracted relation can be represented as:

works extunderscore at(Riya, ABC Technologies).

This converts language into a structured relation that can be stored in a database or knowledge graph.

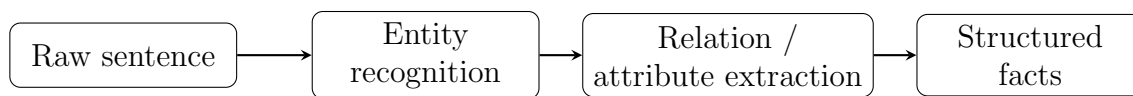


Figure 9.9: Conceptual information extraction pipeline

9.6.3 Information Extraction Example

Suppose the input is:

“OpenAI released a new model in San Francisco in March 2026.”

An extraction system might identify:

- organization: **OpenAI**;
- event: **released**;
- object: **a new model**;
- location: **San Francisco**;
- date: **March 2026**.

The result may be stored in structured form such as:

Release(OpenAI, new model, San Francisco, March 2026).

This example shows why information extraction is useful: the original paragraph is unstructured text, while the extracted representation can be searched, counted, analyzed, or inserted into a database.

9.6.4 Information Extraction and Information Retrieval

The two ideas are closely related but are not identical.

Table 9.2: Information Retrieval versus Information Extraction

Feature	Information Retrieval	Information Extraction
Main question	Which documents are relevant?	Which facts can be extracted?
Output	Documents, passages, or ranked results	Structured entities, relations, dates, values, and events
Input	Usually a collection of documents	Usually individual documents or passages
Example	Find articles about AI in healthcare	Extract disease names and treatment dates from an article

9.7 Putting the Major NLP Tasks Together

NLP tasks often work together rather than operating independently. Consider a student-help chatbot. A simplified workflow may be:

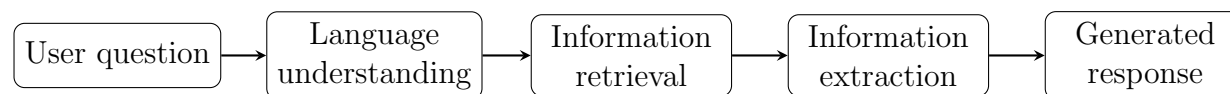


Figure 9.10: How multiple NLP capabilities can cooperate in one system

For example, the system may first interpret the user's question, retrieve relevant documents, extract the required facts, and finally generate a readable answer. This illustrates an important principle: NLP is not a single algorithm. It is a collection of methods for working with different aspects of human language.

9.8 Applications of NLP

NLP is used in many practical systems:

- **Search engines:** understanding queries and retrieving relevant documents;
- **Email filtering:** detecting spam and categorizing messages;
- **Chatbots and assistants:** understanding user questions and generating responses;
- **Machine translation:** translating text between languages;
- **Speech recognition:** converting spoken language into text;
- **Sentiment analysis:** identifying positive, negative, or neutral opinions;
- **Information extraction:** extracting names, locations, dates, events, and relationships;

- **Document analysis:** organizing large collections of reports, articles, and records;
- **Autocomplete and autocorrect:** predicting likely words and correcting probable errors.

9.9 Limitations and Challenges of NLP

Despite major progress, NLP systems still face important challenges.

- Language is ambiguous and context-dependent.
- The same idea may be expressed in many different forms.
- New words, slang, spelling errors, and domain-specific terms are constantly introduced.
- Understanding meaning may require world knowledge.
- Different languages have very different grammatical and morphological structures.
- Long documents may require the system to preserve information over large contexts.
- Training data may contain noise, bias, or incomplete coverage of real-world language.

The reference material emphasizes that multilingual processing is particularly challenging because languages differ in word formation, grammar, and meaning representation.

9.10 Chapter Summary

- Natural Language Processing is the area of AI concerned with machines dealing with human language.
- NLP can work with written text as well as speech.
- Natural Language Understanding converts language into useful machine representations, while Natural Language Generation produces natural-language output.
- Language models estimate the probability of word or token sequences and can predict likely next tokens.
- N-gram models approximate language probability using a limited previous context.
- Text classification assigns predefined categories to text.
- Information Retrieval finds relevant documents or passages for a user query.
- An inverted index maps terms to the documents in which they occur and supports efficient retrieval.

- Precision measures the proportion of retrieved items that are relevant, while recall measures the proportion of relevant items that are retrieved.
- Information Extraction identifies specific facts such as entities, relations, dates, and events from text.
- NLP tasks often cooperate: a real system may understand a query, retrieve documents, extract facts, and generate an answer.

9.11 Key Points to Remember

1. Remember the keyword: **NLP = machines dealing with human language**.
2. NLU focuses on understanding; NLG focuses on generation.
3. A language model predicts or assigns probabilities to language sequences.
4. In an N-gram model, only a limited recent context is used for prediction.
5. Text classification maps text to predefined categories.
6. Bag-of-words is simple but ignores word order.
7. TF-IDF gives more importance to terms that are important within a document but less common across the collection.
8. Information Retrieval finds relevant information in a document collection.
9. An inverted index is a key data structure for efficient retrieval.
10. Precision and recall measure different aspects of retrieval quality.
11. Information Extraction converts unstructured text into structured facts.
12. Named Entity Recognition identifies entities such as people, organizations, locations, and dates.
13. Retrieval asks “which documents?” while extraction asks “which facts?”

9.12 Short Questions

1. What is Natural Language Processing?
2. What is meant by natural language?
3. Define Natural Language Understanding.

4. Define Natural Language Generation.
5. Differentiate between NLU and NLG.
6. Why is natural language difficult for computers to process?
7. What is a language model?
8. What is next-token prediction?
9. Define unigram, bigram, and trigram models.
10. Write the bigram probability formula.
11. What is the zero-frequency problem in language modelling?
12. What is text classification?
13. What is binary classification?
14. What is multi-class classification?
15. What is a bag-of-words representation?
16. What is TF-IDF?
17. What is Information Retrieval?
18. What is an inverted index?
19. What are Boolean retrieval operators?
20. Define precision and recall.
21. What is Information Extraction?
22. What is Named Entity Recognition?
23. What is relation extraction?
24. Differentiate between information retrieval and information extraction.
25. Mention any four applications of NLP.

9.13 Long Questions

1. Explain Natural Language Processing. Discuss its major components, NLU and NLG, with suitable examples.

2. Explain the major challenges involved in processing human language.
3. Explain a general NLP processing pipeline with a suitable diagram.
4. What is a language model? Explain the basic idea of next-token prediction.
5. Derive the chain-rule representation of sentence probability and explain the N-gram approximation.
6. Explain unigram, bigram, and trigram language models with examples.
7. Explain the zero-frequency problem in N-gram language models and describe the basic purpose of smoothing.
8. Explain text classification and its major types. Describe a typical text-classification pipeline.
9. Explain the bag-of-words and TF-IDF representations used in text classification.
10. Explain precision, recall, and F_1 score for a text-classification task.
11. What is Information Retrieval? Explain the major components of an information retrieval system.
12. Explain the construction and use of an inverted index with an example.
13. Explain Boolean retrieval using AND, OR, and NOT operators.
14. Explain precision and recall in Information Retrieval with suitable examples.
15. What is Information Extraction? Explain Named Entity Recognition and relation extraction with examples.
16. Compare Information Retrieval and Information Extraction.
17. Explain how language models, text classification, information retrieval, and information extraction can work together in a practical NLP system.
18. Discuss major applications and challenges of Natural Language Processing.

Chapter 10

Expert Systems: Foundations and Architecture

10.1 Introduction

How can a computer use stored expert knowledge to reason about a problem and provide a useful recommendation?

An **expert system** is an Artificial Intelligence system designed to solve problems in a specific domain by using stored expert knowledge and reasoning procedures. Instead of learning every rule from a large dataset, a classical expert system stores domain knowledge explicitly and applies that knowledge to facts supplied by the user or the environment.

This chapter focuses on the expert-system topics required in the syllabus: **foundations and architecture, knowledge representation and rules, inference mechanisms, handling uncertainty, and knowledge acquisition and engineering**. Historical examples, applications, expert-system shells, advantages, and limitations are included to connect the concepts to real systems.

A useful sentence to remember is:

Expert system = domain knowledge + reasoning + user interaction.

The reference notes define an expert system as a computer-based system that uses knowledge, facts, and reasoning to solve problems that normally require human expertise. They also describe it as a knowledge-based system using a knowledge base and an inference engine.

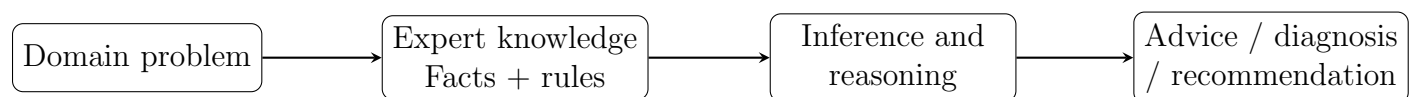


Figure 10.1: Basic idea of an expert system

10.2 What is an Expert System?

An **expert system** is an AI system that attempts to reproduce the decision-making or problem-solving ability of a human expert within a particular domain. The important phrase is **within a particular domain**. A traditional expert system is not a general-purpose human mind. Its knowledge is selected, organized, and encoded for a specific area such as medical diagnosis, equipment troubleshooting, finance, manufacturing, or geological exploration.

An expert system can therefore be viewed from several related perspectives:

- As a **knowledge-based system**, it stores structured domain knowledge.
- As a **reasoning system**, it applies rules or other inference mechanisms to facts.
- As a **decision-support system**, it provides recommendations, diagnoses, or guidance.
- As a **consultative system**, it interacts with a user and may explain the reasoning behind its answer.

10.2.1 Why Expert Systems Were Important

The development of expert systems was an important stage in the history of AI because it showed that a computer could perform useful domain-specific reasoning without being programmed as a simple sequence of fixed instructions. The reference material describes early development in the 1960s, the appearance of systems such as MYCIN in the 1970s, wider business and industrial use in the 1980s, and later integration with broader AI techniques.

10.3 Expert Systems, AI, and Knowledge-Based Systems

Expert systems are closely related to AI and knowledge-based systems, but the terms should not be treated as identical.

Table 10.1: Relationship among AI, knowledge-based systems, and expert systems

Concept	Meaning
Artificial Intelligence	Broad field concerned with building systems that perform tasks associated with intelligent behavior.
Knowledge-Based System	A system that explicitly stores knowledge and uses it to solve a problem or support reasoning.
Expert System	A specialized knowledge-based AI system intended to reproduce or support expert-level reasoning in a particular domain.

A useful way to remember the relationship is:

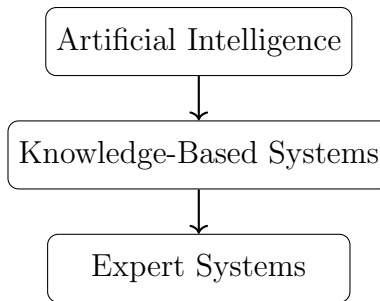


Figure 10.2: Conceptual relationship between AI, knowledge-based systems, and expert systems

The relationship is conceptual rather than a claim that every AI system must contain an expert system. Modern AI also includes machine learning, neural networks, NLP, computer vision, and other approaches.

10.4 Goals and Characteristics of Expert Systems

The main goal is to make specialized knowledge useful to people who may not themselves be experts in that domain. The reference lecture notes identify decision support, efficiency, accessibility of expertise, consistency, training, quality improvement, and cost reduction as major goals.

10.4.1 Major Goals

- **Assist decision-making:** provide recommendations based on available facts and domain rules.
- **Increase efficiency:** automate repetitive and knowledge-intensive tasks.
- **Make expertise accessible:** allow users to obtain specialized guidance even when a human expert is not immediately available.
- **Improve consistency:** apply the same encoded rules repeatedly instead of changing decisions because of fatigue or mood.
- **Support training:** show examples and explanations that can help learners understand domain reasoning.
- **Improve reliability:** produce stable results when the knowledge base and rules are correct.

10.4.2 Characteristics

Typical characteristics of an expert system include:

- high performance in a specific domain;
- understandable interaction with the user;
- rapid response to domain queries;
- explicit knowledge representation;
- reasoning through an inference mechanism;
- the ability to provide an explanation when the system records the reasoning path.

10.5 Architecture of an Expert System

The architecture is one of the most important examination topics. A basic expert system consists of a **user interface**, **knowledge base**, and **inference engine**. In a fuller architecture, we also identify **working memory**, an **explanation facility**, and **knowledge acquisition tools**.

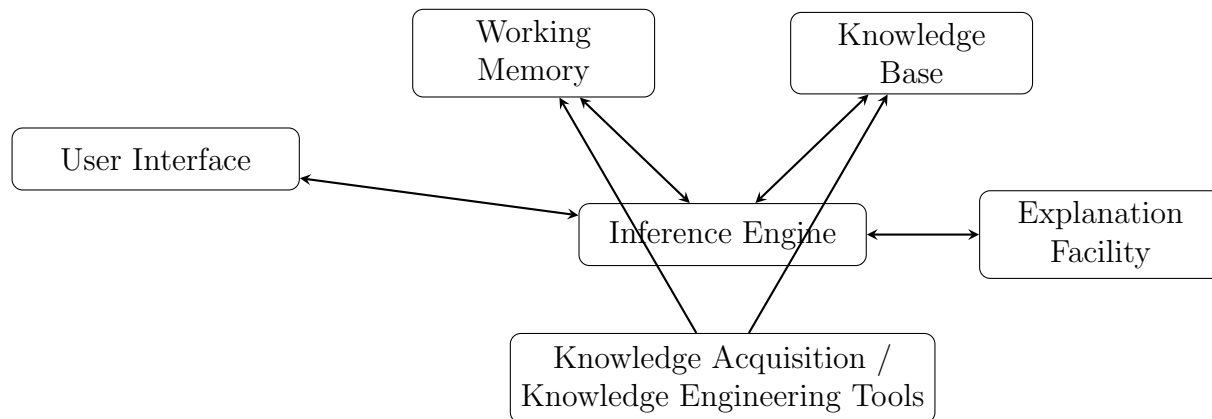


Figure 10.3: General architecture of an expert system

10.5.1 Knowledge Base

The **knowledge base (KB)** stores domain knowledge. It may contain:

- facts about objects, situations, and relationships;
- rules describing domain relationships;
- concepts and their attributes;
- other structured information needed for reasoning.

For example, a simple medical knowledge base might contain:

```
Fact: patient_has(patient1, fever)
Fact: patient_has(patient1, cough)
Rule: IF fever AND cough THEN respiratory_infection
```

The important point is that the knowledge is stored explicitly rather than being hidden completely inside numerical model parameters.

10.5.2 Inference Engine

The **inference engine** is the reasoning part of the expert system. It takes facts, examines applicable rules, derives new information, and eventually produces a conclusion or recommendation.

It can be thought of as the “brain” of a classical rule-based expert system, because the knowledge base stores what the system knows while the inference engine decides how to use that knowledge.

10.5.3 Working Memory

Working memory stores facts relevant to the current consultation or problem. Some systems receive initial facts from the user and then add new facts as rules fire.

For a diagnosis problem, working memory might contain:

```
patient = P1
fever = yes
cough = yes
breathing_difficulty = yes
```

The inference engine can use these temporary facts to derive additional conclusions.

10.5.4 User Interface

The **user interface** connects the human user and the expert system. It collects questions, facts, or observations and presents recommendations, conclusions, or explanations in a readable form.

10.5.5 Explanation Facility

An expert system may provide an **explanation facility**. It can show the rules and reasoning steps that led to a result. This is important because users may need to know not only *what* the system concluded, but also *why* it reached that conclusion.

10.5.6 Knowledge Acquisition Tools

Knowledge acquisition tools help gather and structure knowledge from human experts. The provided reference notes mention interview templates, questionnaires, knowledge-base editors, and rule builders as examples of such tools.

10.6 Knowledge Representation in Expert Systems

Knowledge representation is the way domain knowledge is organized so that a computer can store and reason with it. Chapter 5 introduced knowledge representation using rules, and Chapter 6 connected logical reasoning with Prolog. Expert systems bring these ideas together into a working decision system.

10.6.1 Facts

A **fact** expresses information accepted as true within the current knowledge domain or consultation.

Examples:

- “The patient has fever.”
- “Engine temperature is high.”
- “The machine is not receiving power.”

In symbolic form, a fact may be written as:

$$Fever(P1).$$

10.6.2 Rules

A **rule** connects conditions to a conclusion. The common form is:

IF condition(s) THEN conclusion.

For example:

IF $Fever(x) \wedge Cough(x)$ THEN $RespiratoryInfection(x)$.

The condition part is called the **antecedent** or premise, and the result part is called the **consequent**.

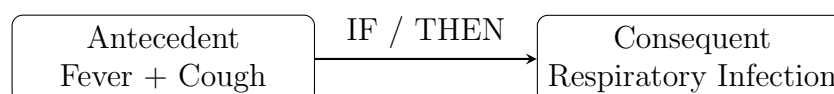


Figure 10.4: Basic structure of an expert-system rule

10.6.3 Frames

A **frame** organizes knowledge about an object or concept as a collection of attributes and values. The reference material describes frame-based systems as representations in which a concept has associated attributes and values.

A simplified medical frame might be:

```
FRAME: Patient
Name: P1
Age: 40
Fever: Yes
Cough: Yes
BreathingDifficulty: Yes
```

Frames are useful when an expert system must keep several properties together rather than treating every fact as an isolated statement.

10.6.4 Knowledge Representation Forms

Different expert systems may use different representations.

Table 10.2: Common knowledge representation forms in expert systems

Representation	Main idea
Facts	Store individual statements about a domain.
Rules	Store conditional knowledge in IF–THEN form.
Frames	Group attributes and values around concepts or objects.
Logic	Represent relationships formally so that logical inference can be applied.
Ontology	Organize concepts, relationships, rules, axioms, and instances in a structured domain model.

The reference material describes ontology as a structured representation of concepts, relationships, rules, axioms, and instances that can support reasoning and knowledge sharing.

10.7 Inference Mechanisms

The inference engine applies reasoning strategies to the knowledge base and the current facts. Two fundamental rule-based approaches are **forward chaining** and **backward chaining**.

10.7.1 Forward Chaining

Forward chaining is **data-driven reasoning**. It begins with known facts and repeatedly applies rules whose conditions are satisfied. The reasoning moves from facts toward conclusions.

A simple example is:

$$A, B$$

$$A \wedge B \rightarrow C$$

$$C \wedge D \rightarrow E$$

If A , B , and D are known, the system can first derive C and then derive E .

The question answered by forward chaining is:

“What can happen next from the facts I already have?”

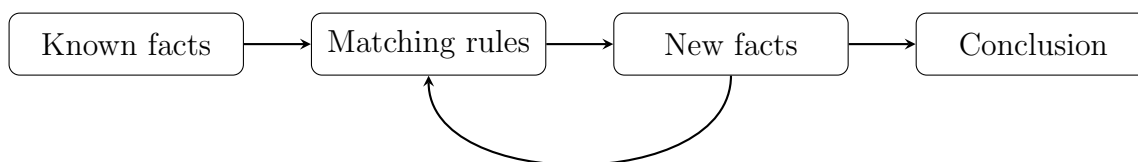


Figure 10.5: Forward chaining: reasoning from facts toward a conclusion

10.7.2 Backward Chaining

Backward chaining is **goal-driven reasoning**. It begins with a desired conclusion or hypothesis and asks which rules could establish it. The system then tries to prove the conditions of those rules.

Suppose the goal is:

$$\text{EngineFailure}(M1).$$

If the knowledge base contains:

$$\text{Overheat}(x) \wedge \text{NoPower}(x) \rightarrow \text{EngineFailure}(x),$$

then the system asks whether $\text{Overheat}(M1)$ and $\text{NoPower}(M1)$ can be established.

The question answered by backward chaining is:

“Why should this conclusion be true, and what conditions would prove it?”



Figure 10.6: Backward chaining: reasoning from a goal toward supporting facts

Table 10.3: Comparison of forward and backward chaining

Feature	Forward Chaining	Backward Chaining
Direction	Facts $\rightarrow$ conclusions	Goal $\rightarrow$ supporting facts
Style	Data-driven	Goal-driven
Starting point	Known observations	Desired conclusion or hypothesis
Typical question	“What can happen next?”	“Why is this conclusion possible?”
Useful for	Deriving consequences from incoming facts	Checking or proving a specific goal

10.7.3 Forward Chaining vs. Backward Chaining

10.8 Handling Uncertainty

Real-world information is not always certain. A patient may show symptoms without a single guaranteed disease. A sensor may be noisy. A financial signal may suggest risk without proving it. Expert systems therefore need methods for reasoning when information is incomplete, uncertain, or imprecise.

The reference material distinguishes deterministic inference, where conclusions are treated as true under the encoded rules, from probabilistic inference, where conclusions carry uncertainty based on probability. It also describes fuzzy systems for imprecise or partially true information.

10.8.1 Sources of Uncertainty

Uncertainty may arise from:

- unreliable information sources;
- measurement or experimental errors;
- faulty equipment;
- incomplete observations;
- changing environmental conditions;
- insufficient knowledge about the domain.

10.8.2 Deterministic Reasoning

In a deterministic rule system, if the required facts are true and the rule applies, the conclusion is derived as true.

For example:

$$\text{Fever}(P1) \wedge \text{Cough}(P1) \rightarrow \text{RespiratoryInfection}(P1).$$

If both facts are known to be true, the rule produces the conclusion according to the system's logic.

10.8.3 Probabilistic Reasoning

Probabilistic reasoning represents uncertainty using probabilities. Instead of saying only “disease” or “no disease,” the system may estimate how likely a hypothesis is given the evidence.

Bayes' rule is a common foundation:

$$P(H | E) = \frac{P(E | H)P(H)}{P(E)},$$

where:

- H is a hypothesis,
- E is observed evidence,
- $P(H)$ is the prior probability,
- $P(E | H)$ is the likelihood of the evidence,
- $P(H | E)$ is the posterior probability.

The reference notes use a medical example to show how evidence such as a stiff neck can change the probability of a disease hypothesis.

10.8.4 Fuzzy Reasoning

Fuzzy systems handle situations in which concepts are not naturally limited to strict true/false values. For example, temperature may be described as “low,” “medium,” or “high” with different degrees rather than a single sharp boundary.

The reference material describes fuzzy expert systems as systems that manage uncertain or imprecise information using partial truths instead of only binary true/false values.

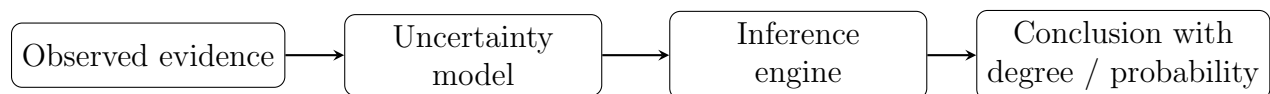


Figure 10.7: Conceptual flow for reasoning under uncertainty

10.8.5 Confidence in Rules

The reference book also allows a confidence factor to be associated with a rule. Conceptually, this allows a system to distinguish between a rule that is strongly supported and one that is less reliable.

For example:

IF Fever AND Cough THEN Infection $[CF = 0.8]$

The precise way a confidence value is combined with other evidence depends on the expert-system design. The important idea is that the system does not always need to treat every rule as equally certain.

10.9 How an Expert System Works

The working process can be understood through a simple diagnosis example.

Suppose the knowledge base contains:

R1: IF fever AND cough THEN infection

R2: IF infection AND breathing_difficulty THEN urgent_case

The user supplies:

fever = yes

cough = yes

breathing_difficulty = yes

The inference engine can proceed as follows:

1. Read the initial facts into working memory.
2. Find rules whose conditions match the available facts.
3. Fire the first applicable rule and add its conclusion.
4. Repeat the process with the newly derived fact.
5. Present the final conclusion through the user interface.
6. If supported, provide the reasoning steps through the explanation facility.

10.10 Knowledge Acquisition and Knowledge Engineering

One of the hardest parts of building a classical expert system is not writing the inference code. It is obtaining good expert knowledge and converting it into a representation that the machine can use.

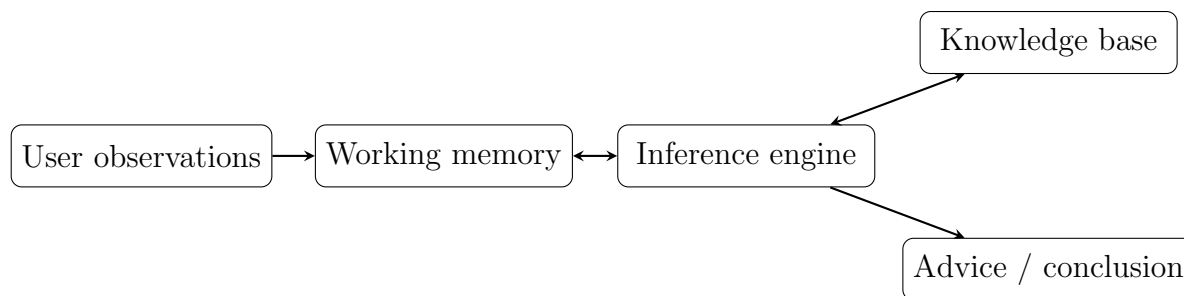


Figure 10.8: Typical consultation cycle of an expert system

Knowledge acquisition is the process of gathering domain knowledge from human experts, documents, cases, measurements, and other sources. The resulting knowledge must then be organized, encoded, tested, and maintained.

A person who performs this work is commonly called a **knowledge engineer**. The knowledge engineer acts as a bridge between domain experts and the software system.

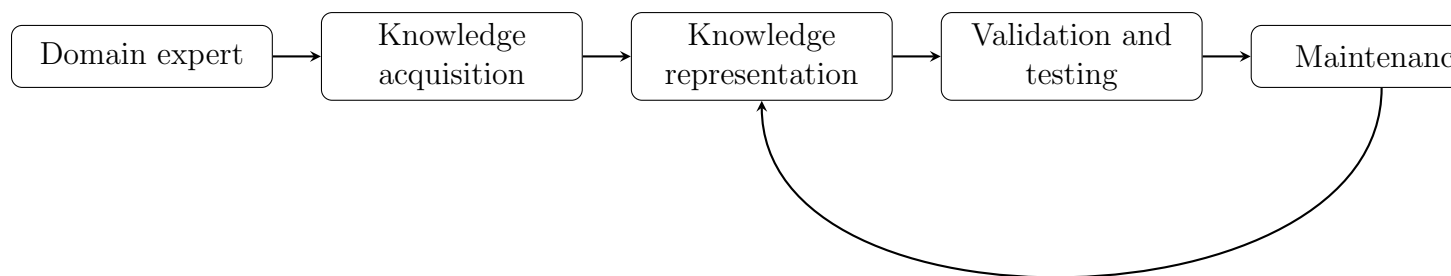


Figure 10.9: Conceptual knowledge-engineering cycle

10.10.1 Sources of Knowledge

Knowledge may be gathered from:

- interviews with domain experts;
- expert questionnaires;
- manuals, reports, and documents;
- previously solved cases;
- measurements and observations;
- existing databases and structured records.

10.10.2 The Knowledge Acquisition Bottleneck

The **knowledge acquisition bottleneck** refers to the difficulty of obtaining expert knowledge in a complete, precise, and machine-usable form.

An expert may be able to make a decision quickly but may find it difficult to explain every small step used in that decision. Some knowledge is also based on experience, intuition, exceptions, and context. Turning such knowledge into explicit rules can therefore require many interviews, examples, tests, and revisions.

10.10.3 Knowledge Validation

After knowledge is encoded, it should be tested. Important questions include:

- Are the rules factually correct?
- Do the rules cover the important cases?
- Are there contradictions between rules?
- Does the system produce reasonable conclusions?
- Can experts understand and validate the encoded knowledge?
- Does a change in one rule create an unexpected effect elsewhere?

Knowledge engineering is therefore iterative. New cases may reveal missing rules, exceptions, or outdated assumptions.

10.11 Expert System Shells

An **expert system shell** is a software environment that provides reusable facilities for building expert systems. Instead of building everything from zero, developers can use an existing inference engine, user interface, explanation facility, and knowledge-acquisition or rule-building tools.

The reference material gives examples such as **EMYCIN**, **CLIPS**, **Jess**, and **Drools**. Prolog can also be used to build expert systems, especially where logical reasoning is central.

10.11.1 Benefits of Shells

- reduce development time;
- reduce implementation cost;
- provide reusable inference mechanisms;
- make knowledge entry and maintenance easier;

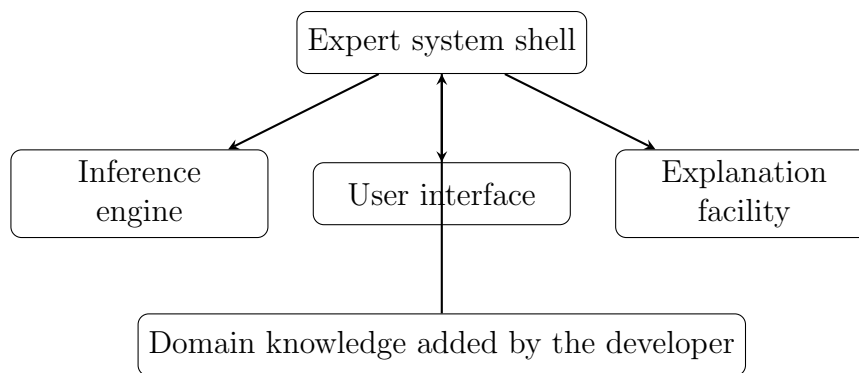


Figure 10.10: Role of an expert-system shell

- support different application domains through different knowledge bases.

10.12 Types of Expert Systems

The provided reference material identifies several broad types.

Table 10.4: Types of expert systems

Type	Main idea
Rule-based	Uses explicit IF–THEN rules for domain reasoning.
Frame-based	Organizes knowledge as frames containing attributes and values.
Fuzzy systems	Represents imprecise knowledge using degrees of truth.
Neural-network-based	Uses learned patterns from neural networks for prediction or decision tasks.
Neuro-fuzzy / hybrid	Combines neural learning with fuzzy handling of uncertainty and imprecision.

The distinction is useful because it shows that the term *expert system* can describe more than one implementation style. Classical rule-based systems are the clearest example for understanding the foundations of expert-system architecture.

10.13 Historical Examples

Several historical expert systems illustrate how the technology was applied to real domains.

10.13.1 MYCIN

MYCIN was an early medical expert system developed at Stanford in the 1970s. The supplied notes describe it as a system intended to help diagnose bacterial infections and recommend

antibiotics from patient symptoms and laboratory information. Its use of confidence levels is an important historical example of handling uncertain expert knowledge.

10.13.2 DENDRAL

DENDRAL was developed for chemistry and used mass-spectrometry data to help identify possible chemical structures.

10.13.3 XCON / R1

XCON, also called R1, was developed for Digital Equipment Corporation to configure computer systems from customer requirements. It is an important example of a business and manufacturing expert system.

10.13.4 PROSPECTOR

PROSPECTOR was developed for mineral exploration and used geological information to estimate the likelihood of mineral deposits.

Table 10.5: Selected historical expert systems

System	Domain	Purpose
MYCIN	Healthcare	Support diagnosis of bacterial infections and antibiotic recommendations.
DENDRAL	Chemistry	Help identify chemical structures from mass-spectrometry information.
XCON / R1	Manufacturing	Configure computer systems from customer requirements.
PROSPECTOR	Geology	Support mineral exploration using geological information.
CLIPS	Expert-system development	Provide a system-building environment for rule-based expert systems.

10.14 Applications of Expert Systems

Expert systems have been used in many domains where specialized knowledge can be represented and applied repeatedly.

- **Healthcare:** diagnosis, treatment support, and monitoring.
- **Finance:** fraud detection, financial analysis, loan-related decisions, and advisory tasks.
- **Manufacturing:** quality control, equipment maintenance, configuration, planning, and scheduling.

- **Agriculture:** crop disease detection and decision support.
- **Customer service:** automated troubleshooting and support.
- **Oil and mineral exploration:** interpretation of geological information.
- **Education and training:** tutoring, simulation, and expert-guided learning.
- **Weather and disaster management:** analysis and decision support.
- **Transportation and logistics:** route planning, fleet management, and supply-chain decisions.
- **Cybersecurity:** analysis of unusual activity and support for threat detection.
- **Power systems:** monitoring, fault detection, load management, and decision support.

The common idea is that the domain contains specialized knowledge that can be encoded and repeatedly applied to similar classes of problems.

10.15 Advantages of Expert Systems

The reference material identifies several advantages:

- **Consistency:** the same encoded rules are applied repeatedly.
- **Speed:** a stored knowledge base can be consulted quickly.
- **Availability:** the system can make expertise available when a human expert is not immediately accessible.
- **Safety:** the system may be used in environments where direct human presence is risky.
- **Reproducibility:** the same input and knowledge can lead to the same encoded conclusion.
- **Explanation:** rule-based reasoning can provide a trace of the reasoning process.

10.16 Limitations of Expert Systems

Expert systems are powerful within their intended domain, but their limitations are equally important.

- **Domain limitation:** an expert system is generally designed for a specific problem area.
- **Knowledge acquisition difficulty:** collecting and encoding expert knowledge can be expensive and time-consuming.

- **Knowledge-base errors:** incorrect or incomplete knowledge can produce incorrect conclusions.
- **Maintenance:** rules and facts may need manual updates as the domain changes.
- **Limited common sense:** a system may not possess the broad everyday knowledge that people use naturally.
- **Lack of creativity:** a classical rule-based system does not automatically create new expert knowledge merely because it can apply existing rules.
- **Development cost:** building, validating, and maintaining a reliable expert system may require substantial effort.

A useful principle is:

The quality of an expert system depends strongly on the quality of its knowledge.

10.17 Expert Systems and Machine Learning

Chapter 7 introduced machine learning, where a model learns patterns from examples. Expert systems represent a different philosophy: domain knowledge is explicitly encoded and then used by an inference mechanism.

Table 10.6: Expert systems and machine learning

Feature	Expert System	Machine Learning
Main source of knowledge	Human experts and structured knowledge	Training data
Knowledge representation	Often explicit rules, facts, frames, or related structures	Often learned model parameters
Reasoning	Rule-based or other explicit inference	Statistical or optimization-based prediction
Updating knowledge	Usually requires explicit maintenance	Can often be retrained with new data
Explanation	Rule paths can be directly inspectable	Explanation may be more difficult for complex models
Best suited to	Stable, specialized domains with expressible expert knowledge	Problems where useful patterns can be learned from data

Neither approach is universally sufficient. The supplied reference material explicitly notes the possibility of **hybrid AI**, in which expert knowledge is combined with machine learning to obtain both explicit domain reasoning and adaptability.

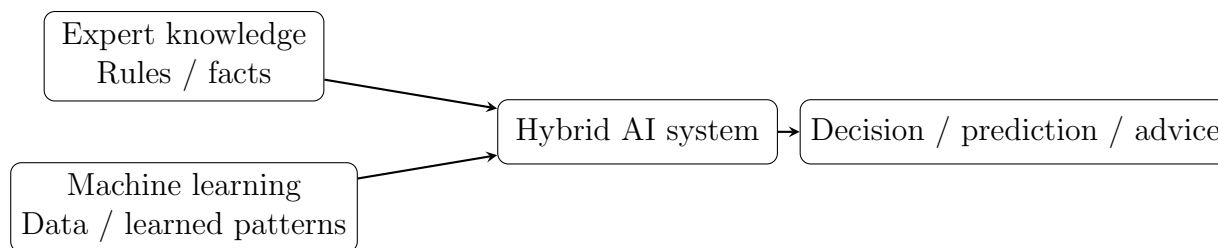


Figure 10.11: Conceptual idea of a hybrid expert system

10.18 A Compact Worked Example

Consider a simple machine-maintenance system.

Facts:

```

HighTemperature(machine1)
NoPower(machine1)
UnusualNoise(machine1)
  
```

Rules:

```

R1: IF HighTemperature THEN CheckCooling
R2: IF NoPower THEN CheckPowerSupply
R3: IF CheckCooling AND UnusualNoise THEN CheckFan
  
```

If the user asks, “What should be checked?” a forward-chaining system can begin with the observed facts and derive actions such as checking the cooling system and power supply. If the user asks, “Why is the fan suspected?” a backward-chaining system can start with the goal `CheckFan` and work backward to the conditions that support it.

This small example captures the central idea of an expert system: **knowledge is explicit, reasoning is systematic, and the result is connected to a domain-specific problem.**

10.19 Chapter Summary

- An expert system is a specialized AI system that uses domain knowledge and reasoning to solve problems or provide advice.
- Expert systems are a type of knowledge-based system, but knowledge-based systems are broader than expert systems.
- The main architecture contains a knowledge base and an inference engine, together with user interaction and, in richer systems, working memory, explanation, and knowledge-acquisition facilities.
- The knowledge base stores facts, rules, and other domain representations.

- An IF–THEN rule contains conditions (antecedents) and a conclusion (consequent).
- Forward chaining reasons from facts toward conclusions.
- Backward chaining reasons from a goal toward the facts or conditions that can support it.
- Expert systems can handle uncertainty using probabilistic or fuzzy approaches, depending on the design.
- Knowledge acquisition converts human or documentary expertise into machine-usable representations.
- Knowledge engineering includes representation, validation, testing, and maintenance of the knowledge base.
- Expert-system shells provide reusable infrastructure such as inference engines, explanation facilities, and rule-building tools.
- Historical examples include MYCIN, DENDRAL, XCON/R1, and PROSPECTOR.
- Expert systems are useful in healthcare, finance, manufacturing, agriculture, logistics, education, cybersecurity, and other specialized domains.
- Their major limitations include domain dependence, knowledge-acquisition difficulty, maintenance cost, and limited common sense.
- Modern AI can combine expert knowledge and machine learning in hybrid systems.

10.20 Key Points to Remember

1. The basic idea is: **knowledge base + inference engine + user interaction.**
2. The knowledge base stores domain knowledge; the inference engine uses it.
3. Facts describe known information; rules describe how conditions lead to conclusions.
4. The antecedent is the condition part of a rule; the consequent is the conclusion part.
5. Forward chaining is data-driven.
6. Backward chaining is goal-driven.
7. Deterministic reasoning treats applicable conclusions as true according to the rules.
8. Probabilistic reasoning represents uncertain conclusions with probabilities.

9. Fuzzy reasoning represents imprecise concepts using degrees rather than only true/false values.
10. Knowledge acquisition is often a major bottleneck in expert-system development.
11. Expert-system shells provide reusable tools for building expert systems.
12. Explanation facilities are important when users need to understand why a system reached a conclusion.
13. Classical expert systems are domain-specific; they should not be confused with general intelligence.
14. Hybrid systems can combine explicit expert knowledge with learned patterns.

10.21 Short Questions

1. What is an expert system?
2. Define a knowledge-based system.
3. What is the relationship between AI and expert systems?
4. What are the main goals of an expert system?
5. List the major characteristics of expert systems.
6. What is a knowledge base?
7. What is an inference engine?
8. What is working memory?
9. What is the role of a user interface in an expert system?
10. What is an explanation facility?
11. Define a rule-based expert system.
12. What is an antecedent?
13. What is a consequent?
14. What is forward chaining?
15. What is backward chaining?
16. Differentiate between forward and backward chaining.

17. What is deterministic inference?
18. What is probabilistic inference?
19. What is fuzzy reasoning?
20. What is a confidence factor in a rule?
21. What is knowledge acquisition?
22. What is the knowledge acquisition bottleneck?
23. Who is a knowledge engineer?
24. What is an expert-system shell?
25. Mention any four expert-system shells or development tools.
26. Name any four historical expert systems.
27. Mention four applications of expert systems.
28. State four advantages of expert systems.
29. State four limitations of expert systems.
30. Differentiate between an expert system and machine learning.

10.22 Long Questions

1. Define an expert system. Explain its goals, characteristics, and relationship with AI and knowledge-based systems.
2. Explain the architecture of an expert system with a neat diagram. Discuss the role of the knowledge base, inference engine, working memory, user interface, and explanation facility.
3. Explain knowledge representation in expert systems using facts, rules, frames, and logical representations.
4. Explain the structure of an IF–THEN production rule. Define antecedent, consequent, and optional confidence factor with examples.
5. Explain forward chaining with a suitable example and diagram.
6. Explain backward chaining with a suitable example and diagram.
7. Compare forward chaining and backward chaining.

8. Explain deterministic and probabilistic inference in expert systems.
9. What is uncertainty? Explain the sources of uncertainty and the role of probabilistic reasoning.
10. Explain fuzzy reasoning in expert systems. Why is fuzzy reasoning useful for imprecise knowledge?
11. What is knowledge acquisition? Explain the knowledge acquisition bottleneck and the role of a knowledge engineer.
12. Explain the knowledge-engineering cycle from knowledge acquisition to representation, validation, testing, and maintenance.
13. What is an expert-system shell? Explain its major facilities and advantages.
14. Discuss rule-based, frame-based, fuzzy, neural-network-based, and neuro-fuzzy expert systems.
15. Explain MYCIN, DENDRAL, XCON/R1, and PROSPECTOR as historical examples of expert systems.
16. Discuss applications of expert systems in healthcare, finance, manufacturing, agriculture, education, logistics, and cybersecurity.
17. Explain the advantages and limitations of expert systems.
18. Compare expert systems and machine learning. Discuss the idea of hybrid AI systems.
19. Explain the complete working of an expert system using a small diagnosis or equipment-maintenance example.
20. Why is the quality of knowledge important in an expert system? Discuss knowledge-base errors, maintenance, and domain limitations.

Chapter 11

Philosophy, Ethics and Future of AI

11.1 Introduction

If machines become more capable, an important question is not only what AI can do, but what it should do.

Artificial Intelligence is not only a technical subject. It also raises questions about intelligence, reasoning, responsibility, safety, human values, and the future relationship between people and machines. A system may be technically powerful and still create serious problems if it is unfair, difficult to understand, unsafe, or used for harmful purposes.

This final chapter connects the technical ideas studied throughout this book with the wider questions surrounding AI. The chapter follows the themes in the supplied course material: **philosophy of AI, the limits of AI, whether machines can really think, ethics and safety, and the future of AI**. The reference material also discusses AI components, AI architectures, General AI, large language models, and Generative AI as future-facing directions.

A useful sentence to remember is:

AI asks what machines can do; AI philosophy asks what intelligence means; AI ethics asks what machines should do.

11.2 Philosophy of Artificial Intelligence

Philosophy of AI studies questions such as: What is intelligence? Can a machine think? Is intelligent behavior the same as having a mind? Can consciousness be created artificially? What should count as evidence that a machine is intelligent?

The supplied material presents two useful distinctions. **Weak AI** refers to the idea that machines can behave intelligently, while **strong AI** refers to the much stronger hypothesis that a machine could possess an actual mind rather than merely simulate intelligent behav-

ior. These are philosophical positions, not names for particular programming languages or software products.

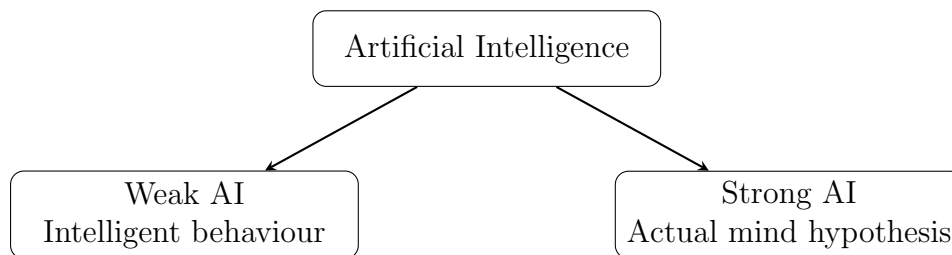


Figure 11.1: Weak AI and strong AI as philosophical positions

11.2.1 Four Approaches to AI

A classic way to understand AI is to divide its goals into four approaches: **acting humanly**, **thinking humanly**, **thinking rationally**, and **acting rationally**. The first two focus on human-like behaviour or thought. The other two focus on rational reasoning or action.

Thinking Humanly Cognitive modelling	Thinking Rationally Laws of thought
Acting Humanly Turing Test approach	Acting Rationally Rational-agent approach

Figure 11.2: Four philosophical approaches to Artificial Intelligence

11.2.2 Thinking Humanly

The thinking-humanly approach asks whether a machine can reproduce, or at least model, the processes used by human beings when they solve problems. This approach is related to cognitive science and attempts to understand the mechanisms of human thought.

For example, if people solve a puzzle by remembering earlier experiences, forming a hypothesis, testing it, and correcting errors, a cognitive model may try to reproduce a similar sequence of operations.

The important question is not only whether the final answer is correct, but also whether the internal process resembles human thinking.

11.2.3 Thinking Rationally

The thinking-rationally approach is associated with the “laws of thought” idea. It asks whether a system can derive correct conclusions from appropriate premises using logical reasoning.

For example:

All humans are mortal.
Socrates is a human.
Therefore, Socrates is mortal.

This approach is closely related to the knowledge representation and logical reasoning topics studied earlier in this book.

11.2.4 Acting Humanly

The acting-humanly approach asks whether a machine can behave in a way that is difficult to distinguish from human behaviour. The most famous example is the **Turing Test**, proposed by Alan Turing in 1950.

11.3 Can Machines Really Think?

The question “Can machines think?” is more difficult than it appears because both words — *machine* and *think* — can be interpreted in different ways.

A computer can perform calculations, classify images, translate languages, play games, generate text, and control physical systems. But these abilities do not automatically settle philosophical questions about consciousness, subjective experience, or whether a machine has a mind.

The supplied lecture notes emphasize that Turing replaced the abstract question with a behavioural test. In other words, instead of trying to define “thinking” directly, we can examine what a machine can do in interaction with people.

11.3.1 The Turing Test

In the classic imitation-game formulation, a human evaluator communicates with a hidden participant through text. If the evaluator cannot reliably distinguish whether the responses come from a human or a machine, the machine is said to have performed successfully in the test.

The Turing Test is a test of observable behaviour. It does not by itself prove the existence of consciousness or subjective experience. The supplied material explicitly notes that

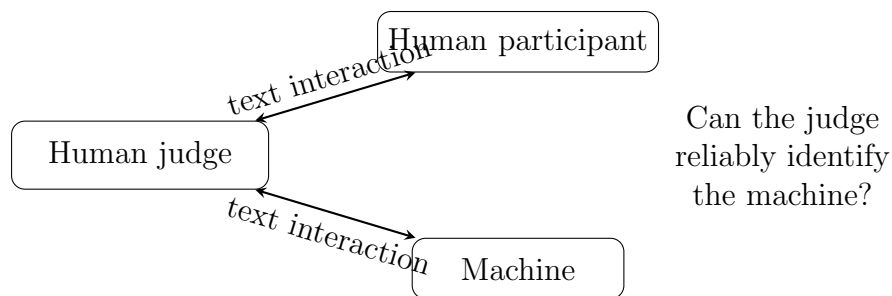


Figure 11.3: Conceptual view of the Turing Test

consciousness remains an open mystery.

11.3.2 The Difference Between Performance and Consciousness

A system may produce highly convincing language without providing evidence that it has subjective experience. This distinction is important:

Table 11.1: Behavioural performance and philosophical questions

Aspect	Central question
Behaviour	Can the system perform the task successfully?
Reasoning	Can the system derive or learn useful conclusions?
Understanding	Does the system represent or use meaningful structure?
Consciousness	Does the system have subjective experience?
Self-awareness	Does the system have an internal model of itself?

The first three questions can be studied experimentally. The last two move into difficult philosophical questions that are not settled simply by improving benchmark scores.

11.4 Limits of Artificial Intelligence

AI systems can be extremely strong at particular tasks while still being limited in other ways. A chess system may be excellent at chess without being able to perform ordinary household tasks. A language model may generate fluent text without having the broad physical experience of a human being.

The reference material uses this difference to explain why modern AI has often been described as **narrow or domain-specific AI**. A system may achieve human-level or superhuman performance in a restricted domain while lacking generality across unrelated tasks.

11.4.1 Narrow Intelligence and General Intelligence

Artificial General Intelligence (AGI) is used to describe a hypothetical or future form of AI with much broader competence across different domains rather than expertise in only

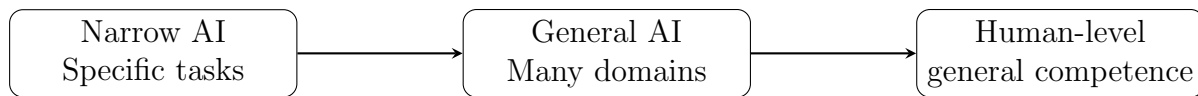


Figure 11.4: A conceptual continuum from task-specific to more general intelligence

one carefully defined task.

It is important not to confuse the concept of AGI with a guaranteed future technology. The supplied material presents general intelligence as an emerging concept and emphasizes that safety becomes much harder when a system operates across many novel contexts.

11.4.2 Common Limitations of Current AI

The limitations highlighted across the supplied materials can be organized as follows:

- **Domain limitation:** a model may perform well only on tasks represented in its training or design.
- **Dependence on data:** performance can depend strongly on the quantity, quality, and coverage of training data.
- **Lack of common sense:** successful pattern recognition does not automatically provide human-like everyday knowledge.
- **Explainability limitation:** some complex models are difficult to inspect or explain in simple terms.
- **Robustness limitation:** systems can fail when conditions differ from the situations represented during development.
- **Context limitation:** a system may produce a useful answer without possessing the broad context in which humans normally interpret a situation.
- **Value limitation:** a technically optimized system does not automatically know which human goals are desirable.

11.5 Why AI Ethics Matters

The supplied ethics material argues that when AI systems take over cognitive or social functions previously performed by humans, ordinary social requirements become design requirements. A decision system that affects a person's loan, employment, safety, access to services, or reputation should not be judged only by accuracy.

A useful way to summarize this is:

**Good AI = capability + safety + fairness + accountability + transparency +
respect for human values**

11.6 Ethical Principles for AI

The supplied AI philosophy notes identify several principles for responsible AI development and use.

11.6.1 Safety

AI systems should be designed so that foreseeable failures do not cause unacceptable harm. Safety becomes especially important when AI can control physical systems, make high-impact decisions, or operate with limited human supervision.

11.6.2 Fairness

AI systems can reproduce or amplify patterns present in data. Fairness therefore requires asking what counts as fair in the particular application, whose interests are affected, and whether the system produces unjustified differences in treatment.

The supplied course material emphasizes that there are multiple concepts of fairness and that they cannot always be maximized simultaneously. Therefore, fairness is not simply a single mathematical number; it is also a design and policy decision.

11.6.3 Privacy

AI systems often process large amounts of personal information. Privacy concerns include unnecessary data collection, inappropriate sharing, weak access controls, inference of sensitive information, and the long-term retention of data.

11.6.4 Transparency and Explainability

A user affected by an automated decision may reasonably want to know why the decision occurred. The supplied material introduces **Explainable AI (XAI)** as an approach in which an AI system can provide meaningful explanations of its outputs.

A good explanation should, according to the supplied material, be understandable and convincing to the user, accurately reflect the reasoning of the system, be sufficiently complete, and be specific to the situation and outcome.

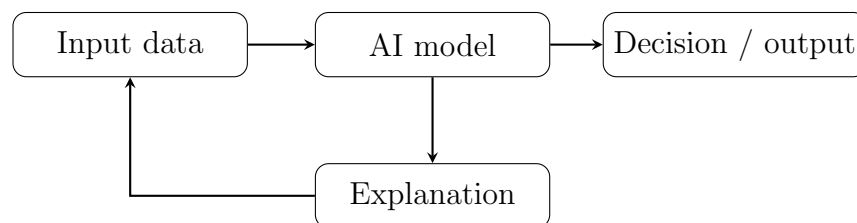


Figure 11.5: Conceptual role of Explainable AI

11.6.5 Accountability and Responsibility

When an AI-assisted decision causes harm, responsibility should not disappear simply because a machine was involved. The supplied ethics reference highlights the importance of being able to identify who is responsible for a system's operation, maintenance, deployment, and decisions.

In a practical AI project, responsibility may be distributed across designers, data providers, system owners, deployers, operators, and institutions. Clear roles are therefore part of responsible system design.

11.6.6 Human Rights and Human Values

AI systems should operate within human rights and values rather than treating people only as data points. The supplied material also highlights diversity and inclusion, legal and policy implications, employment, and the danger of excessive concentration of power.

11.6.7 Limiting Harmful Uses

AI can be used for beneficial applications, but the same technical capability may also support harmful activities. The supplied lecture material mentions risks including security and privacy breaches, unintended side effects, errors, malicious misuse, and lethal autonomous weapons.

11.7 Bias and Fairness in AI

Bias can enter an AI system at many stages: problem formulation, data collection, labelling, feature selection, model design, evaluation, deployment, and feedback.

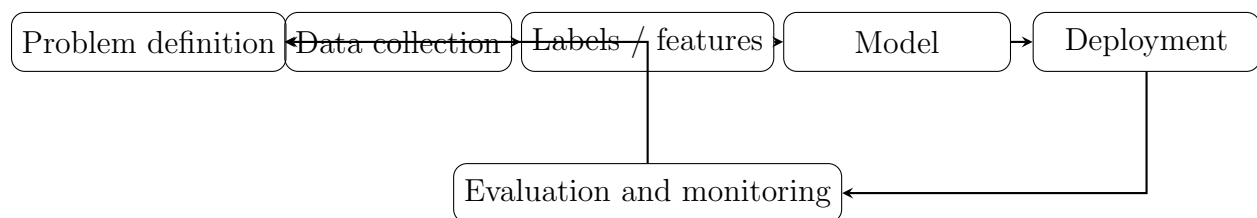


Figure 11.6: Bias can enter at multiple stages of an AI lifecycle

A simple example is a decision system that excludes a sensitive attribute but still uses another variable strongly correlated with it. Hiding one field from the model does not automatically guarantee fairness.

11.8 Safety and Robustness of AI

Safety is broader than simply making a system accurate. A reliable AI system should also behave safely when inputs are unusual, incomplete, manipulated, or outside the conditions expected during development.

The ethics reference highlights **robustness against manipulation** as an important requirement. Security is therefore closely connected to AI ethics when attackers can deliberately search for weaknesses in an AI system.

11.8.1 A Practical Safety View

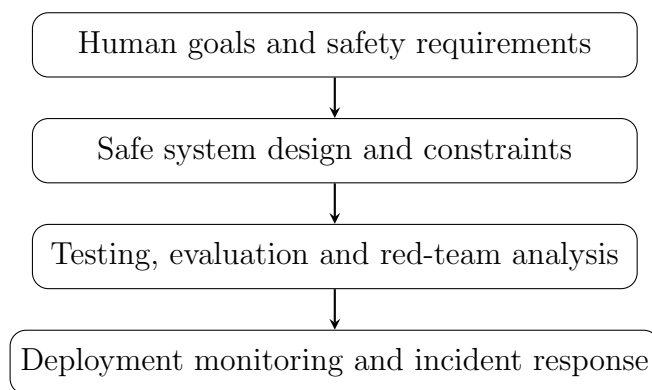


Figure 11.7: A layered view of AI safety

No single technique is enough for every AI application. Safety is better understood as a lifecycle activity that continues from problem definition through deployment and monitoring.

11.9 Asimov’s Laws of Robotics as a Thought Experiment

Isaac Asimov introduced fictional laws of robotics that have become a popular way to discuss machine behaviour and human safety. The supplied lecture material lists the familiar three laws in which protection of humans has the highest priority, followed by obedience to human orders and protection of the robot itself.

These laws should be understood as a **fictional thought experiment**, not as an engineering standard or a complete solution to AI ethics. Real systems must deal with ambiguity, conflicting values, incomplete information, accountability, and real-world uncertainty.

11.10 Machines with Moral Status

The supplied ethics paper introduces another deep question: if future AI systems were conscious or possessed morally relevant characteristics, would humans have moral obligations

toward those systems?

A useful term here is **moral status**. A being has moral status when its own interests count morally; it should not be treated only as a tool for someone else.

The paper discusses two candidate characteristics:

- **Sentience:** the capacity for subjective experience, such as feeling pain or suffering.
- **Sapience:** higher cognitive capacities associated with self-awareness and reason-responsive agency.

The supplied source states that current AI systems are generally treated as lacking moral status, while the status of possible future artificial minds is uncertain. This is a philosophical issue rather than an established fact about future technology.

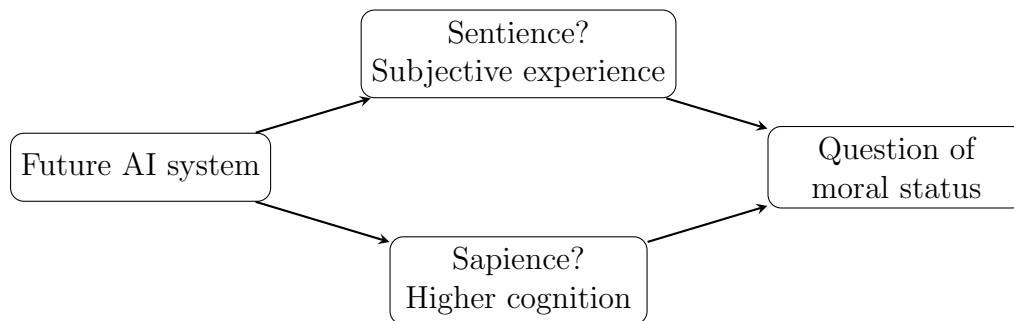


Figure 11.8: Philosophical questions surrounding possible machine moral status

The purpose of this discussion is not to claim that present systems are conscious. It is to show why future AI may create questions that do not arise for ordinary software.

11.11 Artificial General Intelligence and AI Safety

The ethics reference argues that safety becomes qualitatively more difficult for systems that operate across many novel contexts. A domain-specific system can often be tested against a relatively clear collection of expected situations. A more general system may encounter situations that designers did not explicitly imagine.

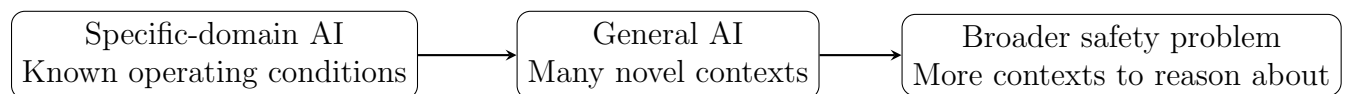


Figure 11.9: Why broader AI capability can create broader safety requirements

The supplied Bostrom and Yudkowsky material emphasizes a distinction between verifying a limited system and verifying a system that may reason about distant consequences in situations not explicitly envisioned by the engineers. The central lesson for students is simple: **greater generality can make safety assurance harder, not merely easier.**

11.12 The Future of Artificial Intelligence

The exact future of AI cannot be stated with certainty. The supplied course material presents future developments as possible directions rather than guaranteed outcomes.

Important directions discussed in the reference material include:

- improved natural language processing;
- greater ability to learn and adapt;
- wider integration with the Internet of Things;
- continued progress in machine learning;
- increasing use of AI in healthcare, transportation, and other sectors;
- stronger ethical and regulatory frameworks;
- more capable and more general-purpose AI systems;
- continued development of large transformer-based models and Generative AI.

11.12.1 AI Components

The supplied future-of-AI material identifies several components that an intelligent system may need to perform useful work:

- sensors for observing the environment;
- actuators or other mechanisms for taking actions;
- representations of the state of the world;
- decision mechanisms for selecting actions;
- goals or preferences defining what the agent is trying to achieve;
- learning mechanisms;
- computational resources, shared data, and shared models.

11.12.2 Real-Time AI and Bounded Decision-Making

The supplied material notes that increasingly complex AI systems may operate under time limits. An intelligent agent often cannot spend unlimited time computing the mathematically perfect action. It may instead need to make the best decision available within the time and computational resources it has.

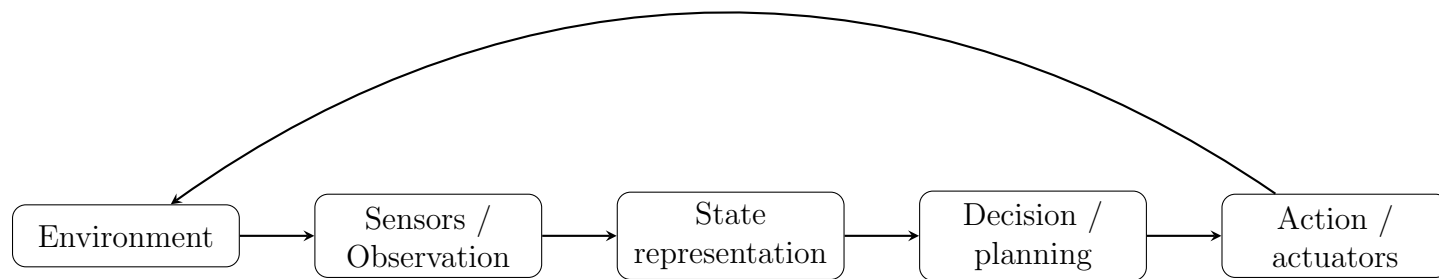


Figure 11.10: A general intelligent-agent loop

This idea connects AI theory with practical engineering: intelligent behaviour must work not only in principle, but also under real-world resource limits.

11.13 AI Architectures

An AI architecture describes how perception, internal state, reasoning, planning, learning, and action are organized in a system. The supplied course material emphasizes that practical systems may combine several architectural ideas rather than choosing one simple architecture.

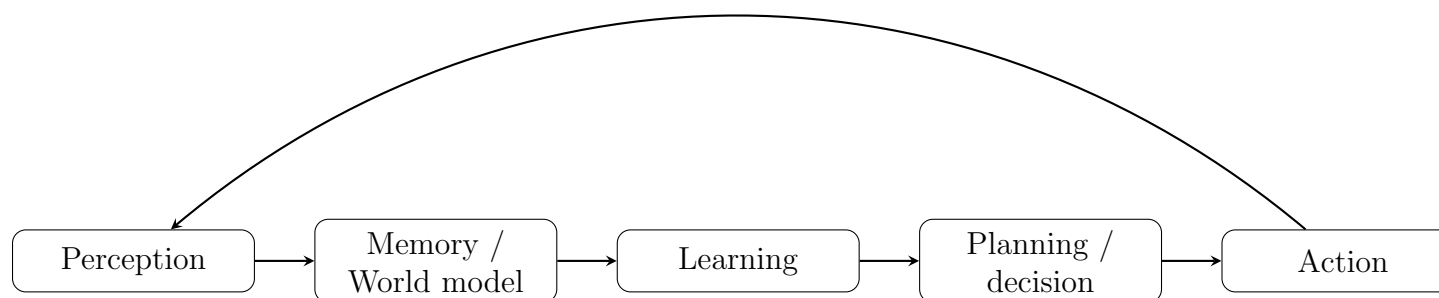


Figure 11.11: A conceptual architecture combining perception, memory, learning, planning, and action

The phrase **agent = architecture + program** is useful for understanding this idea. The architecture supplies the computational organization and interfaces; the program or learned model determines how the system behaves inside that organization.

11.14 From Narrow AI to More General Systems

A major theme in the supplied future-of-AI notes is a shift from building a separate system for every task toward using broad, reusable models.

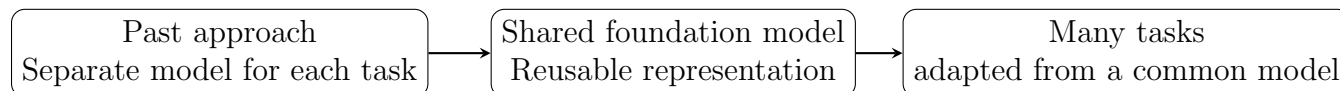


Figure 11.12: Conceptual transition from task-specific systems to reusable AI models

The supplied notes use large transformer language models as examples of this trend. Models such as BERT, GPT-series systems, PaLM, BLOOM, and ChatGPT illustrate the growth of large-scale language modelling in the source material. The important conceptual lesson is the movement toward models that can be adapted for many tasks rather than built separately from scratch.

11.15 Generative AI

Generative AI refers to AI systems designed to produce new content such as text, images, audio, code, or other media. It is an important modern development because the system is not limited to choosing one label from a fixed set; it can construct a new output based on a prompt or other input.

Examples of generation tasks include:

- text generation and summarization;
- question answering and dialogue;
- code generation and documentation;
- image generation and editing;
- speech and audio generation;
- music generation;
- video generation.

The supplied future-of-AI material shows the emergence of transformer-based language models and Generative AI as important developments. It also presents ChatGPT as an example of a conversational language model.

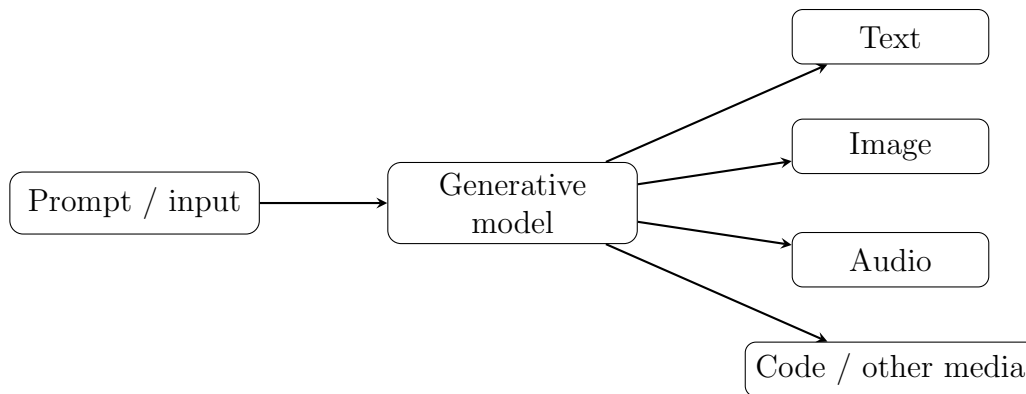


Figure 11.13: Conceptual view of Generative AI

11.16 The Future of AI: Opportunities and Challenges

Future AI systems may create significant opportunities while also creating difficult questions. It is useful to examine both sides.

Table 11.2: Possible opportunities and challenges of future AI

Possible opportunities	Possible challenges
Better decision support	Bias and unfair decisions
More accessible expertise	Privacy risks
Automation of repetitive work	Employment and job redesign
Improved healthcare and scientific tools	Safety and reliability
More natural human-computer interaction	Misuse and security threats
New creative and educational tools	Accountability for high-impact decisions
Broader access to information	Concentration of technological power

The future will therefore depend not only on technical progress, but also on how individuals, organizations, governments, and societies choose to develop and use AI. This point is explicitly emphasized in the supplied future-of-AI material.

11.17 AI and Employment

Automation can change the nature of work. Some tasks may be automated, some jobs may be redesigned, and new roles may emerge around AI development, deployment, evaluation, governance, and human oversight.

It is therefore more useful to think about AI and employment in terms of **tasks** rather than assuming that an entire occupation will simply disappear or remain unchanged. A

human worker may use AI as a tool while continuing to provide judgement, communication, responsibility, domain expertise, or physical interaction.

11.18 Human-AI Collaboration

An important future direction is not necessarily “humans versus machines.” Many useful systems are designed as collaborative systems in which humans and AI perform complementary tasks.

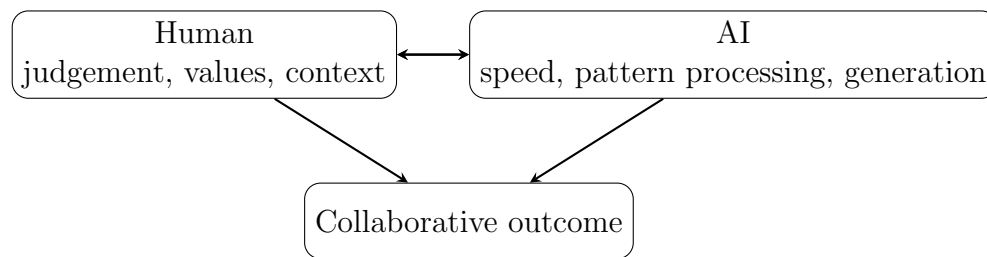


Figure 11.14: Human-AI collaboration as a possible design pattern

Good collaboration requires clear responsibility, good interfaces, appropriate human oversight, and an understanding of where the AI system is reliable and where it is not.

11.19 A Balanced View of the Future

AI progress should not be described only as a story of either perfect machines or catastrophic failure. A more useful academic approach is to ask concrete questions:

1. What task is the system designed to perform?
2. What evidence shows that the system performs the task reliably?
3. What are the failure modes?
4. Who may be affected by those failures?
5. Can users understand important decisions?
6. Who is responsible for the system?
7. What data does the system need and how is that data protected?
8. How can the system be tested, monitored, and corrected after deployment?

These questions turn the discussion of “future AI” into an engineering and social responsibility problem that can be studied systematically.

11.20 Chapter Summary

Artificial Intelligence is both a technical discipline and a subject of philosophical and ethical inquiry. Philosophy asks what intelligence, thinking, consciousness, and rationality mean. The four classic approaches to AI distinguish between thinking humanly, thinking rationally, acting humanly, and acting rationally.

The Turing Test provides a behavioural approach to the question of machine intelligence, but successful behaviour does not by itself settle the philosophical question of consciousness. Current AI systems can be highly capable in specific tasks while still having important limitations related to generality, data, context, robustness, explanation, and value alignment.

AI ethics adds practical requirements such as safety, fairness, privacy, transparency, explainability, accountability, respect for human rights, diversity and inclusion, and limits on harmful use. The ethics reference also raises deeper future questions concerning Artificial General Intelligence and the possible moral status of artificial minds.

The future of AI may include more capable learning systems, broader AI architectures, real-time intelligent agents, large reusable models, Generative AI, and deeper integration with physical and digital environments. These developments create both opportunities and challenges. The future therefore depends not only on what engineers can build, but also on how responsibly AI is designed, deployed, governed, and used.

11.21 Key Points to Remember

- **Weak AI** concerns intelligent behaviour; **strong AI** is the stronger hypothesis of actual machine minds.
- The four classic approaches are **thinking humanly, thinking rationally, acting humanly, and acting rationally**.
- The **Turing Test** evaluates machine behaviour through interaction rather than directly measuring consciousness.
- A machine can be highly capable in a narrow domain without possessing general intelligence.
- AI ethics includes **safety, fairness, privacy, transparency, explainability, accountability, human rights, and responsible use**.
- **XAI** attempts to make important AI decisions understandable to users.
- AI safety includes robustness, testing, monitoring, and clear responsibility.
- Future AI may include more general systems, large reusable models, real-time agents, and Generative AI.

- The future of AI depends on both technical development and responsible human choices.

11.22 Short Questions

11.22.1 1. What is the philosophy of AI?

The philosophy of AI studies questions about intelligence, thinking, reasoning, consciousness, and the relationship between human minds and artificial systems.

11.22.2 2. What is weak AI?

Weak AI is the hypothesis that machines can behave intelligently even if this behaviour does not imply that the machine has an actual human-like mind.

11.22.3 3. What is strong AI?

Strong AI is the stronger philosophical hypothesis that an artificial system could possess an actual mind rather than merely simulate intelligent behaviour.

11.22.4 4. What are the four approaches to AI?

Thinking humanly, thinking rationally, acting humanly, and acting rationally.

11.22.5 5. What is the Turing Test?

It is a behavioural test proposed by Alan Turing in which a human evaluator communicates with hidden participants and attempts to determine whether one of them is a machine.

11.22.6 6. What is AGI?

Artificial General Intelligence refers to the concept of an AI system with broad competence across many domains rather than expertise restricted to one task.

11.22.7 7. What is Explainable AI?

Explainable AI refers to techniques and system designs that provide understandable explanations of important AI outputs or decisions.

11.22.8 8. Mention four ethical principles of AI.

Safety, fairness, privacy, transparency, accountability, human rights, and limiting harmful uses are examples.

11.22.9 9. What is moral status?

Moral status is the idea that a being's own interests count morally and that it should be considered for its own sake.

11.22.10 10. What is Generative AI?

Generative AI is AI designed to create new content such as text, images, audio, code, or other forms of media.

11.23 Long Questions

1. Explain the philosophy of AI and discuss the four approaches to AI.
2. Explain the Turing Test and discuss the difference between behavioural intelligence and consciousness.
3. What are the major limitations of present-day AI systems? Explain with examples.
4. Explain the major ethical principles that should guide the design and use of AI systems.
5. Discuss fairness, privacy, transparency, explainability, and accountability in AI.
6. Explain the importance of AI safety and robustness. Why do these become more difficult for general AI systems?
7. What is Artificial General Intelligence? Discuss its possible opportunities and safety challenges.
8. Explain the concept of moral status in relation to possible future artificial minds.
9. Discuss major future directions of AI, including large reusable models and Generative AI.
10. Explain the role of human-AI collaboration in the future of intelligent systems.

11.24 Discussion Questions

1. If a machine behaves exactly like a human in conversation, does that prove that it thinks? Give arguments for and against.
2. Should an AI system explain every decision, or only high-impact decisions? Discuss.
3. Can an AI system be accurate but still unfair? Explain with a suitable example.
4. Who should be responsible when an AI-assisted decision causes harm: the developer, the organization, the operator, or all of them?

5. Will future AI mainly replace human workers, assist them, or change the tasks they perform? Discuss using evidence rather than assumptions.

11.25 Selected References for This Chapter

1. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2020.
2. Nick Bostrom and Eliezer Yudkowsky, “The Ethics of Artificial Intelligence,” in the *Cambridge Handbook of Artificial Intelligence*.
3. Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*, 2nd Edition, 2018.
4. Lewis Tunstall, Leandro von Werra, and Thomas Wolf, *Natural Language Processing with Transformers*, O’Reilly Media, 2022.

End of Chapter 11

Chapter A

Prolog Programs

Chapter B

LISP Programs

Chapter C

Slot and Filler Structures (Supplementary)

Chapter D

University Question Bank